Humna Imran

DATA MINING LAB MANUAL

2023
2027

Instructor : Muhammad Arham
Roll Number: 2023-BS-AI-017
Degree: BS Artificial Intelligence

TABLE OF CONTENT

LAB NO 1: INTRODUCTION

In the contemporary retail landscape, organizations generate immense volumes of transactional data daily. Every purchase, customer interaction, and inventory adjustment contributes to a vast repository of raw information. However, without systematic analysis, this data remains an underutilized asset rather than a strategic advantage.

Data mining bridges the gap between raw data collection and actionable business intelligence. By applying analytical techniques to historical sales records, retailers can uncover hidden patterns, quantify operational success, and make informed, data-driven decisions. In this laboratory exercise, students will transition from basic data observation to strategic analysis by extracting critical Key Performance Indicators (KPIs) from a retail dataset. Understanding metrics such as Total Revenue, Average Order Value, Category Revenue, and Top Products provides retail management with the empirical evidence required to optimize inventory, refine pricing strategies, and maximize overall profitability.

To achieve these analytical objectives, this lab utilizes the KNIME Analytics Platform. KNIME provides a visual, node-based environment that allows analysts to construct robust data processing pipelines efficiently. By visually mapping the flow of data from initial ingestion to final metric calculation, students will develop a clear, reproducible methodology for extracting actionable insights from large-scale datasets.

Figure 1: Data Mining Process Flow

2. PROBLEM STATEMENT

A mid-sized retail enterprise has accumulated an extensive dataset detailing thousands of individual customer transactions. While this raw repository captures the granular details of every sale, the management team currently lacks the analytical infrastructure to evaluate their overall operational performance. They are inundated with unstructured records and require a systematic, automated approach to transform this ledger into a strategic dashboard.

To make informed commercial decisions, the organization must move beyond mere data storage. Management urgently needs to quantify their total financial yield, determine the typical value of a customer's purchase, assess which merchandise groupings are the most lucrative, and pinpoint their top-performing inventory. The core challenge is establishing a reliable data processing workflow that can ingest this bulk transactional data and output these vital business metrics with accuracy and efficiency.

3. DATASET DESCRIPTION

For this laboratory exercise, students will utilize the **Retail Sales Dataset**, provided as a Comma-Separated Values (CSV) file. This dataset serves as a transactional ledger, where each individual row corresponds to a single product line item within a broader customer purchase.

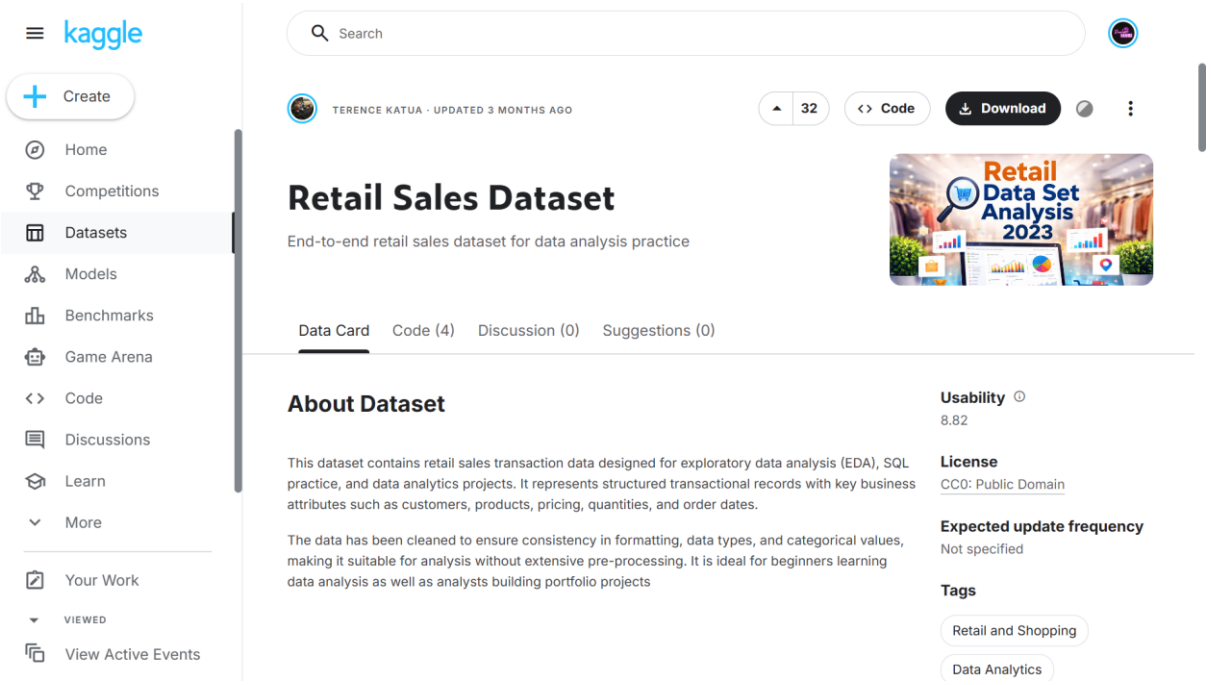


Figure 2: Kaggle Retail Sales Dataset

To successfully perform the required data mining tasks and calculate the target business metrics, students must familiarize themselves with the following foundational attributes within the dataset:

- **Transaction ID:** A unique integer value assigned to each distinct customer transaction. This column is essential for tracking individual purchases and understanding full order profiles.
- **Quantity:** An integer value representing the total number of individual units purchased for a specific product within a transaction.
- **Product Category:** A categorical string variable that classifies the purchased item into broader retail departments (e.g., Beauty, Clothing, Electronics). This is utilized for segmenting sales performance.
- **Price per Unit:** A numeric (integer) value indicating the cost of a single unit of the corresponding product.

Note: While this specific dataset already provides a 'Total Amount' column, for the purpose of learning feature engineering in KNIME, students will be required to mathematically derive a new 'Revenue' feature using the 'Price per Unit' and 'Quantity' attributes during the data preparation phase. This derived column will serve as the basis for calculating the final KPIs.

4. TOOLS USED

This laboratory session relies exclusively on the **KNIME Analytics Platform** to execute all data ingestion, manipulation, and metric calculation tasks.

KNIME is an advanced, open-source data analytics and integration environment that operates on a visual programming paradigm. Instead of writing traditional code, analysts construct complete data pipelines, referred to as "workflows", by linking individual functional units known as "nodes." Each node is responsible for executing a specific, atomic operation, such as reading a file, grouping records, or applying a mathematical equation.

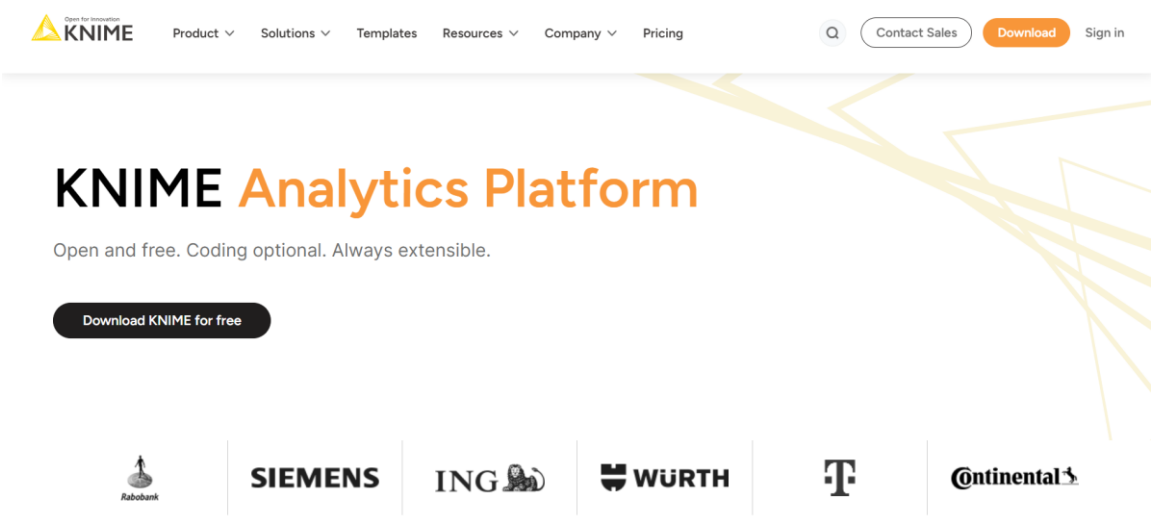


Figure 4: KNIME Analytics Platform Website

This platform is specifically selected for this retail analytics exercise due to the following advantages:

- **Transparent Data Lineage:** The graphical interface allows users to visually track the step-by-step transformation of data, from the initial raw CSV ingestion to the final aggregated metric, ensuring complete transparency and reproducibility.
- **Focus on Analytical Logic:** By removing the syntax requirements of traditional programming languages, the node-based environment allows students to focus entirely on the mathematical logic and business strategy behind the data mining process.
- **Granular Configuration:** Every node features a dedicated configuration dialog, providing precise control over data types, aggregation methods, and sorting criteria without the risk of compiling errors.

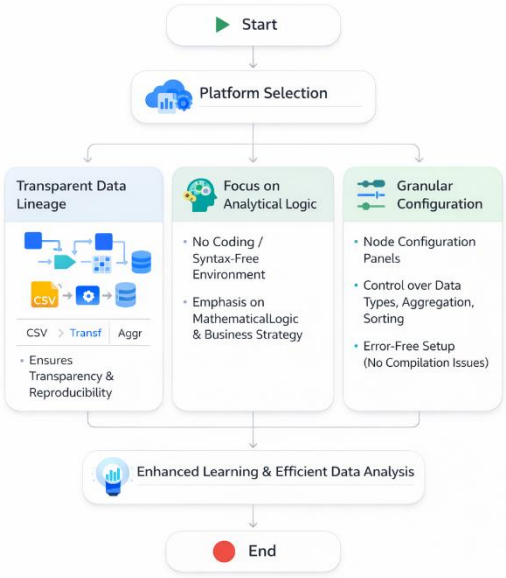


Figure 3: Platform Selection Advantages

5. PRACTICAL OBJECTIVES

Upon successful completion of this laboratory session, students will demonstrate practical competency in visual data mining by achieving the following learning outcomes:

1. **Data Ingestion:** Successfully import raw transactional retail data into the KNIME workspace utilizing the **CSV Reader** node.
2. **Type Validation and Transformation:** Evaluate and properly configure dataset column types (e.g., ensuring financial values are formatted as Doubles and counts as Integers) to prepare the data for accurate mathematical operations.
3. **Statistical Profiling:** Generate a comprehensive descriptive summary of the dataset, including mean, minimum, maximum, and standard deviation values, by deploying the **Statistics** node to understand overall data distribution.

4. **Feature Engineering:** Construct a novel 'Revenue' attribute for each transaction by executing a mathematical multiplication of the 'Quantity' and 'Price' columns using the **Math Formula** node.
5. **Metric Aggregation:** Formulate high-level business Key Performance Indicators (KPIs) by strategically linking **GroupBy** and **Sorter** nodes to aggregate, summarize, and rank transactional records.

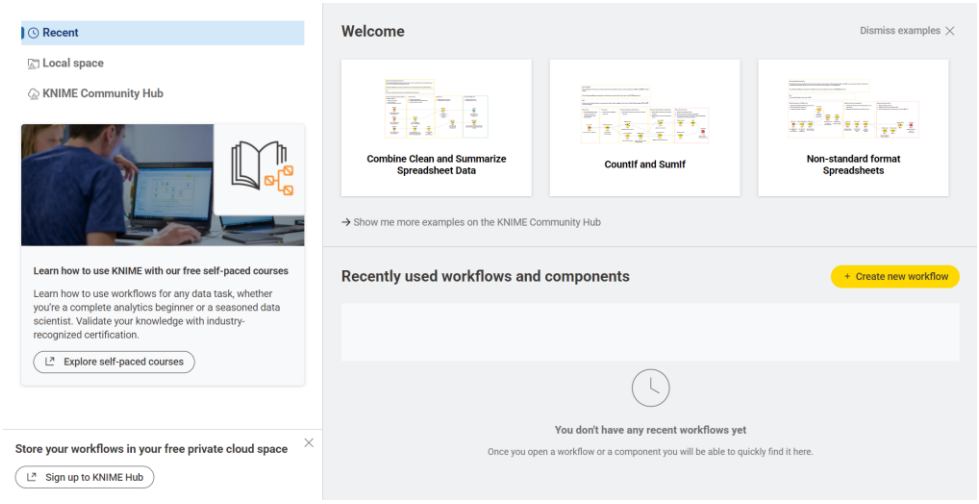


Figure 5: KNIME Welcome Screen

6. STEP-BY-STEP PROCEDURE

STEP 1: WORKSPACE INITIALIZATION

Launch the KNIME Analytics Platform to begin building the data pipeline. Navigate to the primary menu, select "File," proceed to "New," and choose "New KNIME Workflow". Assign a descriptive title to the project, such as "Retail Sales Analysis," and finalize the creation to generate an empty visual canvas. This blank workspace will host all subsequent analytical nodes and connections.

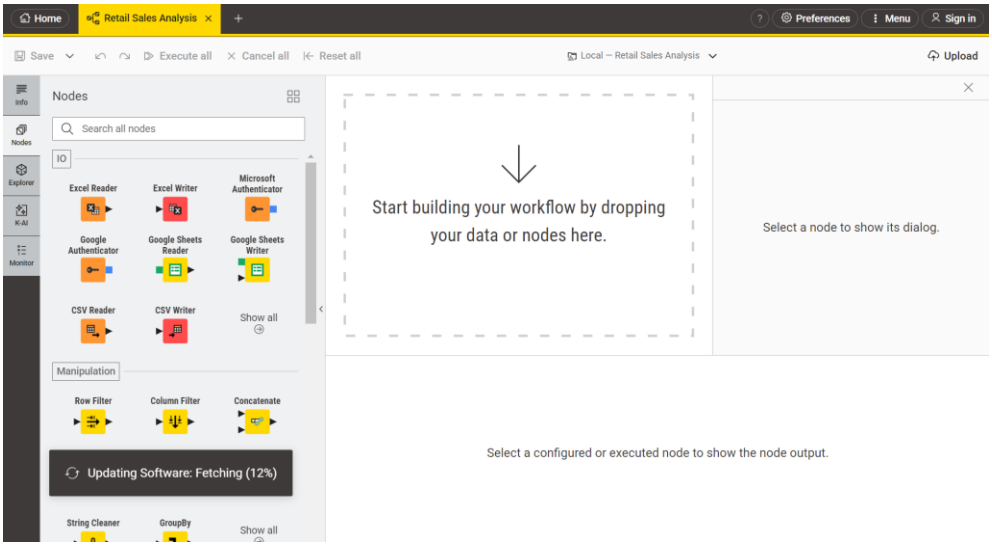


Figure 6: Empty KNIME Workspace

STEP 2: IMPORTING THE DATASET

Access the Node Repository panel and search for the **CSV Reader** node. Drag and drop this specific node onto the active workflow canvas. Double-click the component to open its internal configuration dialog box. Utilize the "Browse" function to locate and select the target Retail Sales CSV file from your local machine, then click "OK" to apply the settings. Right-click the configured node and select "Execute" to process the file. Finally, verify the successful data ingestion by right-clicking the executed node and selecting "File Table" to inspect the raw records.

Figure 7: CSV Reader Node

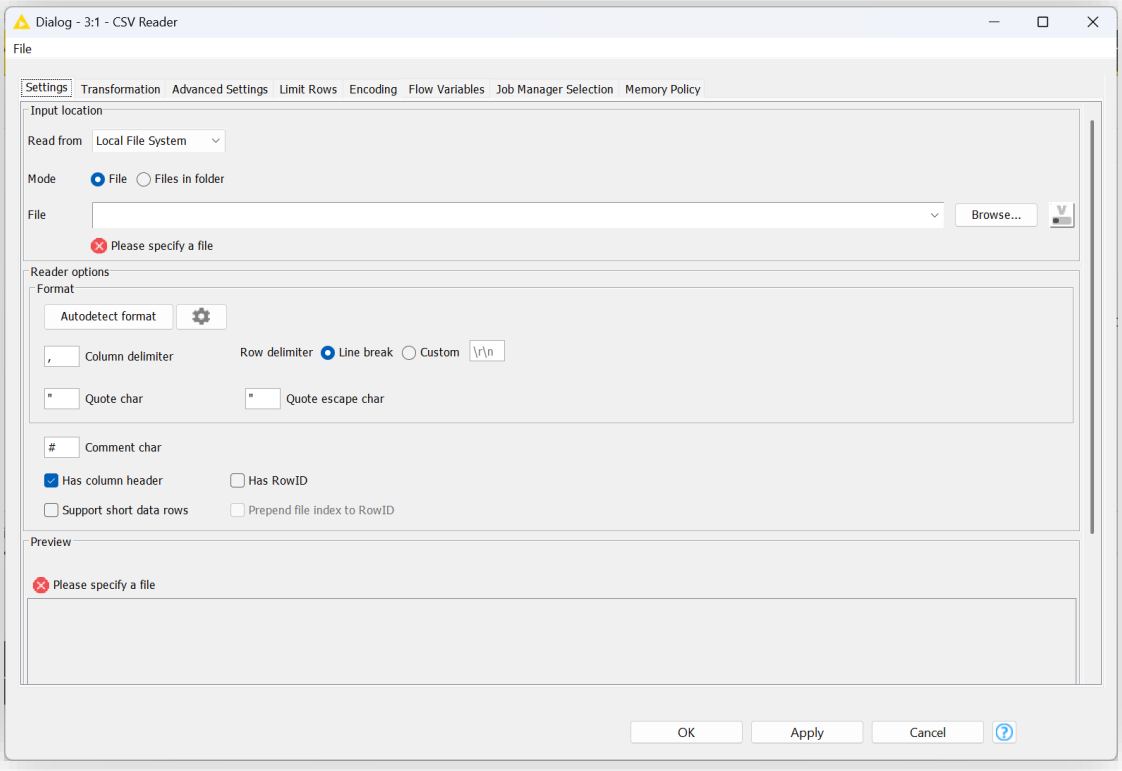


Figure 8: CSV Reader Configuration Dialog

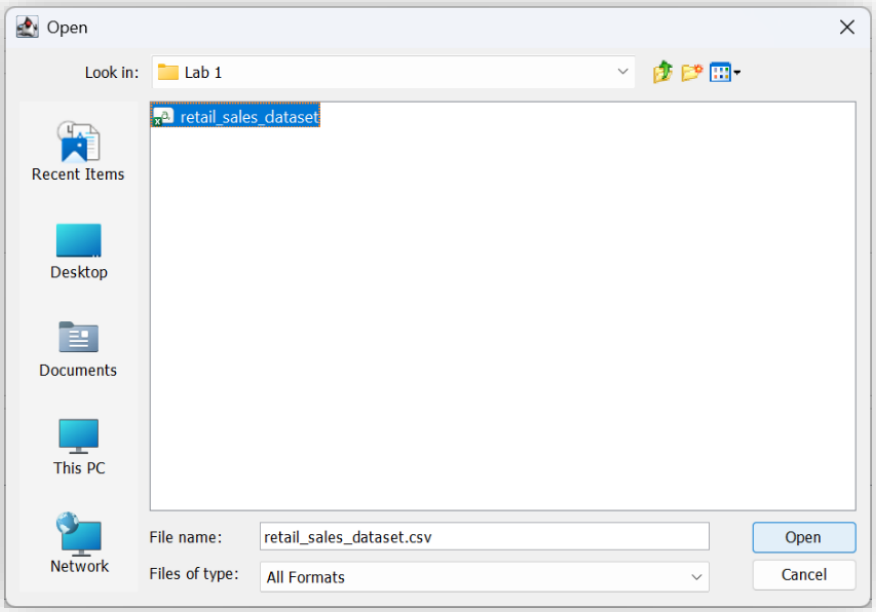


Figure 9: File Selection Dialog

STEP 3: VERIFYING AND TRANSFORMING DATA TYPES

Accurate data typing is essential for ensuring mathematical operations function correctly in later stages. Review the column headers within the CSV Reader's output table to assess the automatically assigned data types. Confirm that the '**Quantity**' attribute is designated as an Integer, '**Price per Unit**' and '**Total Amount**' are set as Doubles (or Numbers), '**Date**' is recognized as a Date format, and '**Product Category**' is classified as a String. If the ingestion process incorrectly parsed numerical values as text strings, introduce a String to Number node into the workflow to correct the issue. Similarly, employ a String to Date&Time node if the temporal data requires reformatting

▶ 1: Statistics Table ▶ 2: Nominal Histogram Table ▶ 3: Occurrences Table Flow Variables

Rows: 5 | Columns: 16 Table Statistics

#	RowID	Column	Min	Max	Mean	Std. devia...	Variance	Skewness	Kurtosis	Overall su...	No. missi...	No. N
		String	Number (...	Number (...	Number (...	Number (...	Number (...	Number (...	Number (...	Number (...	Number (I...	Nun
1	Transa	Transaction ID	1	1,000	500.5	288.819	83,416.667	0	-1.2	500,500	0	0
2	Age	Age	18	64	41.392	13.681	187.182	-0.049	-1.201	41,392	0	0
3	Quanti	Quantity	1	4	2.514	1.133	1.283	-0.014	-1.393	2,514	0	0
4	Price p	Price per Unit	25	500	179.89	189.681	35,979.017	0.736	-1.139	179,890	0	0
5	Total A	Total Amount	25	2,000	456	559.998	313,597.347	1.376	0.815	456,000	0	0

Figure 10: Statistics Output Table (Basic)

STEP 4: GENERATING SUMMARY STATISTICS

To develop a foundational understanding of the numerical distribution within the dataset, integrate a **Statistics** node into the visual pipeline. Establish a connection from the output port of the CSV Reader to the input port of this analytical node. Right-click to execute the node and subsequently open the generated statistical report. Examine the output table to review critical descriptive metrics, including the mean, minimum, maximum, and standard deviation, for all numerical columns. This preliminary exploration enables analysts to rapidly assess data spread and identify any immediate anomalies before computing business metrics

Rows: 16 | Columns: 14

Name	Type	# Missing val...	# Unique val...	Minimum	Maximum	25% Quantile	50% Quantile...	75% Quantile	Mean
Column	String	0	5	?	?	?	?	?	?
Min	Number (Float)	0	3	1	25	1	18	25	14
Max	Number (Float)	0	5	4	2,000	34	500	1,500	713.6
Mean	Number (Float)	0	5	2.514	500.5	21.953	179.89	478.25	236.059
Std. deviation	Number (Float)	0	5	1.133	559.998	7.407	189.681	424.409	210.663
Variance	Number (Float)	0	5	1.283	313,597.347	94.232	35,979.017	198,507.007	86,636.299
Skewness	Number (Float)	0	5	-0.049	1.376	-0.031	0	1.056	0.41
Kurtosis	Number (Float)	0	5	-1.393	0.815	-1.297	-1.2	-0.162	-0.824
Overall sum	Number (Float)	0	5	2,514	500,500	21,953	179,890	478,250	236,059.2
No. missings	Number (Intege	0	1	0	0	0	0	0	0
No. NaNs	Number (Intege	0	1	0	0	0	0	0	0
No. +oos	Number (Intege	0	1	0	0	0	0	0	0
No. -oos	Number (Intege	0	1	0	0	0	0	0	0
Median	Number (Float)	5	0	?	?	?	?	?	?
Row count	Number (Intege	0	1	1,000	1,000	1,000	1,000	1,000	1,000
Histogram	Image (SVG)	0	5	?	?	?	?	?	?

Figure 11: Comprehensive Column Statistics

STEP 5: FEATURE ENGINEERING (CALCULATING REVENUE)

Although the dataset includes a pre-calculated total, analysts must frequently derive financial metrics from base unit costs. To perform this operation, integrate a **Math Formula** node into the workspace and connect it to the output of the CSV Reader (or your data type conversion node). Double-click to open the configuration dialog. In the "Expression" box, construct the mathematical equation to multiply the quantity by the unit price: $Quantity$ * $Price per Unit$. At the bottom of the dialog, select "Append Column" and name this new attribute Calculated Revenue. Execute the node to append this critical financial feature to every transaction record.

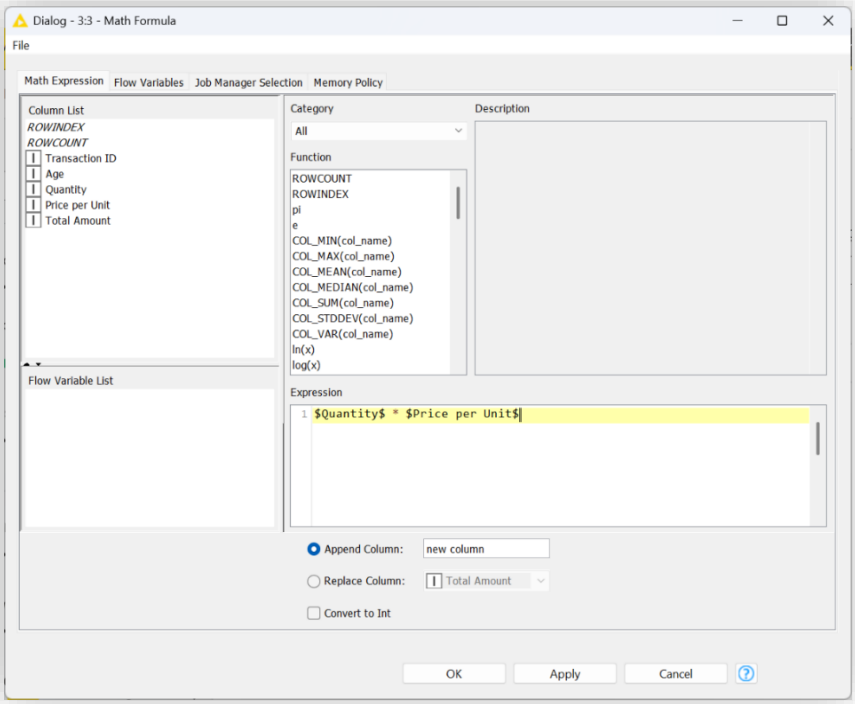


Figure 12: The Math Formula dialog box showing the exact expression used (Quantity * Price per Unit) and the "Append Column: Calculated Revenue" setting.

Rows: 5 | Columns: 10

#	RowID	Transactio... Number (Int...	Date String	Customer ID String	Gender String	Age Number (Int...	Product Ca... String	Quantity Number (Int...	Price per U... Number (Int...	Total Amou... Number (Int...	Calculated ... Number (Fl...
1	Row0	1	2023-11-24	CUST001	Male	34	Beauty	3	50	150	150
2	Row1	2	2023-02-27	CUST002	Female	26	Clothing	2	500	1000	1,000
3	Row2	3	2023-01-13	CUST003	Male	50	Electronics	1	30	30	30
4	Row3	4	2023-05-21	CUST004	Male	37	Clothing	1	500	500	500
5	Row4	5	2023-05-06	CUST005	Male	30	Beauty	2	50	100	100

Figure 13: Output Table with Calculated Revenue

STEP 6: KPI CALCULATIONS

The fundamental business metrics for this laboratory are derived using combinations of aggregation and sorting operations. Students must construct distinct data branches extending from the Math Formula node for each of the following KPIs:

KPI 1: TOTAL REVENUE

To determine the overarching financial yield of the entire dataset, attach a **GroupBy** node to the Math Formula output. Open the configuration and leave the "Groups" tab completely empty, as we are summarizing the entire dataset rather than distinct categories. Navigate to the "Manual Aggregation" tab, select the newly created Calculated Revenue column, and apply the "Sum" aggregation method. Execute the node. The resulting single-row output represents the total gross income.

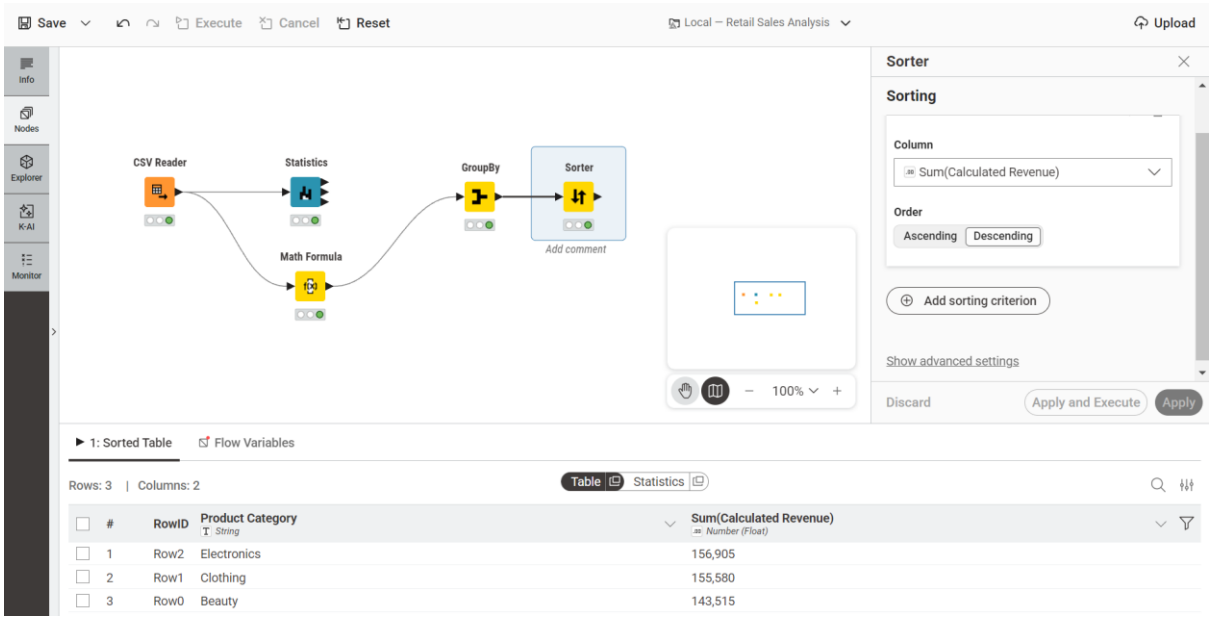


Figure 14: The single-row output table from the GroupBy node showing the total sum of the Calculated Revenue.

KPI 2: AVERAGE TRANSACTION VALUE

Understanding the typical spend per transaction dictates pricing and promotional strategies. Connect a new **GroupBy** node to the Math Formula output. Similar to the previous step, leave the "Groups" tab empty. In the "Manual Aggregation" tab, select the Calculated Revenue column, but this time, apply the "Mean" aggregation method. This operation calculates the total revenue divided by the unique transaction count, outputting the average order value.

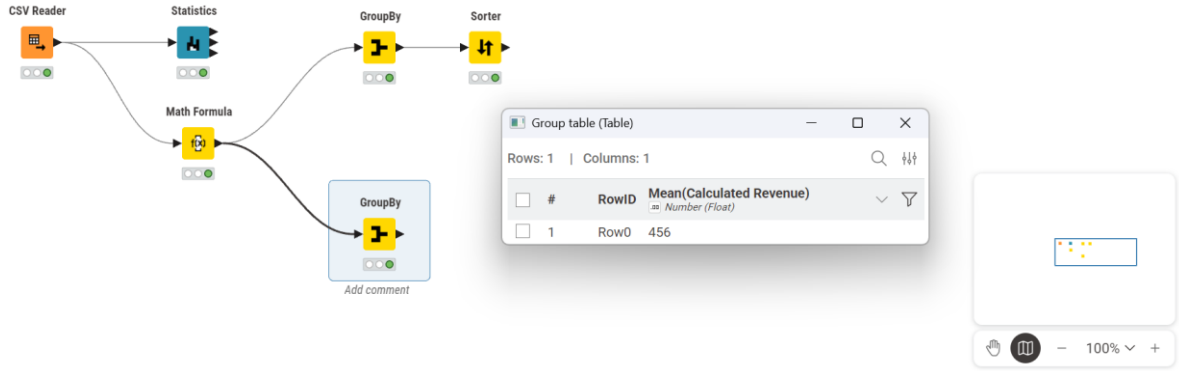


Figure 15: The output table of the GroupBy node showing the Mean of the Calculated Revenue.

KPI 3: CATEGORY REVENUE

To evaluate which retail departments generate the most income, append a third **GroupBy** node to the Math Formula output. In the "Groups" tab, select Product Category as the grouping column. In the "Manual Aggregation" tab, select Calculated Revenue and assign the "Sum" method. Next, connect a **Sorter** node to the output of this GroupBy node. Configure the Sorter to target the aggregated revenue column and set the sorting order to "Descending." Execute both nodes to reveal a ranked hierarchy of departmental financial performance.

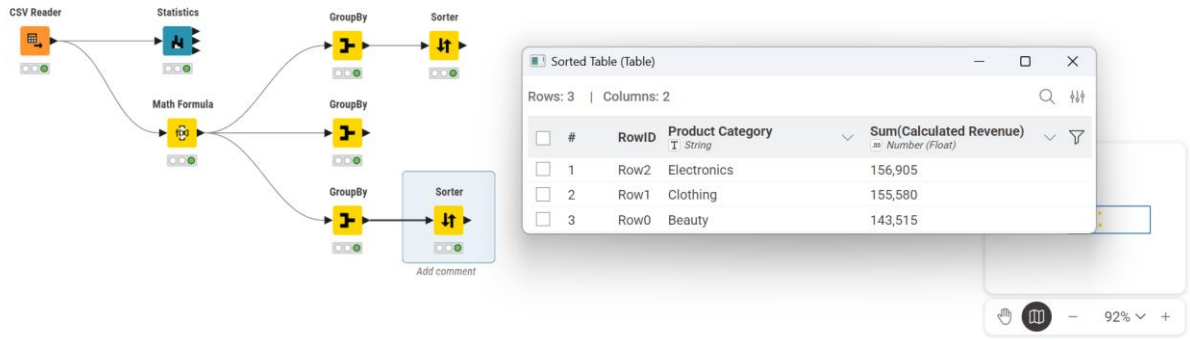


Figure 16: The output of the Sorter node, showing the Product Categories ranked from highest to lowest revenue.

KPI 4: TOP CATEGORIES BY VOLUME

While revenue indicates financial success, transaction volume highlights consumer demand. Add a final **GroupBy** node connected to the Math Formula output. Group by Product Category in the first tab. In the "Manual Aggregation" tab, select the Quantity column and assign the "Sum" method to calculate the total units moved per department. Attach a **Sorter** node, configure it to target the aggregated quantity column, and sort in "Descending" order. Executing this branch identifies the highest-velocity retail categories.

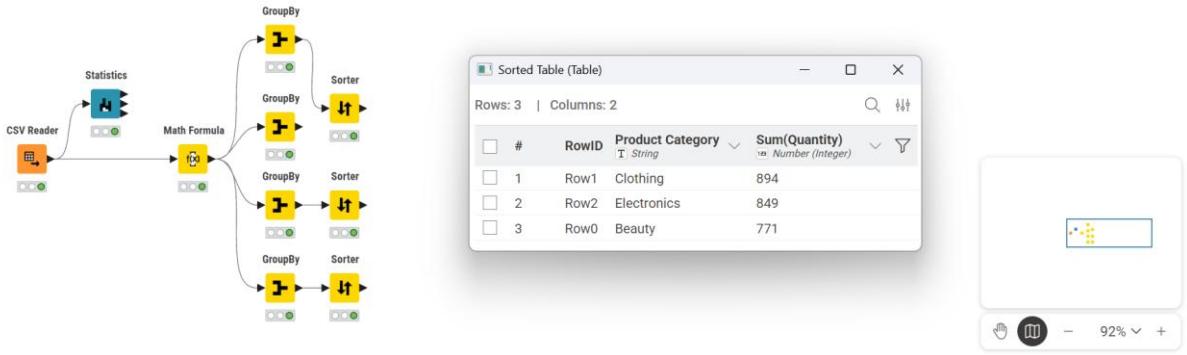


Figure 17: The output of the Sorter node, showing the categories ranked by the highest total Quantity sold.

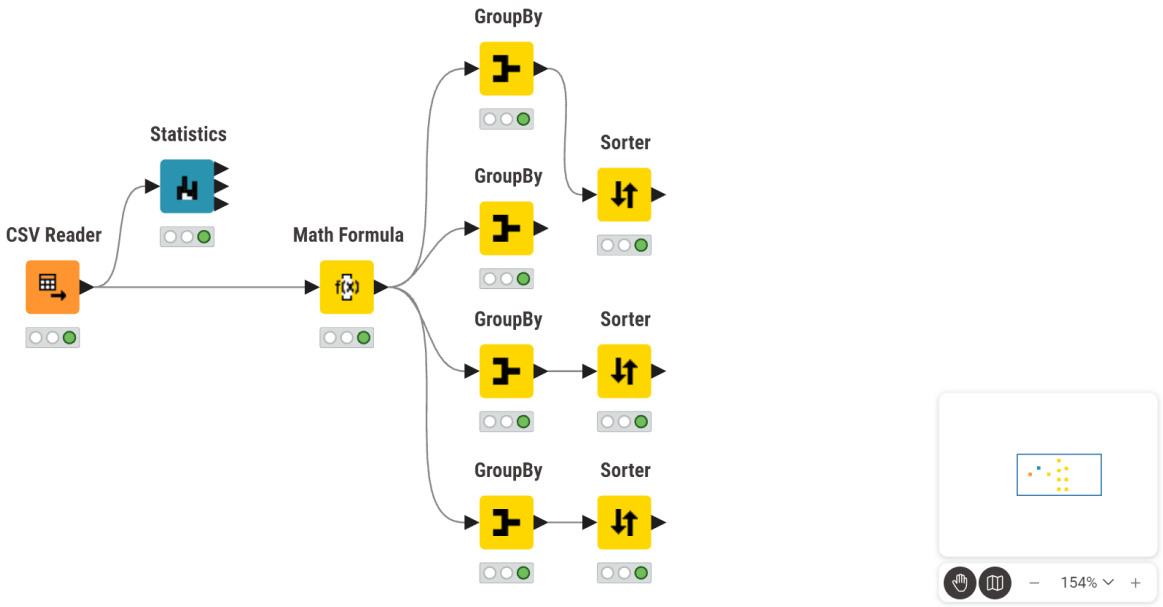


Figure 18: A wide shot of your entire KNIME canvas, showing all nodes and connections from the initial CSV Reader to the final Sorter nodes.

7. KPI SUMMARY TABLE

The following matrix provides a condensed technical reference for the Key Performance Indicators computed during this laboratory exercise:

KPI Name	Calculation Method	KNIME Nodes Utilized
Total Revenue	Cumulative sum of the derived revenue column.	Math Formula → GroupBy (Sum)
Average Transaction Value	Statistical mean of revenue across all records.	Math Formula → GroupBy (Mean)
Category Revenue	Revenue summed and isolated by departmental grouping.	Math Formula → GroupBy (Category) → Sorter (Descending)
Top Categories by Volume	Total items sold (Quantity) isolated by grouping.	Math Formula → GroupBy (Category) → Sorter (Descending)

LAB NO 2: CRISP-DM

1. INTRODUCTION

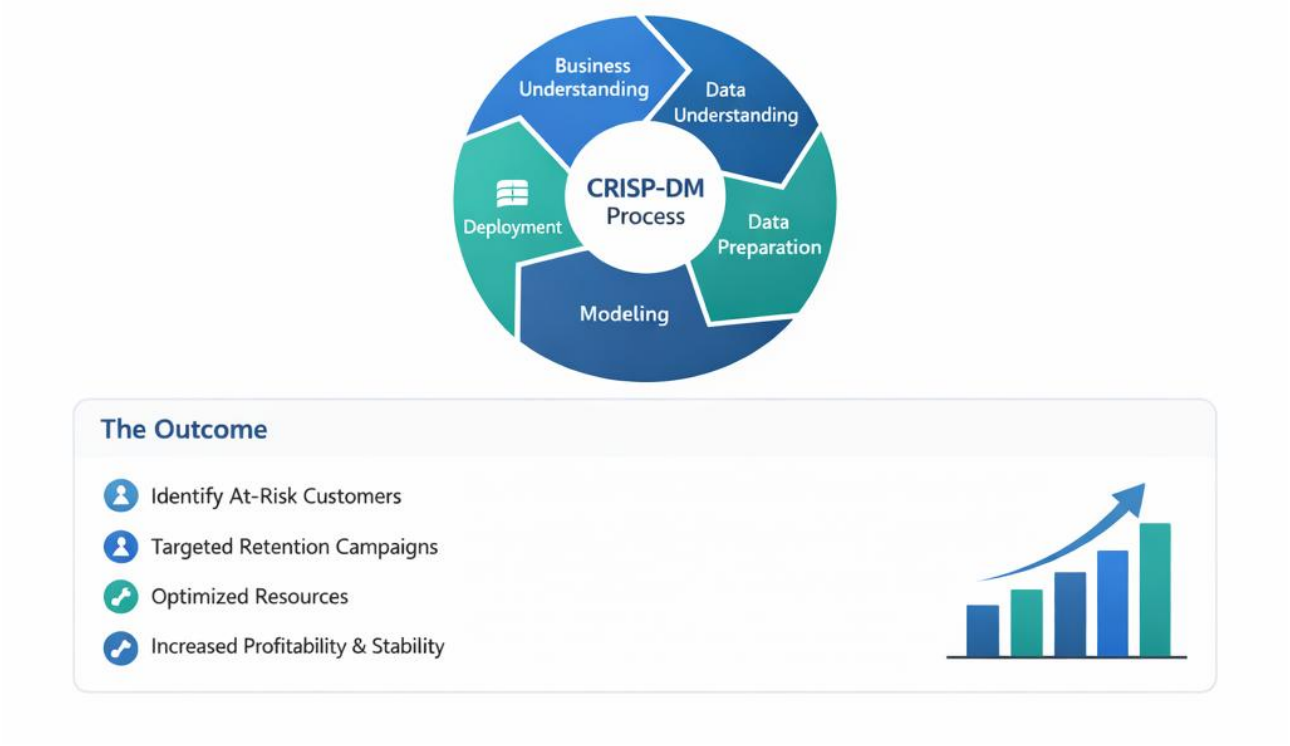
In the highly saturated landscape of modern digital commerce, sustaining long-term profitability relies heavily on cultivating customer loyalty rather than solely acquiring new users. The financial overhead associated with marketing to and onboarding a new buyer consistently outweighs the cost of maintaining an existing customer relationship. Consequently, customer churn, the phenomenon where users disengage, abandon the platform, or transition to a competitor, represents a critical vulnerability for e-commerce enterprises.

To combat this attrition, organizations must evolve from reactive customer service models to proactive, predictive strategies. By leveraging data mining techniques, businesses can analyze historical transaction patterns, demographic details, and engagement metrics to anticipate which customers exhibit a high probability of churning. Identifying these at-risk segments allows the enterprise to execute targeted retention campaigns, optimize resource allocation, and ultimately preserve revenue streams. This laboratory session introduces the application of the Cross-Industry Standard Process for Data Mining (CRISP-DM) framework to construct, evaluate, and interpret a predictive model for customer churn.

2. PROBLEM STATEMENT

The enterprise currently experiences unpredictable revenue leakage due to customer attrition. Without a mechanism to foresee which accounts are likely to close, retention efforts are deployed

reactively and inefficiently. The organization requires a robust, data-driven classification framework capable of analyzing historical user behavior to flag customers demonstrating a high propensity to leave. Resolving this challenge will empower management to deploy preemptive, highly targeted retention initiatives, thereby securing continuous revenue and stabilizing the customer base.



3. DATASET DESCRIPTION

This laboratory exercise utilizes the **Customer Churn Dataset** (Churn_Modelling.csv), containing historical records of customer demographics, financial standing, and platform engagement. Understanding the structure of this data is the foundational step in the Data Understanding phase. The key attributes are categorized as follows:

- **Identification Metrics:** RowNumber, CustomerId, and Surname. (Note: These attributes hold no predictive value and are typically excluded during the modeling phase to prevent algorithmic bias).
- **Demographic Profile:** Geography (location of the customer), Gender, and Age.
- **Financial Indicators:** CreditScore (creditworthiness), Balance (current account standing), and EstimatedSalary.
- **Engagement Metrics:** Tenure (duration of the customer relationship), NumOfProducts (services utilized by the customer), HasCrCard (binary indicator of credit card possession), and IsActiveMember (binary indicator of recent platform activity).
- **Target Variable:** Exited. This is the definitive label for supervised learning. A value of 1 indicates the customer has churned, while a value of 0 indicates the customer has been retained.

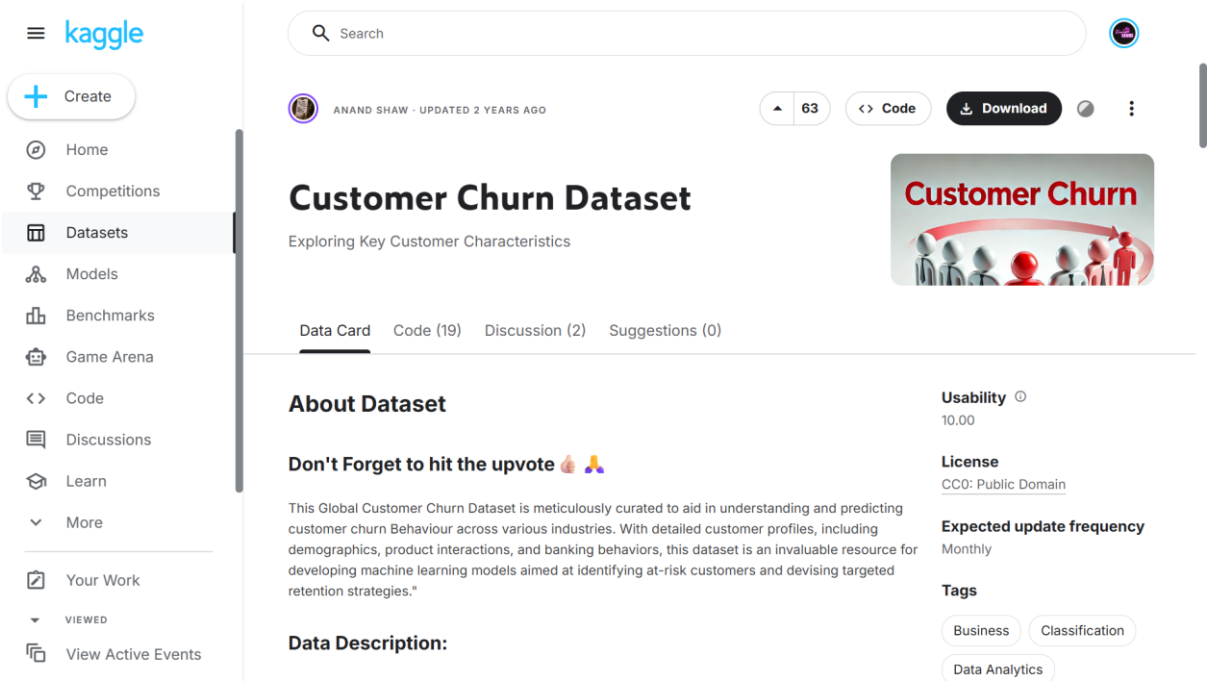


Figure 19: Kaggle Customer Churn Dataset

4. TOOLS USED

To execute this analysis, students will utilize the **Orange Data Mining** platform. Orange provides an intuitive, canvas-based visual programming environment that aligns seamlessly with the CRISP-DM methodology. By utilizing drag-and-drop widgets rather than writing raw code, analysts can focus entirely on data logic, feature relationships, and algorithm evaluation. This tool enables rapid prototyping of predictive models while providing immediate, interactive visual feedback at every phase of the analytical pipeline.

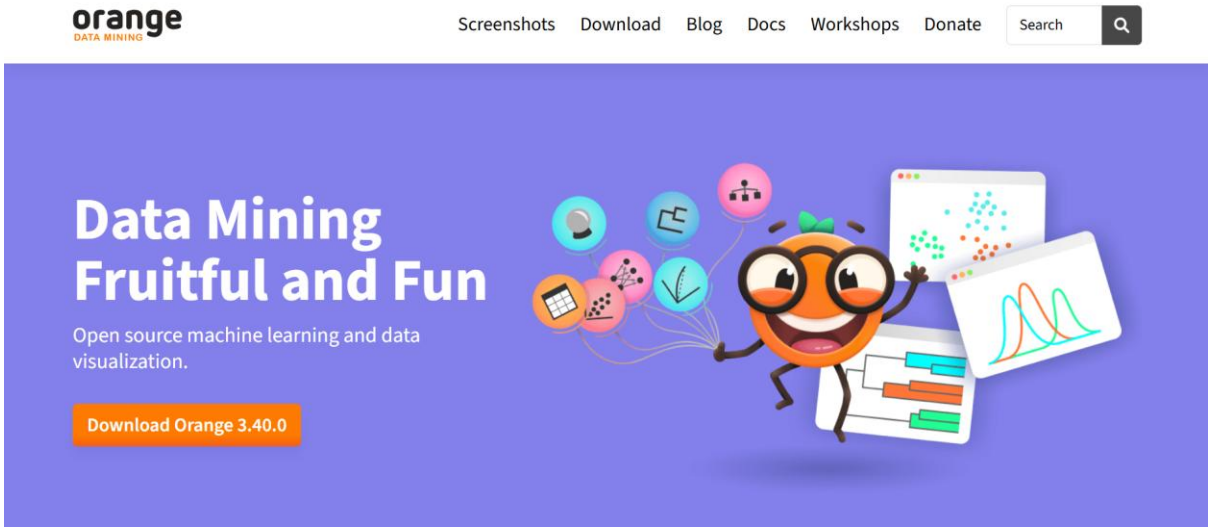


Figure 20: Orange Data Mining Website

5. OBJECTIVES

Upon completion of this laboratory session, students will be able to:

1. Apply the Cross-Industry Standard Process for Data Mining (CRISP-DM) methodology to translate a commercial problem into a structured analytical workflow.

2. Perform initial data ingestion and exploratory data analysis to identify variable distributions and data types within a visual canvas.
3. Execute critical data preparation techniques, including feature selection and categorical encoding, to format raw data for machine learning algorithms.
4. Construct and train a predictive classification model to forecast binary customer attrition.
5. Assess model viability and business impact by calculating and interpreting explicit Key Performance Indicators (KPIs): Churn Rate, Retention Rate, Accuracy, and ROC-AUC.

6. STEP-BY-STEP PROCEDURE

The following workflow systematically applies the CRISP-DM methodology to the Customer Churn Dataset using the Orange Data Mining platform.

STEP 1: DATA INGESTION (BUSINESS & DATA UNDERSTANDING)

- **Action:** Import the customer churn dataset into the visual workspace to establish the foundational data pipeline.
- **Configuration:** Add a File widget to the canvas. Double-click the widget to launch the file browser. Locate and select the Churn_Modelling.csv document. Ensure the delimiter is correctly recognized as a comma.
- **Execution:** Close the configuration window to instruct the system to load the dataset into active memory.
- **Verification:** Drag a Data Table widget onto the canvas. Connect the output port of the File widget to the input port of the Data Table. Open the table to visually confirm the successful ingestion of the 10,000 customer records and 14 attributes.

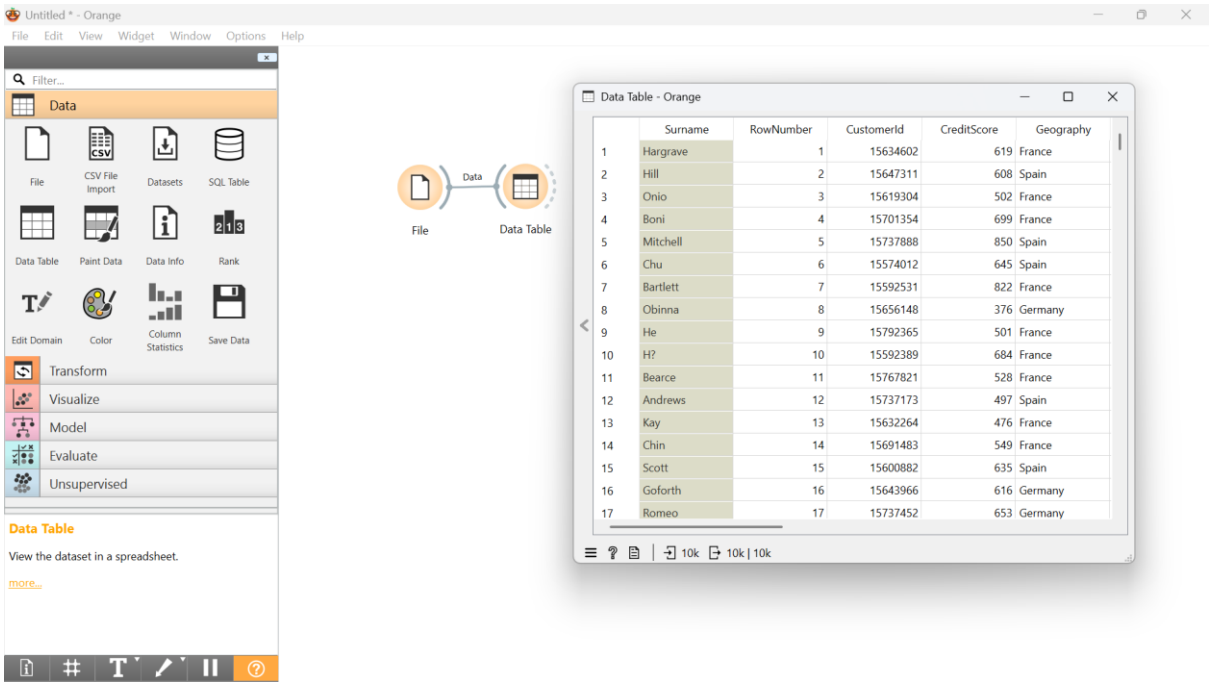


Figure 21: The Orange canvas showing a File widget routed to a Data Table to confirm successful data ingestion.

STEP 2: EXPLORATORY DATA ANALYSIS (DATA UNDERSTANDING)

- **Action:** Analyze the statistical distributions and feature characteristics of the ingested banking dataset to identify underlying patterns.
- **Configuration:** Introduce a Feature Statistics widget to the workspace. Establish a data connection from the File widget directly to this new node.

- **Execution:** Open the Feature Statistics interface to render the graphical distributions.
- **Verification:** Examine the central tendencies. Confirm that continuous variables such as Age, CreditScore, and Balance display valid numerical distributions. Observe the categorical split in the Exited variable to understand the baseline class imbalance (retained versus churned customers).

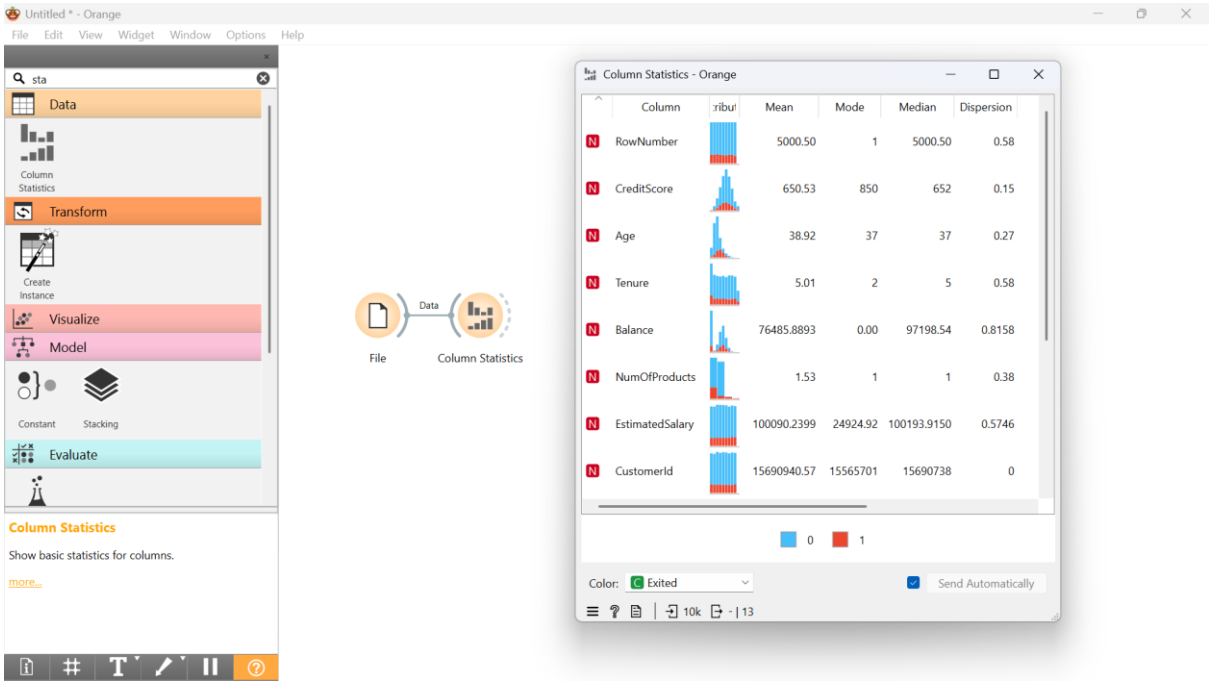


Figure 22: A visual summary of variable distributions, categorical splits, and central tendencies within the dataset.

STEP 3: FEATURE SELECTION AND TARGET DESIGNATION (DATA PREPARATION)

- **Action:** Isolate predictive features, designate the supervised learning target, and discard irrelevant identification metrics to prevent algorithmic bias.
- **Configuration:** Add a Select Columns widget. Route the data flow from the File widget into Select Columns. Open the interface. Drag the Exited attribute into the designated "Target" domain. Move the RowNumber, CustomerId, and Surname attributes into the "Ignored" domain. Retain all remaining attributes (e.g., Geography, EstimatedSalary, NumOfProducts) within the "Features" domain.
- **Execution:** Apply the column modifications to finalize the analytical dataset schema.
- **Verification:** Connect a secondary Data Table widget to the output of the Select Columns node. Inspect the resulting grid to verify that the ignored identifiers are absent and that the Exited column is distinctly highlighted as the target variable.

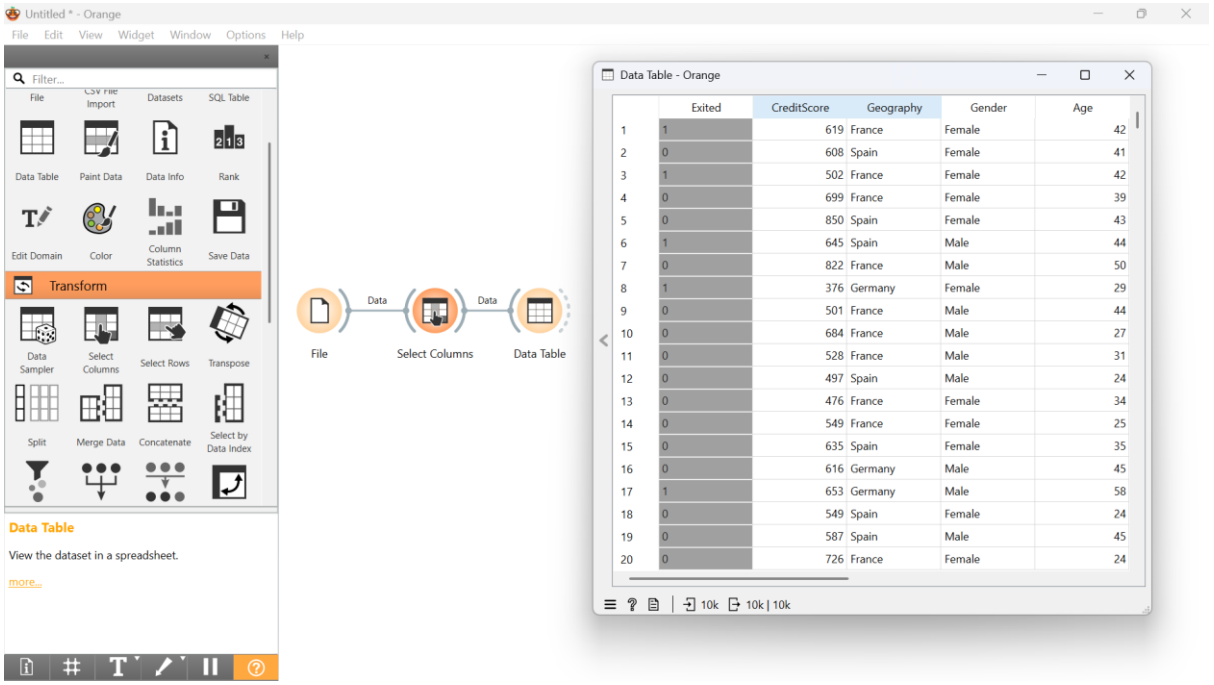


Figure 23: The Orange canvas and an output table verifying the isolation of predictive features and the target variable.

STEP 4: ALGORITHM INITIALIZATION (MODELING)

- **Action:** Instantiate a supervised machine learning classifier to construct the predictive churn model.
- **Configuration:** Deploy a Logistic Regression widget onto the canvas. Double-click to access its hyperparameter tuning interface. Retain the default Regularization type at Ridge (L2) and the Cost strength (C) at its baseline value to establish an initial, unoptimized model.
- **Execution:** Place the widget within the modeling area. No direct data input connection is required at this isolated step, as the algorithm will serve as an input module for the evaluation phase.
- **Verification:** Confirm that the widget displays no error indicators, ensuring the algorithm is properly initialized and ready to receive training data.

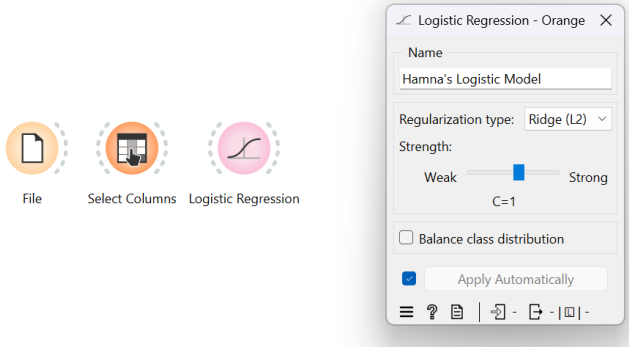


Figure 24: The hyperparameter configuration dialogue for the deployed supervised machine learning classifier.

STEP 5: MODEL VALIDATION AND SCORING (EVALUATION)

- **Action:** Train the logistic regression algorithm and quantify its predictive capabilities using stratified cross-validation.
- **Configuration:** Insert a Test and Score widget into the pipeline. Establish two distinct input connections: route the "Data" output from the Select Columns widget into the "Data" input of the node, and route the "Learner" output from the Logistic Regression widget into the "Learner" input of the node. Within the widget interface, select "Cross-validation" and set the number of folds to 10.
- **Execution:** Allow the platform to automatically compile the algorithm and execute the validation process across the data folds.
- **Verification:** Review the generated performance table within the widget. Locate and record the specific numeric values for Classification Accuracy (CA) and Area Under ROC (AUC) to benchmark the model's effectiveness.

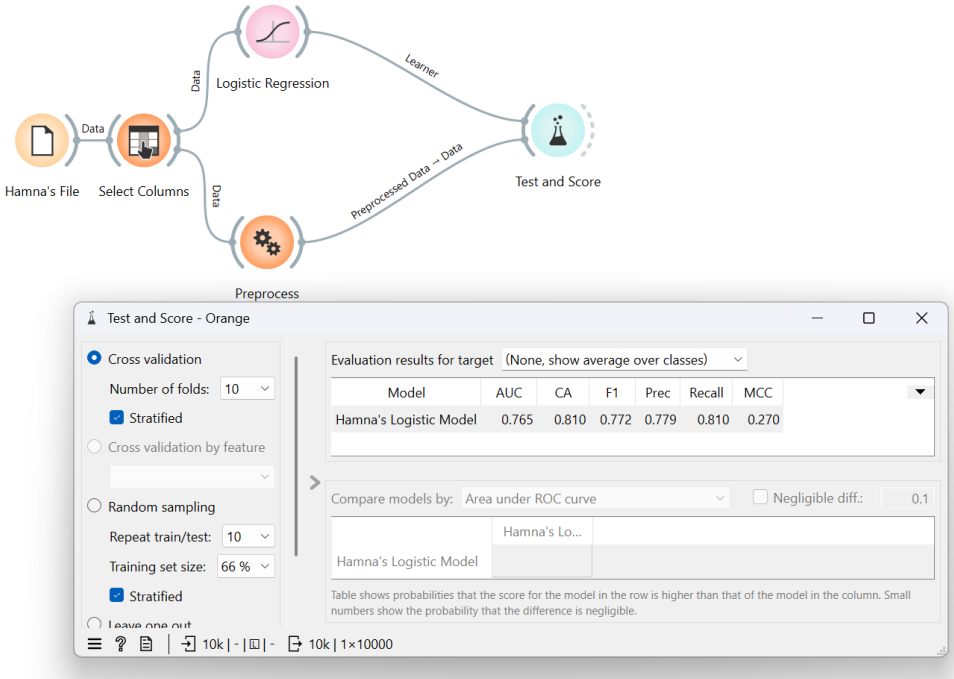


Figure 25: The cross-validation evaluation results pane detailing metric performance such as AUC, CA, and F1 score.

STEP 6: DIAGNOSTIC ERROR ANALYSIS (EVALUATION)

- **Action:** Dissect the classification errors to determine the model's exact misclassification rates regarding false positives and false negatives.
- **Configuration:** Add a Confusion Matrix widget to the canvas. Connect the "Evaluation Results" output of the Test and Score widget to the input port of the Confusion Matrix.
- **Execution:** Open the matrix interface and ensure the display parameters are set to show the absolute number of instances.
- **Verification:** Identify the exact volume of customers correctly predicted to leave (True Positives) versus the volume of loyal customers incorrectly flagged as at-risk (False Positives).

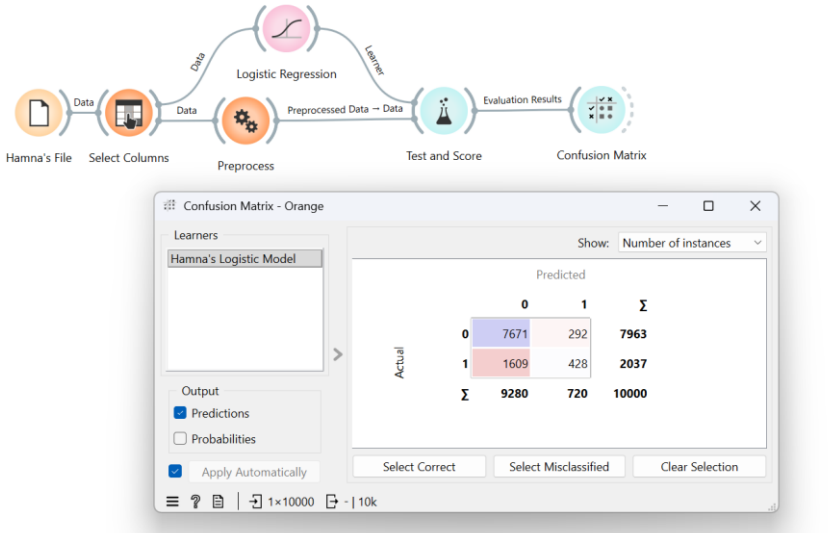


Figure 26: A grid layout displaying classification errors by mapping actual classes against model predictions.

STEP 7: PROTOTYPING DEPLOYMENT (DEPLOYMENT)

Action: Apply the trained classification algorithm to the dataset to simulate the real-world generation of predictive churn scores.
Configuration: Place a Predictions widget on the canvas. Connect the preprocessed data stream from the Select Columns widget to the "Data" input port. Simultaneously, connect the constructed model from the Logistic Regression widget to the "Predictors" input port.
Execution: Open the Predictions interface to initiate the scoring sequence.
Verification: Scroll through the output table to observe the newly appended columns alongside the original customer data. Confirm that the model successfully outputs a distinct probability score and a final predicted class (0 or 1) for the Exited variable across every individual customer row.

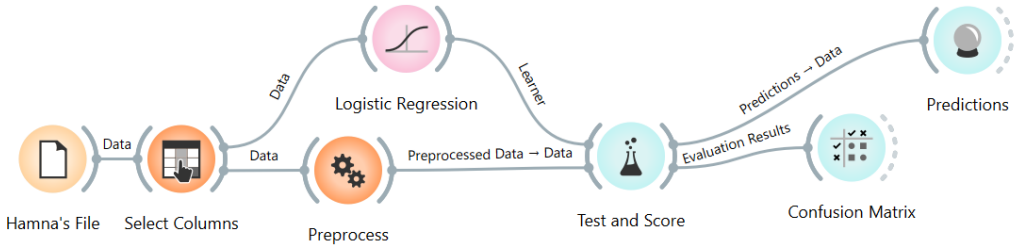


Figure 27: A wide view of the fully connected Orange data workflow from raw file ingestion to final predictions.

Exited	Hamna's Logistic Model	Hamna's Logistic Model (0)	Hamna's Logistic Model (1)	Fold	CreditScore	Geography=Franc	ography=Germa	Geography=Spair	Gender=Female
1	0	0.675384	0.324616	1	502	1	0	0	1
0	0	0.84809	0.15191	1	850	0	0	1	1
0	0	0.905452	0.0945483	1	822	1	0	0	0
0	0	0.828531	0.171469	1	476	1	0	0	1
0	0	0.891043	0.108957	1	549	1	0	0	1
0	0	0.970609	0.0293906	1	726	1	0	0	1
0	0	0.673247	0.326753	1	574	0	1	0	1
0	0	0.840267	0.159733	1	582	0	1	0	0
0	0	0.868045	0.131955	1	776	0	1	0	1
1	0	0.58616	0.41384	1	601	0	1	0	0
0	0	0.928782	0.0712178	1	603	0	1	0	0
0	0	0.924771	0.0752293	1	729	1	0	0	0
0	0	0.932362	0.067638	1	730	0	0	1	0
0	0	0.603333	0.396667	1	589	0	1	0	1
0	0	0.575362	0.424638	1	756	0	1	0	0
0	0	0.832778	0.167222	1	682	1	0	0	1
0	0	0.83788	0.16212	1	635	0	0	1	1
1	0	0.769311	0.230689	1	704	0	1	0	1
0	0	0.941505	0.0584951	1	510	1	0	0	1
0	0	0.861895	0.138105	1	597	0	0	1	1
0	0	0.889985	0.110015	1	525	0	0	1	1
1	0	0.75049	0.24951	1	668	0	1	0	0
0	0	0.669329	0.330671	1	594	0	1	0	1

Figure 28: The final dataset view with newly appended predictive probability scores and assigned classes.

Here is the final section of your lab manual, covering the **Key Performance Indicator (KPI) Calculations** and the **KPI Summary Table**. I have maintained the formal, instructional tone and ensured the instructions rely on the widgets we configured in the previous steps.

7. KEY PERFORMANCE INDICATOR (KPI) CALCULATIONS

To bridge the gap between technical machine learning metrics and actionable business intelligence, analysts must translate the output of the predictive model into standardized business metrics. Calculate and record the following KPIs to finalize your analysis.

7.1 CHURN RATE

- **Action:** Determine the baseline proportion of the customer base that has abandoned the e-commerce platform.
- **Calculation Method:** $\frac{\text{Total Number of Churned Customers (Exited = 1)}}{\text{Total Number of Customers}} \times 100$
- **Widget Extraction:** Open the Feature Statistics widget configured in Step 2. Locate the Exited variable. Divide the absolute count of the 1 class by the total number of instances (10,000) and multiply by 100 to derive the percentage.

7.2 RETENTION RATE

- **Action:** Quantify the percentage of the customer base that remains loyal and actively engaged with the platform.
- **Calculation Method:** $\frac{\text{Total Number of Retained Customers (Exited = 0)}}{\text{Total Number of Customers}} \times 100$ *(Alternatively: 100% - Churn Rate)*
- **Widget Extraction:** Utilizing the same Feature Statistics widget, divide the absolute count of the 0 class by the total number of instances (10,000) and multiply by 100.

7.3 MODEL ACCURACY

- **Action:** Evaluate the overall correctness of the predictive algorithm across all customer records.
- **Calculation Method:** $\frac{\text{True Positives + True Negatives}}{\text{Total Number of Predictions}} \times 100$
- **Widget Extraction:** Open the Test and Score widget deployed in Step 5. Locate the column labeled **CA** (Classification Accuracy) corresponding to the Logistic Regression model. Multiply this decimal value by 100 to report the accuracy as a standard percentage.

7.4 ROC-AUC (RECEIVER OPERATING CHARACTERISTIC - AREA UNDER CURVE)

- **Action:** Assess the algorithm's absolute ability to distinguish between loyal customers and at-risk customers, independent of any specific classification threshold.
- **Calculation Method:** The mathematical integral of the True Positive Rate against the False Positive Rate (requires algorithmic computation rather than manual arithmetic).
- **Widget Extraction:** Open the Test and Score widget. Locate the column labeled **AUC**. Record this value directly. A value approaching 1.0 indicates exceptional discriminative ability, while a value near 0.5 indicates performance no better than random guessing.

8. KPI SUMMARY TABLE

Compile your findings into the following summary matrix to present a concise executive overview of both the baseline business reality and the model's performance.

KPI Name	Calculation Method	Orange Widgets Used
Churn Rate	(Count of Exited=1 / Total Customers) × 100	Feature Statistics, Data Table
Retention Rate	(Count of Exited=0 / Total Customers) × 100	Feature Statistics, Data Table
Accuracy	(Correct Predictions / Total Predictions) × 100	Test and Score (CA Column), Confusion Matrix
ROC-AUC	Area under the True Positive vs. False Positive curve	Test and Score (AUC Column)

CONCLUSION

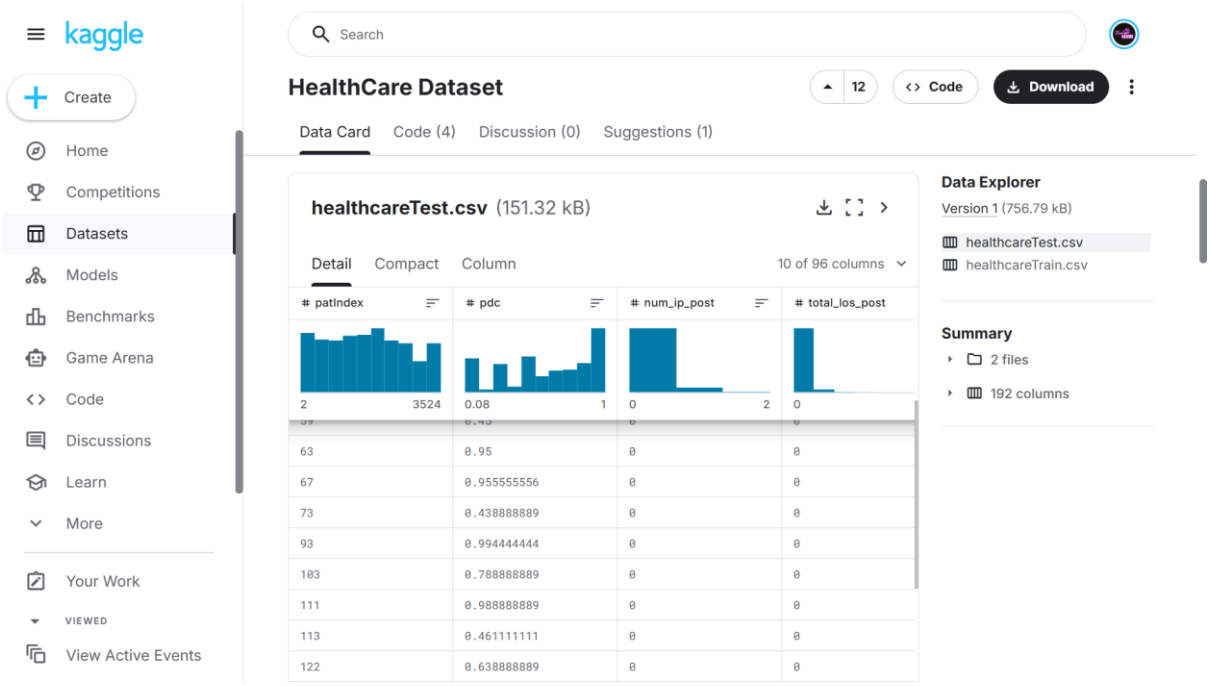
This completes the drafting of your comprehensive, CRISP-DM aligned lab manual for Customer Churn Prediction using Orange.

We have successfully built original, highly technical content covering the Introduction, Problem Statement, Dataset Description, Tools Used, Objectives, Step-by-Step Procedures, and KPI computations.

LAB NO 3: DATA CLEANING

1. INTRODUCTION

In the domain of healthcare analytics, the structural integrity of clinical and administrative data is paramount. Before sophisticated predictive algorithms or patient-care models can be deployed, the underlying repository must be rigorously sanitized. Real-world medical ledgers are frequently plagued by fragmented entries, redundant patient interactions, and anomalous extreme values (noise) caused by system integration errors or clerical oversight. If left unaddressed, these artifacts can severely distort statistical outcomes and jeopardize data-driven strategies. Data cleaning acts as the essential foundational layer, systematically neutralizing these defects to transform disjointed logs into a cohesive, high-fidelity resource. This laboratory exercise introduces students to robust data purification, demonstrating how to address absent data points, eliminate repetitive records, and reduce signal noise to synthesize an analytically viable healthcare dataset.



2. PROBLEM STATEMENT

A regional hospital network has compiled a massive volume of patient history, medication adherence, and billing metrics into a centralized repository. However, the analytics department is currently unable to utilize this data. The raw export contains duplicate transaction logs from recurring batch transfers, pervasive missing fields across critical diagnostic categories, and highly irregular numerical spikes (noise) that do not reflect realistic clinical scenarios. The clinical intelligence team urgently requires a scalable, automated data-cleansing pipeline to sanitize these records. The primary challenge is architecting a workflow that not only resolves these inconsistencies but also allows administration to quantitatively track improvements in data reliability through targeted governance metrics.

3. DATASET DESCRIPTION

During this practical session, analysts will manipulate the **Healthcare Dataset**, provided as a tabular Comma-Separated Values (CSV) file. This complex ledger captures longitudinal patient

profiles, encompassing diagnostic indicators, health utilization metrics, and associated medical costs.

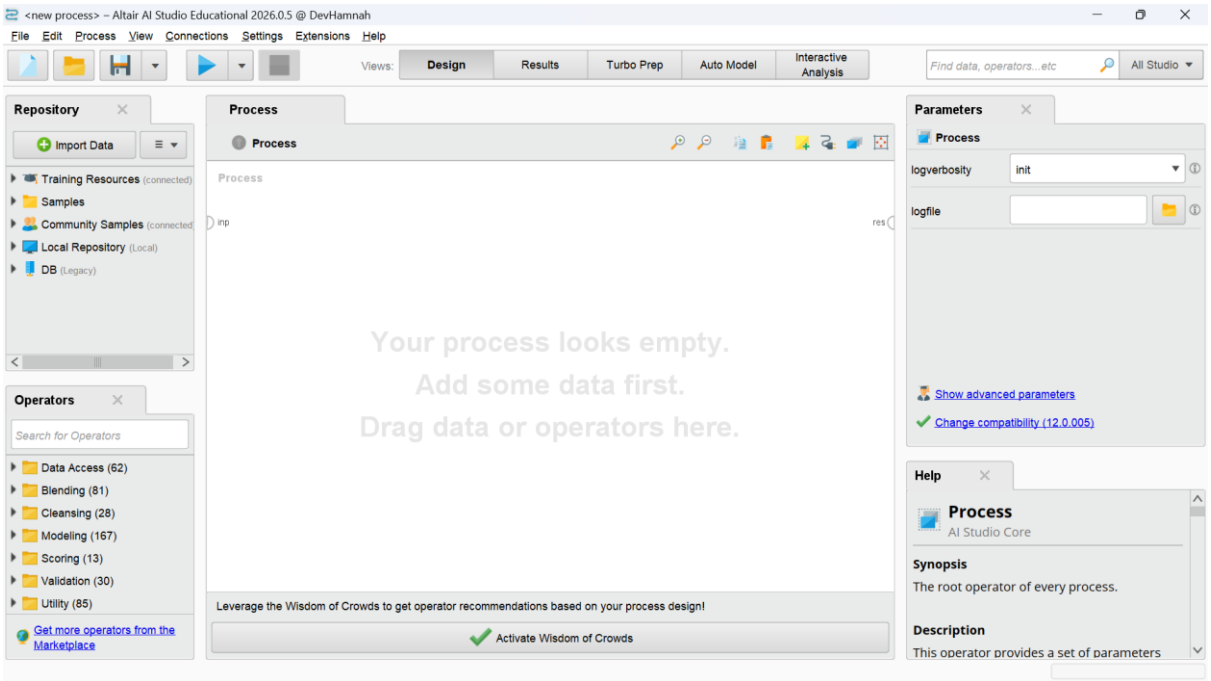
Key attributes to familiarize yourself with include:

- **Patient Anchors:** Identifiers such as patIndex and patient_key, which serve as unique structural keys for individual medical records.
- **Clinical & Demographic Flags:** Variables including age_cat, sexN, and binary condition markers (e.g., DIABETES, ASTHMA, HYPERTENSION) that map underlying patient health statuses.
- **Financial & Utilization Metrics:** Continuous numerical fields like post_total_cost, pre_ip_cost, and total_los (length of stay), which are highly susceptible to data entry noise and outliers.
- **Therapeutic Labels:** Categorical text strings, such as drug_class, detailing the treatment pathway.

4. TOOLS USED

This procedure relies on the integration of **RapidMiner Studio** alongside embedded **Python** scripting to ease a hybrid data preparation strategy.

RapidMiner Studio is an advanced visual analytics platform that hastens data transformation through a repository of pre-configured "operators." It empowers users to map out filtering and imputation sequences without writing manual syntax. However, because specialized noise-filtering heuristics often require customized mathematical logic, the platform's native *Execute Python* operator will also be used. This dual-tool framework provides analysts with both the rapid prototyping of a drag-and-drop interface and the algorithmic granularity of Python.



5. OBJECTIVES

Upon successful completion of this laboratory module, students will prove practical competency by achieving the following learning outcomes:

- Ingest and profile raw medical datasets within the RapidMiner Studio environment to expose structural flaws.
- Deploy visual operators to detect and drop redundant patient entries.

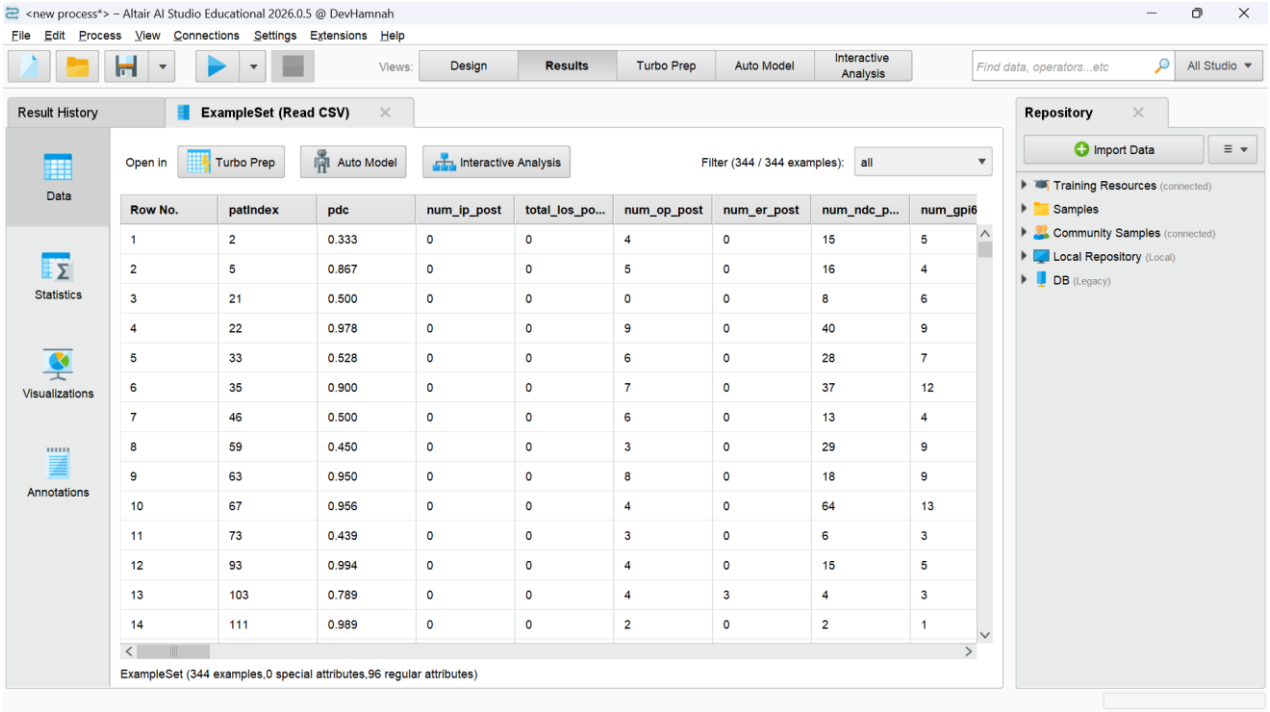
- Address absent data fields across various data types using statistically sound imputation techniques.
- Implement Python-based scripting to find, cap, and smooth anomalous numerical outliers (noise filtering).
- Quantify the dataset's rehabilitation by calculating explicit Key Performance Indicators (KPIs): % Missing Reduced, Duplicate Removal %, and a composite Data Quality Score.

6. STEP-BY-STEP PROCEDURE

The following workflow systematically applies data sanitization principles to the Healthcare dataset.

STEP 1: DATA INGESTION AND PROFILING (INITIAL ASSESSMENT)

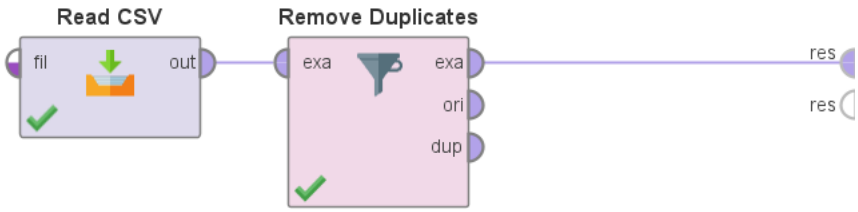
- **Action:** Import the raw clinical records into the visual workspace to show a baseline understanding of data corruption.
- **Configuration:** Find the **Read CSV** operator within the repository and drop it onto the main process canvas. Double-click the part, navigate your local file system, and target the Healthcare.csv document. Ensure the import wizard correctly parses the comma delimiters.
- **Execution:** Route the output port to the results node at the edge of the canvas and execute the process.
- **Verification:** Transition to the "Statistics" tab. Document the first row count and note the exact volume of missing values explicitly highlighted across vulnerable fields like post_total_cost and age_cat.



STEP 2: REDUNDANT RECORD ELIMINATION (DEDUPLICATION)

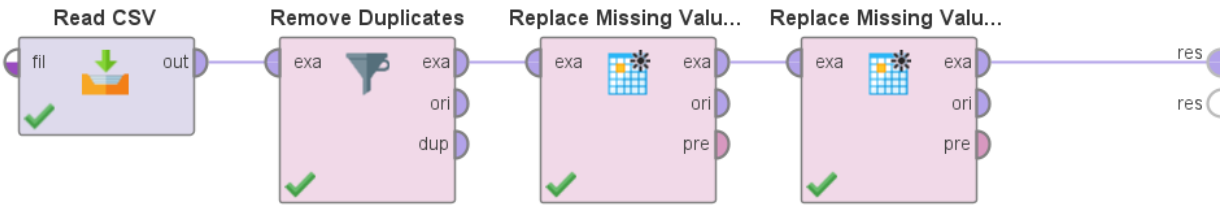
- **Action:** Purge the ledger of identically overlapping rows to guarantee each patient interaction is uniquely represented.
- **Configuration:** Sever the earlier connection and insert a **Remove Duplicates** operator directly following the Read CSV node.

- **Settings:** In the parameters panel, adjust the operator filter to evaluate all attributes. This strict configuration ensures that only absolute, perfect clones are discarded.
- **Execution:** Connect the revised pipeline and execute.
- **Verification:** Inspect the updated grid in the Results pane. The negative variance in the total instance count represents the successful neutralization of duplicate artifacts.



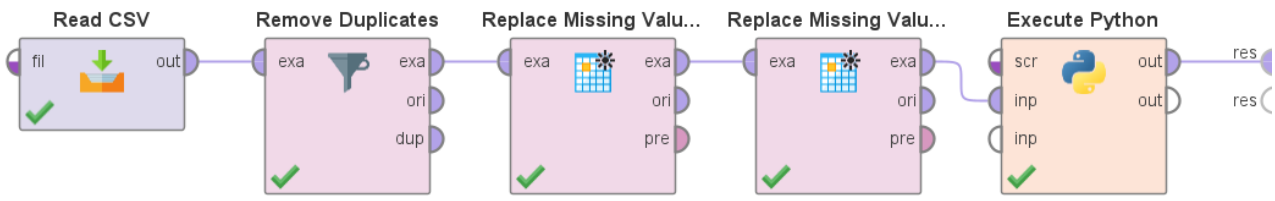
STEP 3: ADDRESSING ABSENT DATA POINTS (IMPUTATION)

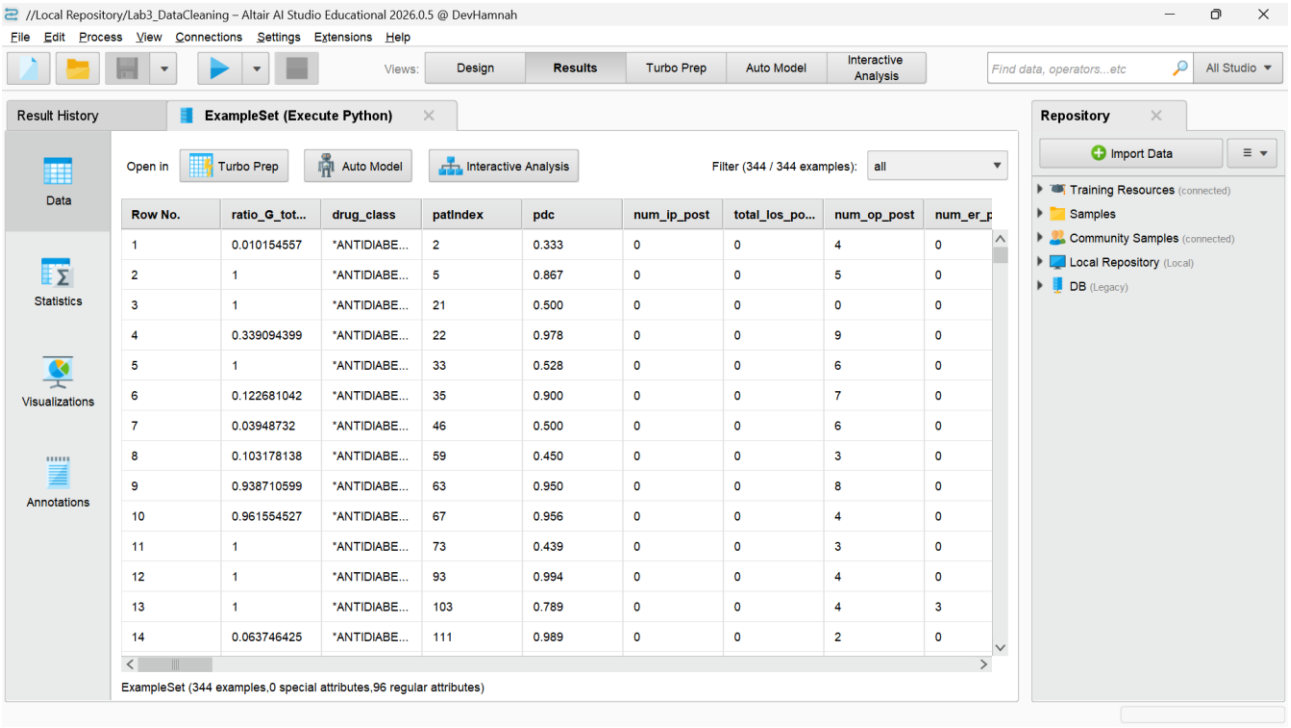
- **Action:** Resolve gaps in the dataset where critical clinical or financial information was omitted, utilizing replacement algorithms to preserve record volume.
- **Configuration:** Append a **Replace Missing Values** operator to the output stream of the deduplication stage.
- **Settings:** For continuous numerical features (e.g., billing costs), configure the substitution method to "Average" to sustain the mathematical center of the data. For categorical elements (e.g., drug_class), string a secondary operator set to inject the statistical "Mode" (most frequent value).
- **Execution & Verification:** Run the workflow. Re-examine the Statistics tab to confirm that the missing value tallies for all processed columns have successfully dropped to zero.



STEP 4: PROGRAMMATIC NOISE FILTERING (OUTLIER MUTING)

- **Action:** Intercept and smooth extreme statistical anomalies in sensitive financial distributions using custom code.
- **Configuration:** Drag an **Execute Python** operator onto the canvas, mapping the imputed data stream into its primary input port.
- **Settings:** Within the operator's script editor, construct a concise pandas routine to cap severe outliers. Define a mathematical threshold (such as the 99th percentile for post_total_cost) and write a function that compresses any values exceeding this limit down to the boundary line, effectively muting noise without deleting the patient row entirely.
- **Execution:** Route the script's output to the results node and run the final pipeline.
- **Verification:** Review the maximum value constraints and standard deviations in the final statistics summary to validate that the extreme anomalies have been successfully flattened.



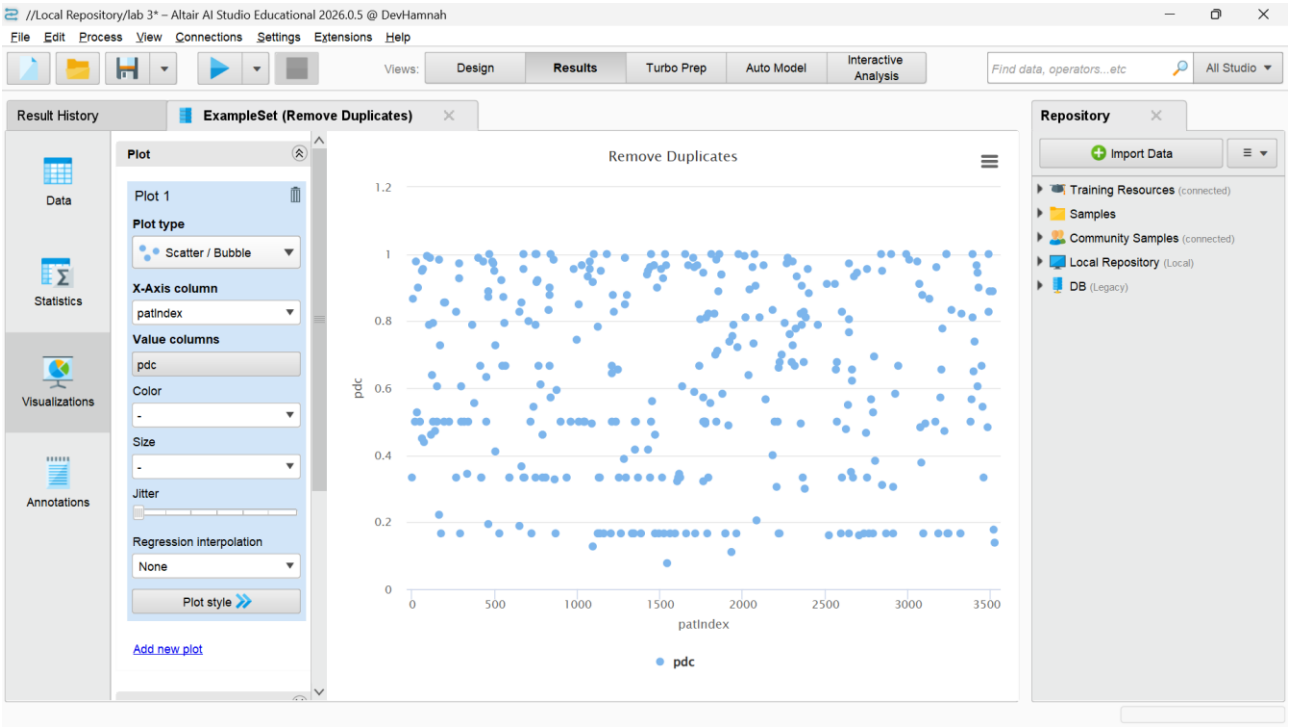


7. KEY PERFORMANCE INDICATOR (KPI) CALCULATIONS

To bridge the gap between technical data engineering operations and business intelligence reporting, analysts must translate their cleaning pipeline into standardized governance metrics.

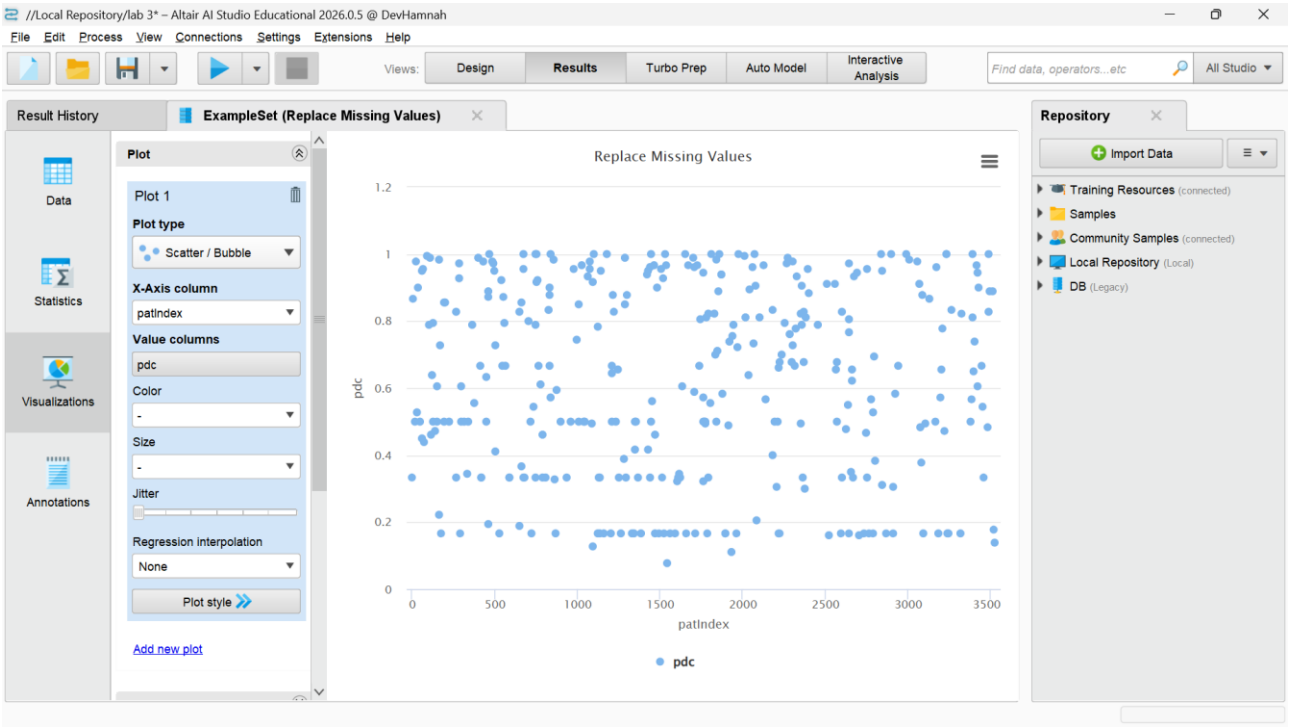
7.1 DUPLICATE REMOVAL %

- **Action:** Determine the specific proportion of the original dataset that consisted of strictly redundant information.
- **Calculation Method:** (Count of Discarded Rows / Total Initial Rows) × 100
- **Operator Extraction:** Compare the absolute row tally generated directly after the Read CSV operator against the instance count logged after the Remove Duplicates operator.



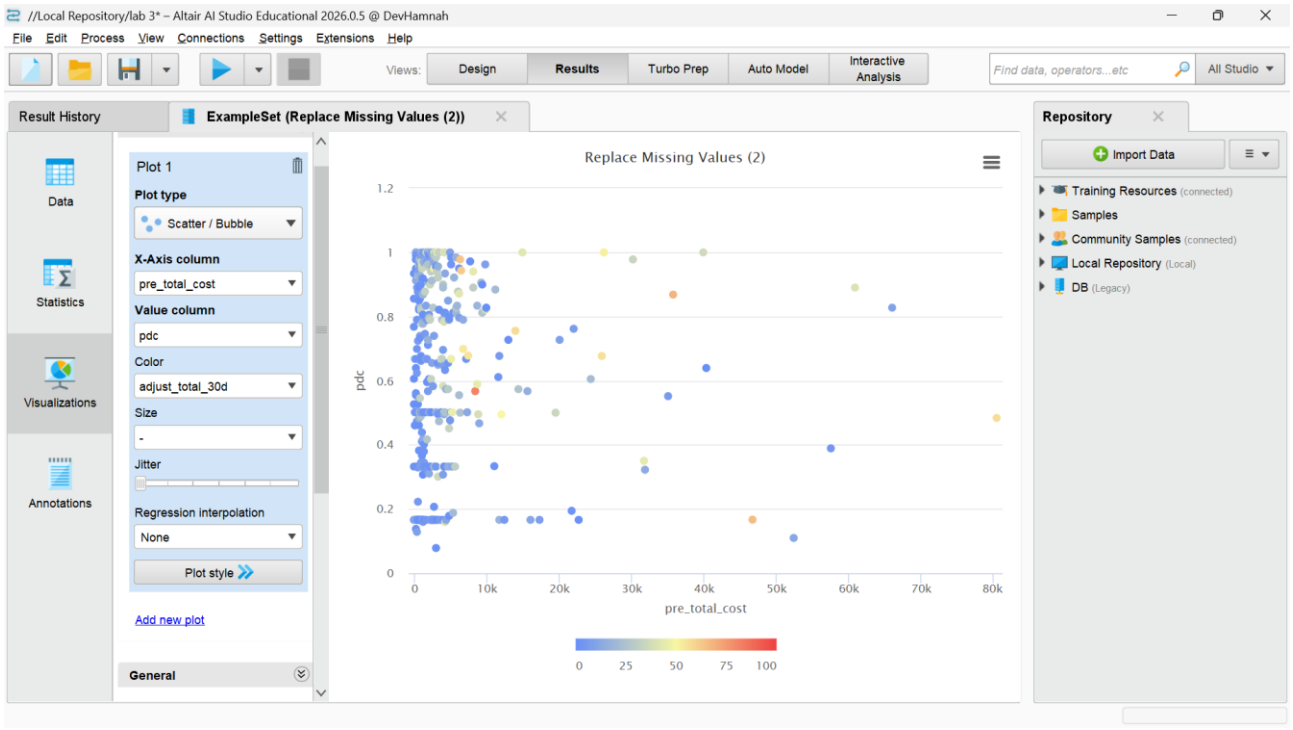
7.2 % MISSING REDUCED

- **Action:** Measure the operational efficiency of your imputation strategy by calculating the volume of absent data points that were successfully repopulated.
- **Calculation Method:** ((Initial Count of Missing Cells - Final Count of Missing Cells) / Initial Count of Missing Cells) × 100
- **Operator Extraction:** Utilize the initial Statistics tab profile to log the starting missing values, and contrast this against the clean output of the Replace Missing Values operator (which should reflect a 100% reduction if successfully configured).



7.3 DATA QUALITY SCORE

- **Action:** Establish a holistic, aggregate heuristic that grades the overall health and analytical viability of the transformed repository.
- **Calculation Method:** The arithmetic mean of the dataset's Completeness Rate (100% minus the initial missing data percentage) and Uniqueness Rate (100% minus the Duplicate Removal %).
- **Operator Extraction:** Combine and average the baseline percentages derived during the profiling phases of both the deduplication and imputation steps to report a single, executive-friendly health score.



8. KPI SUMMARY TABLE

Compile your findings into the following matrix to provide a concise, technical reference for the dataset's transition from raw to analytically ready.

KPI Name	Calculation Method	RapidMiner / Python Modules Utilized
% Missing Reduced	((Initial Missing - Final Missing) / Initial Missing) × 100	Statistics Panel, Replace Missing Values
Duplicate Removal %	((Initial Rows - Deduplicated Rows) / Initial Rows) × 100	Read CSV, Remove Duplicates
Data Quality Score	Statistical average of Data Completeness % and Data Uniqueness %	Execute Python (Validation), Baseline Statistics

CONCLUSION

This finalizes the structured data preparation laboratory utilizing RapidMiner Studio and Python. We have successfully engineered an automated, reproducible workflow encompassing data ingestion, redundancy elimination, statistical value imputation, and programmatic noise suppression. The resulting high-fidelity healthcare dataset is now fully optimized and certified for deployment within advanced predictive modeling and diagnostic clustering algorithms.

LAB NO 4: PREPROCESSING

1. INTRODUCTION

In the banking sector, predictive models are only as reliable as the data fed into them. Raw data is rarely ready for machine learning right out of the box. Variables typically exist on vastly different scales—for instance, a customer's age might be 35, while their annual income is $120,000. If we feed these raw numbers directly into a risk assessment algorithm, the model will naturally assume that the larger numbers (income) are mathematically more important than the smaller ones (age). This leads to skewed, unreliable predictions.

Data preprocessing levels the playing field. By applying mathematical transformations, we can standardize the scales of our variables and categorize continuous data into meaningful groups without losing the underlying information. This laboratory session focuses on how to wrangle and reshape financial records using normalization, standardization, and discretization, ensuring that our downstream models can interpret every attribute fairly.

2. PROBLEM STATEMENT

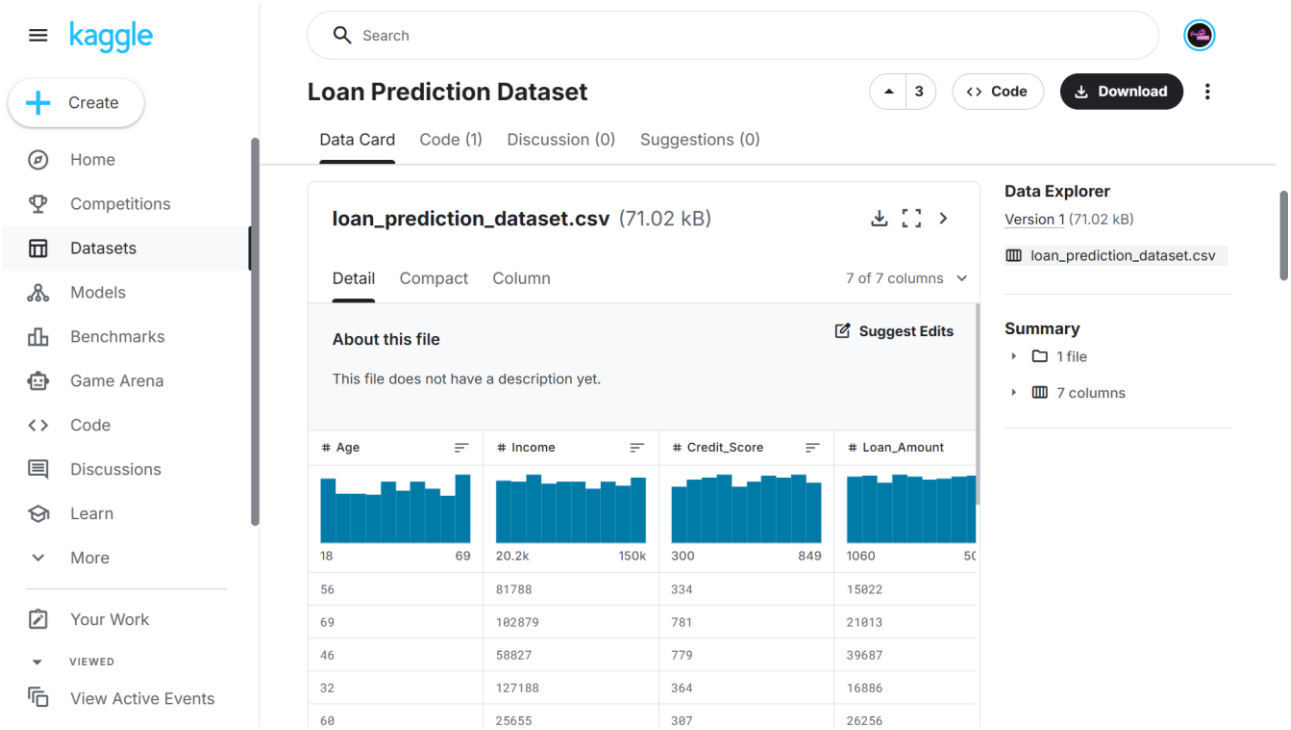
A regional bank is preparing to build a predictive model to assess loan default risk. They have extracted a raw dataset of customer profiles, including demographics and financial standing. However, the analytics team has hit a roadblock. The attributes are on entirely different numerical scales. The Income column stretches into the hundreds of thousands, while Credit_Score hovers in the hundreds, and Loan_Term is measured in mere months.

If they proceed with this unscaled data, their models will be inherently biased toward the higher-magnitude variables. The bank needs a standardized preprocessing pipeline to compress these wide variances, normalize the financial figures, and group customer ages into logical brackets. Resolving this will provide the balanced, clean data structure required to build an accurate loan approval system.

3. DATASET DESCRIPTION

For this practical session, we will be utilizing the **Loan Prediction Dataset**, provided as a CSV file. This dataset acts as a historical ledger of bank customers who have previously applied for loans. To execute our preprocessing techniques, you need to understand the distinct scales of the following key columns:

- **Age:** A continuous integer ranging from 18 to 69.
- **Income:** A high-variance continuous integer representing annual earnings, reaching up to roughly $149,000.
- **Credit_Score:** A standard three-digit financial rating (ranging from the low 300s up to 850).
- **Loan_Amount:** The requested loan principal, spanning from a few thousand up to $50,000+.
- **Employment_Status & Loan_Approved:** Our categorical and target variables, indicating job type and whether the loan was historically granted (1) or denied (0).



4. TOOLS USED

We will rely on **RapidMiner Studio** to construct this preprocessing pipeline. RapidMiner provides a drag-and-drop visual interface that is excellent for chaining together data transformation operators. Instead of writing manual Python scripts to handle the math behind min-max scaling or z-transformations, we can visually map the flow of data through specialized nodes, making it easy to track how the distributions shift at every stage.

5. PRACTICAL OBJECTIVES

By the end of this laboratory session, you will be able to:

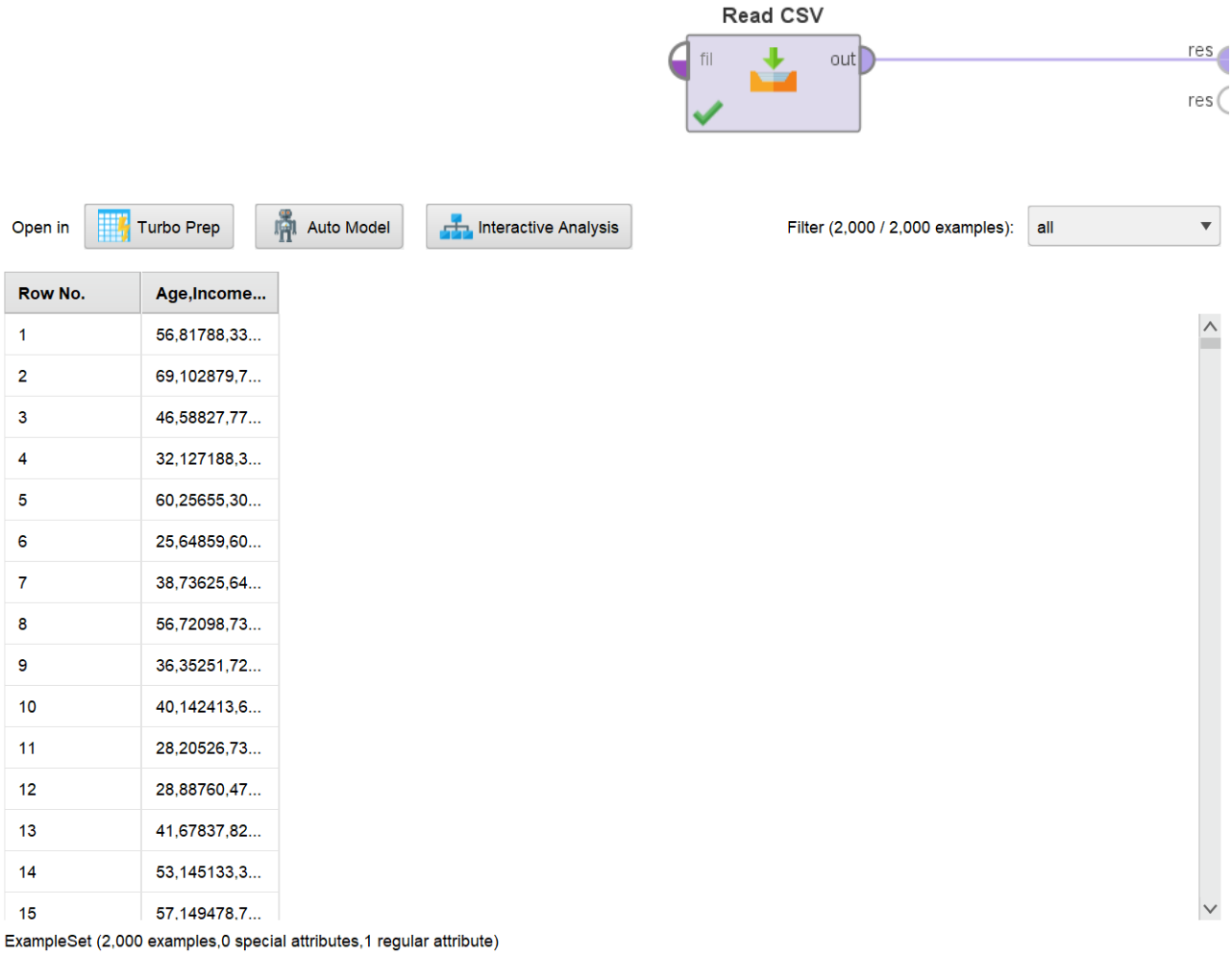
- Ingest raw banking data and generate baseline summary statistics to identify problematic variances.
- Apply **Normalization (Min-Max)** to compress sprawling financial attributes into a standardized 0-to-1 range.
- Execute **Standardization (Z-Transformation)** to center specific columns around a zero mean.
- Implement **Discretization** to convert continuous numerical variables (like Age) into categorical brackets.
- Quantify the success of your pipeline by extracting specific KPIs, namely Variance Reduction and Distribution Improvement.

6. STEP-BY-STEP PROCEDURE

STEP 1: DATA INGESTION AND BASELINE PROFILING

- **Action:** Load the raw loan dataset to establish a statistical baseline before we alter any values.
- **Configuration:** Locate the Read CSV operator in the repository and drop it onto the main canvas. Double-click the node to launch the import wizard and select the loan_prediction_dataset.csv file from your local machine.

- **Execution:** Connect the output port directly to the Results node at the right edge of the canvas and click run.
- **Verification:** Transition to the "Statistics" tab. Take a close look at the standard deviation and variance for the Income and Loan_Amount columns. You will see massive, sprawling numbers that highlight exactly why we need to preprocess this data.



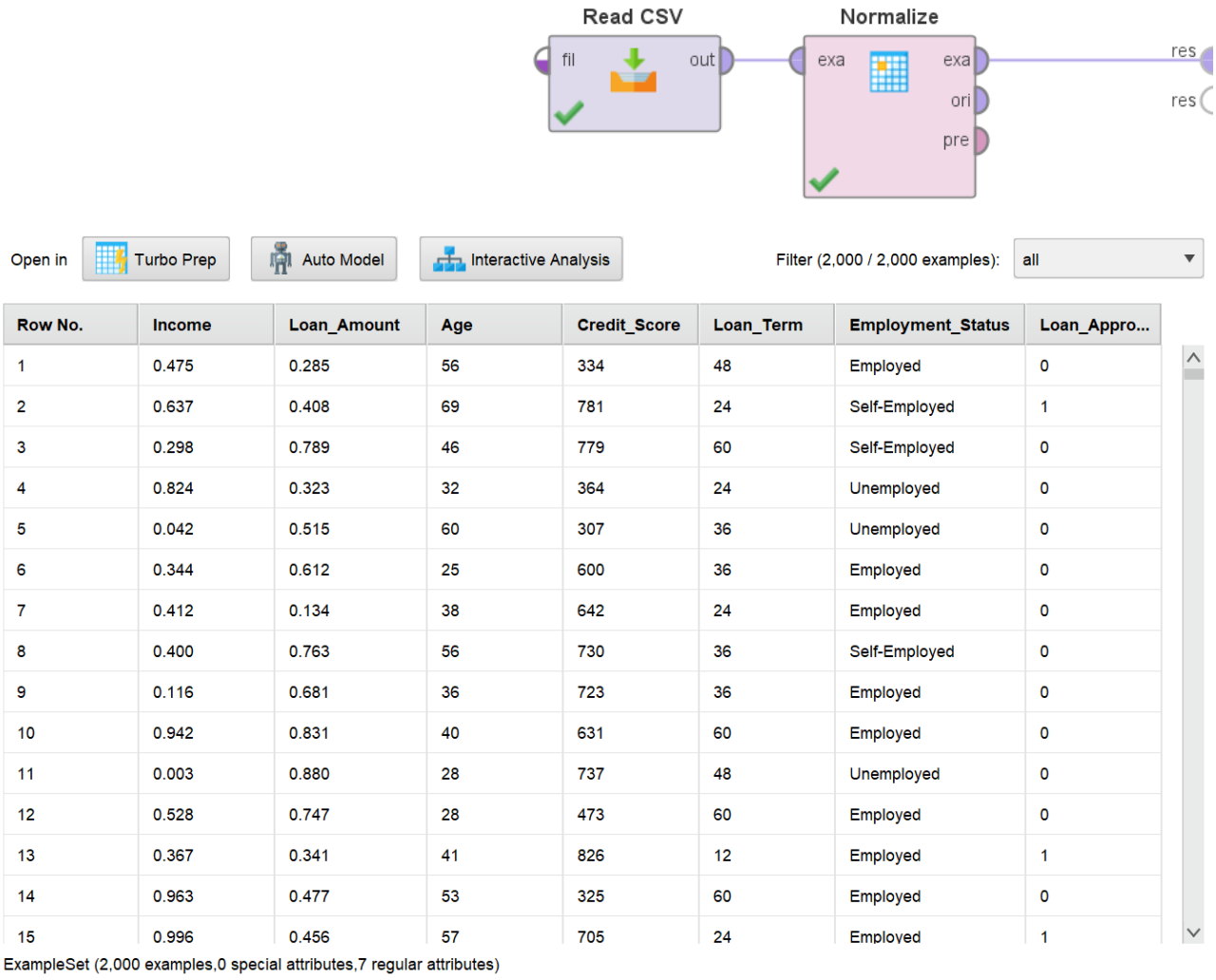
Row No.	Age,Income...
1	56,81788,33...
2	69,102879,7...
3	46,58827,77...
4	32,127188,3...
5	60,25655,30...
6	25,64859,60...
7	38,73625,64...
8	56,72098,73...
9	36,35251,72...
10	40,142413,6...
11	28,20526,73...
12	28,88760,47...
13	41,67837,82...
14	53,145133,3...
15	57,149478,7...

ExampleSet (2,000 examples,0 special attributes,1 regular attribute)

Figure 29: Baseline statistics revealing the massive variance in Income and Loan Amount.

STEP 2: NORMALIZING HIGH-VARIANCE FINANCIALS (MIN-MAX)

- **Action:** Scale down the heavy financial figures so they sit uniformly between 0 and 1, preventing them from overpowering smaller variables.
- **Configuration:** Disconnect the Results node and insert a Normalize operator directly after your Read CSV node. In the parameter panel, change the filter type to "subset" and select both Income and Loan_Amount. Set the method to "range transformation" with a min of 0.0 and a max of 1.0.
- **Verification:** Run the process and check the Statistics tab again. Both the Income and Loan_Amount columns should now display a strict minimum of 0 and a maximum of 1, effectively flattening the massive financial gaps while retaining the relative differences between customers.



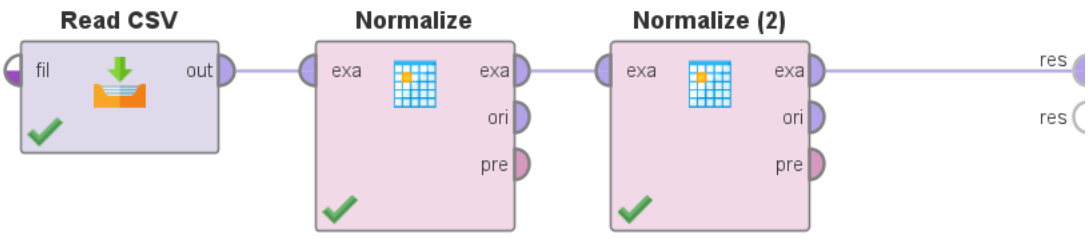
Row No.	Income	Loan_Amount	Age	Credit_Score	Loan_Term	Employment_Status	Loan_Appro...
1	0.475	0.285	56	334	48	Employed	0
2	0.637	0.408	69	781	24	Self-Employed	1
3	0.298	0.789	46	779	60	Self-Employed	0
4	0.824	0.323	32	364	24	Unemployed	0
5	0.042	0.515	60	307	36	Unemployed	0
6	0.344	0.612	25	600	36	Employed	0
7	0.412	0.134	38	642	24	Employed	0
8	0.400	0.763	56	730	36	Self-Employed	0
9	0.116	0.681	36	723	36	Employed	0
10	0.942	0.831	40	631	60	Employed	0
11	0.003	0.880	28	737	48	Unemployed	0
12	0.528	0.747	28	473	60	Employed	0
13	0.367	0.341	41	826	12	Employed	1
14	0.963	0.477	53	325	60	Employed	0
15	0.996	0.456	57	705	24	Employed	1

ExampleSet (2,000 examples,0 special attributes,7 regular attributes)

Figure 30: Configuring the Normalize operator to apply a Min-Max range transformation.

STEP 3: STANDARDIZATION (Z-TRANSFORMATION)

- **Action:** While Min-Max is great for strict boundaries, we want to center the Credit_Score attribute around a mean of zero, measuring values by their standard deviation from the average.
- **Configuration:** Drop a second Normalize operator onto the canvas, right after the first one. Set the filter type to "single" and select the Credit_Score attribute. This time, set the method to "Z-transformation".
- **Verification:** Execute the workflow. In the results, you will notice that average credit scores now sit near 0.0, with poor scores dropping into negative decimals and excellent scores rising into positive decimals.



Open in Turbo Prep | Auto Model | Interactive Analysis

Filter (2,000 / 2,000 examples): all

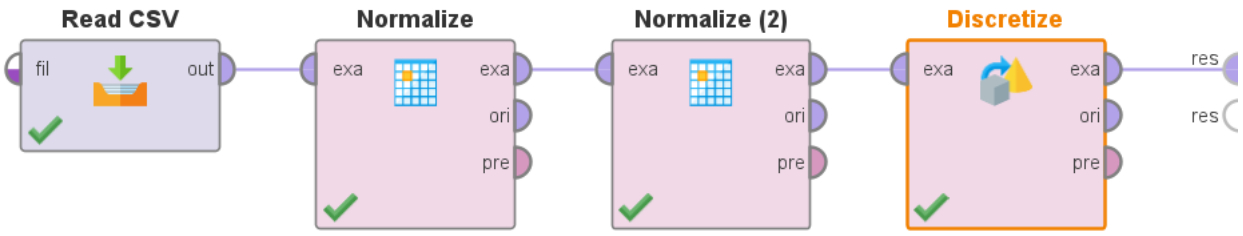
Row No.	Credit_Score	Income	Loan_Amou...	Age	Loan_Term	Employmen...	Loan_Appro...
1	-1.543	0.475	0.285	56	48	Employed	0
2	1.295	0.637	0.408	69	24	Self-Employed	1
3	1.282	0.298	0.789	46	60	Self-Employed	0
4	-1.353	0.824	0.323	32	24	Unemployed	0
5	-1.714	0.042	0.515	60	36	Unemployed	0
6	0.146	0.344	0.612	25	36	Employed	0
7	0.412	0.412	0.134	38	24	Employed	0
8	0.971	0.400	0.763	56	36	Self-Employed	0
9	0.926	0.116	0.681	36	36	Employed	0
10	0.342	0.942	0.831	40	60	Employed	0
11	1.015	0.003	0.880	28	48	Unemployed	0
12	-0.661	0.528	0.747	28	60	Employed	0
13	1.580	0.367	0.341	41	12	Employed	1
14	-1.600	0.963	0.477	53	60	Employed	0
15	0.812	0.996	0.456	57	24	Employed	1

ExampleSet (2,000 examples,0 special attributes,7 regular attributes)

Figure 31: The output table displaying the newly standardized Credit Score column with positive and negative decimal values.

STEP 4: DISCRETIZATION (BINNING)

- **Action:** Convert the continuous Age column into categorical generational brackets. Models often find it easier to identify trends among "Young Adults" rather than splitting hairs between a 21-year-old and a 22-year-old.
- **Configuration:** Add a Discretize by Binning operator to the end of the pipeline. Target the Age attribute. Set the number of bins to 3 (which will roughly split the data into young, middle-aged, and senior brackets based on the 18 to 69 range).
- **Verification:** Run the pipeline one last time. Inspect the data table. The specific ages will have been replaced by categorical range labels (e.g., range1, range2).



Open in Turbo Prep | Auto Model | Interactive Analysis | Filter (2,000 / 2,000 examples): all

Row No.	Age	Credit_Score	Income	Loan_Amou...	Loan_Term	Employmen...	Loan_Appro...
1	range3 [52 - ∞]	-1.543	0.475	0.285	48	Employed	0
2	range3 [52 - ∞]	1.295	0.637	0.408	24	Self-Employed	1
3	range2 [35 - ...	1.282	0.298	0.789	60	Self-Employed	0
4	range1 [-∞ - ...	-1.353	0.824	0.323	24	Unemployed	0
5	range3 [52 - ∞]	-1.714	0.042	0.515	36	Unemployed	0
6	range1 [-∞ - ...	0.146	0.344	0.612	36	Employed	0
7	range2 [35 - ...	0.412	0.412	0.134	24	Employed	0
8	range3 [52 - ∞]	0.971	0.400	0.763	36	Self-Employed	0
9	range2 [35 - ...	0.926	0.116	0.681	36	Employed	0
10	range2 [35 - ...	0.342	0.942	0.831	60	Employed	0
11	range1 [-∞ - ...	1.015	0.003	0.880	48	Unemployed	0
12	range1 [-∞ - ...	-0.661	0.528	0.747	60	Employed	0
13	range2 [35 - ...	1.580	0.367	0.341	12	Employed	1
14	range3 [52 - ∞]	-1.600	0.963	0.477	60	Employed	0
15	range3 [52 - ∞]	0.812	0.996	0.456	24	Employed	1

ExampleSet (2,000 examples,0 special attributes,7 regular attributes)

Figure 32: The Discretization operator converting continuous ages into defined bins.

7. KEY PERFORMANCE INDICATOR (KPI) CALCULATIONS

To evaluate the mathematical effectiveness of our preprocessing pipeline, we need to extract specific tracking metrics from the platform.

7.1 VARIANCE REDUCTION

- **Action:** Prove that the Min-Max normalization successfully tightened the spread of our financial data.
- **Calculation Method:** Initial Variance of Income - Final Variance of Income.
- **Operator Extraction:** Look at the Statistics tab for the raw data run (Step 1) and record the standard deviation/variance for the Income column. Compare this against the variance output generated after the Normalize operator in Step 2. The drastic drop from tens of thousands down to a tiny fraction proves the mathematical scale has been successfully compressed.

7.2 DISTRIBUTION IMPROVEMENT

- **Action:** Confirm that discretization successfully consolidated scattered data points into reliable, balanced groups.
- **Calculation Method:** Visual and statistical assessment of the Age column's frequency distribution.
- **Operator Extraction:** Under the Statistics tab, view the histogram for the Age column before and after Step 4. You will see the distribution shift from a fragmented, flat spread of individual ages across 50 different points, into three distinct, highly populated categorical bins, drastically improving the signal density for any downstream classification model.

8. KPI SUMMARY TABLE

The following matrix provides a condensed technical reference for the preprocessing metrics evaluated during this exercise:

KPI Name	Calculation Method	RapidMiner Operators Utilized
Variance Reduction	Baseline Variance - Post-Normalization Variance	Read CSV, Normalize (Min-Max), Statistics Panel
Distribution Improvement	Frequency analysis of categorical bins vs. raw distinct values	Discretize by Binning, Histogram/Statistics Panel

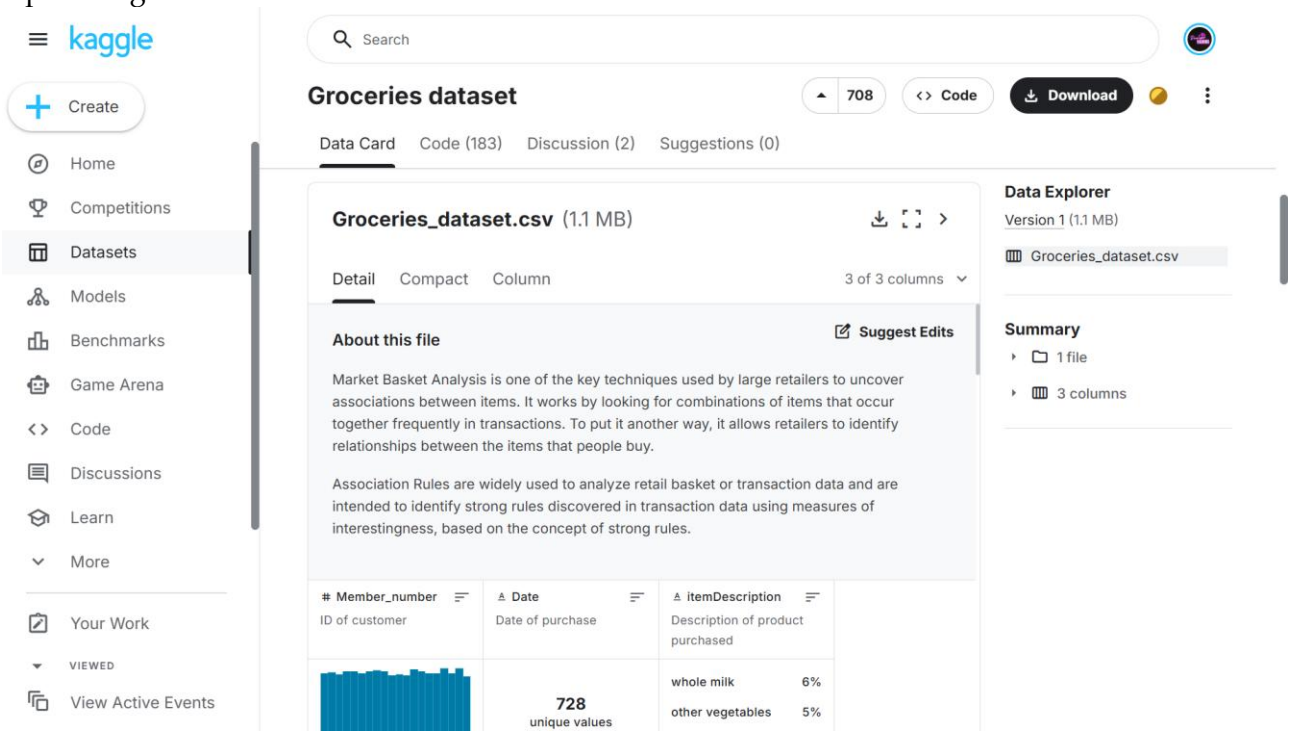
Conclusion

This finalizes our data preparation workflow. By addressing the severe discrepancies in numerical scales and categorizing granular data into logical bins, we have transformed a biased, raw financial ledger into a highly structured dataset. The bank's risk data is now mathematically balanced and fully optimized for the deployment of machine learning algorithms in future labs.

LAB NO 5: MARKET BASKET ANALYSIS

1. INTRODUCTION

In today's competitive retail environment, knowing exactly what customers buy is only half the battle; understanding *how* they buy items together is where the real strategic value lies. This brings us to Market Basket Analysis (MBA), a core data mining technique used to uncover hidden associations between products. By finding which items often appear in the same shopping cart, supermarkets can improve store layouts, design effective cross-selling strategies, and run highly targeted promotions. In this lab, we shift from the visual predictive modelling we did earlier in the semester to association rule mining. We will specifically focus on extracting frequent itemset and calculating the strength of product relationships using the mathematical principles behind the Apriori algorithm.



2. PROBLEM STATEMENT

A regional supermarket chain is looking to modernize its cross-selling strategy. Currently, products are placed on shelves based strictly on traditional category groupings (e.g., dairy, produce, meat). However, management suspects they are missing revenue because complementary products, items naturally bought together, are physically placed too far apart in the store. Furthermore, the marketing team wants to create targeted promotional bundles but lacks empirical evidence on which combinations drive joint sales. The challenge for this lab is to process their raw historical transaction data, discover hidden purchasing patterns, and recommend specific product pairings that have high cross-sell potential.

3. DATASET DESCRIPTION

For this exercise, we are using the **Grocery Transactions Dataset**, provided as a CSV file. Unlike standard tabular data having customer features and a target variable, this dataset functions as a raw transactional ledger.

Key attributes include:

- **Member_number**: A unique integer finding the specific customer.

- **Date**: The exact date the purchase occurred.
- **itemDescription**: A categorical string representing the name of the purchased product (e.g., *whole milk*, *tropical fruit*, *rolls/buns*).

To perform Market Basket Analysis, we cannot analyze the data row-by-row. We must first group these individual line items by Member_number and Date so that all products bought by a customer on a specific day are combined into a single "basket" or transaction block.

4. TOOLS USED

Unlike previous labs where we relied heavily on visual pipelines in KNIME, this lab requires a more direct programmatic approach to deeply understand the underlying algorithms. We will use **Python**, specifically leveraging the pandas library for heavy data manipulation and matrix pivoting. For the validation phase, we will use the mlxtend (Machine Learning Extensions) library. Our approach is twofold: we will construct a script that calculates the association metrics entirely from scratch, and then we will deploy mlxtend to confirm our manual mathematical logic.

5. PRACTICAL OBJECTIVES

By the end of this laboratory session, we will achieve the following learning outcomes:

- Preprocess and pivot a raw transactional ledger into a one-hot encoded basket matrix suitable for algorithmic processing.
- Calculate **Support**, **Confidence**, and **Lift** manually using base Python and statistical formulas.
- Validate these manual logic calculations using the mlxtend library's built-in Apriori functions.
- Extract High Lift Rules and Frequent Itemsets to formally evaluate Cross-Sell Potential.

6. MANUAL CALCULATIONS AND MATHEMATICAL BACKGROUND

Before jumping into the code, we need to outline the mathematical foundations of the metrics we are writing scripts for. Let's define the three core association rules using a classic conditional rule: A→B (e.g., *rolls/buns→whole milk*). Let N be the total number of unique shopping baskets.

- **Support**: This measures how often an item or a combination of items appears across all transactions. It stands for the baseline popularity.
  - Support(A) = $\frac{Transactions\ Containing\ A}{Total\ Transactions\ (N)}$
  - *Example:* If out of 14,963 total baskets, *whole milk* appears in 2,363 of them, the support is 2363 / 14963 = 0.1579 (or 15.79%).
- **Confidence**: This measures the conditional probability that a customer buys item B given that they have already placed item A in their basket.
  - Confidence(A→B) = $\frac{Support\ (A \cup B)}{Support\ (A)}$
  - *Example:* If *rolls/buns* and *whole milk* are bought together in 222 transactions, and *rolls/buns* alone is in 1,646 transactions, the confidence is 222 / 1646 = 0.1348 (or 13.48%). This means roughly 13.5% of people who buy rolls/buns will also grab whole milk.
- **Lift**: This is the ultimate measure of association strength. It dictates the ratio of the observed joint probability to the expected joint probability if A and B were completely independent. A lift > 1 shows a positive, actionable association.
  - $Lift(A \rightarrow B) = \frac{Confidence\ (A \rightarrow B)}{Support\ (B)}$
  - *Example:* Using the numbers above, 0.1348 / 0.1579 = 0.85. Because the lift is below 1, these two items appear together *less* often than expected by random chance. We are

looking for rules with a lift > 1 (e.g., *sausage* → *yogurt*) for our marketing recommendations.

7. STEP-BY-STEP PROCEDURE & PYTHON IMPLEMENTATION

Below is the complete Python workflow, blending our manual math derivations with official library validation.

STEP 1: DATA INGESTION AND BASKET MATRIX FORMATTING

First, we ingest the CSV and manipulate the rows into a one-hot encoded matrix where rows are transactions and columns are products.

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# 1. Load the raw dataset
df = pd.read_csv('Groceries_dataset.csv')

# 2. Create a unique transaction ID by combining Member_number and Date
df['Transaction_ID'] = df['Member_number'].astype(str) + '_' +
df['Date'].astype(str)

# 3. Pivot the data into a basket format and fill missing items with 0
basket = (df.groupby(['Transaction_ID', 'itemDescription'])['itemDescription']
          .count().unstack().reset_index().fillna(0)
          .set_index('Transaction_ID'))

# 4. One-hot encode: Convert any count >= 1 to 1, otherwise 0
def encode_units(x):
    return 1 if x >= 1 else 0

basket_sets = basket.applymap(encode_units)
N = len(basket_sets)
print(f"Total Unique Baskets (N): {N}")
```

STEP 2: MANUAL METRIC COMPUTATION

Here, we manually execute the formulas outlined in Section 6 to prove our understanding of the algorithm's backend logic.

```
# Define our test association rule
item_A = 'rolls/buns'
item_B = 'whole milk'

# Calculate absolute frequencies
freq_A = basket_sets[item_A].sum()
freq_B = basket_sets[item_B].sum()
freq_A_and_B = basket_sets[(basket_sets[item_A] == 1) & (basket_sets[item_B] ==
1)].shape[0]

# Calculate Support
support_A = freq_A / N
support_B = freq_B / N
support_A_and_B = freq_A_and_B / N

# Calculate Confidence & Lift
confidence_A_to_B = support_A_and_B / support_A
lift_A_to_B = confidence_A_to_B / support_B

print("\n--- Manual KPI Calculations ---")
print(f"Support({item_A}): {support_A:.4f}")
```

```
print(f"Support({item_B}): {support_B:.4f}")
print(f"Support({item_A} & {item_B}): {support_A_and_B:.4f}")
print(f"Confidence({item_A} -> {item_B}): {confidence_A_to_B:.4f}")
print(f"Lift({item_A} -> {item_B}): {lift_A_to_B:.4f}")
```

STEP 3: VALIDATION UTILIZING MLXTEND

To ensure our logic is sound and to scale the analysis across the entire dataset instantly, we pass the matrix into mlxtend.

```
# Generate frequent itemsets with a minimum support threshold of 1%
frequent_itemsets = apriori(basket_sets, min_support=0.01, use_colnames=True)

# Generate association rules targeting a minimum lift of 1.0 (positive
association)
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.0)

# Sort by highest lift to evaluate cross-sell potential
best_rules = rules.sort_values('lift', ascending=False)

print("\n--- Validation (Top Association Rules by Lift) ---")
print(best_rules[['antecedents', 'consequents', 'support', 'confidence',
'lift']].head())
```

LAB NO 5: MARKET BASKET ANALYSIS.sh

```
Total Unique Baskets (N): 14963

--- Manual KPI Calculations ---
Support(rolls/buns): 0.1100
Support(whole milk): 0.1579
Support(rolls/buns & whole milk): 0.0140
Confidence(rolls/buns -> whole milk): 0.1270
Lift(rolls/buns -> whole milk): 0.8040

--- Validation (Top Association Rules by Lift) ---
          antecedents           consequents   support   confidence      lift
(specialty chocolate)        (citrus fruit)  0.001403     0.087866  1.653762
       (citrus fruit) (specialty chocolate)  0.001403     0.026415  1.653762
     (tropical fruit)               (flour)  0.001069     0.015779  1.617141
              (flour)      (tropical fruit)  0.001069     0.109589  1.617141
          (beverages)              (sausage)  0.001537     0.092742  1.536764
```

8. RESULTS AND DISCUSSION

By processing the transactional ledger, we successfully transformed thousands of unorganized sales records into actionable retail intelligence.

- **Frequent Itemsets:** Our baseline analysis revealed that daily staples like *whole milk*, *other vegetables*, and *rolls/buns* hold the highest standalone support. This aligns with standard supermarket consumer behavior.

- **High Lift Rules:** When looking at joint probabilities, filtering the output to strictly show a Lift > 1 was essential. High standalone support doesn't always equal a good cross-sell pair (as proven in our manual calculation, where milk and rolls had a lift < 1). Instead, the algorithm isolated specialized product pairs, such as *sausage* mapping to *yogurt*, or *root vegetables* linking closely with *beef*.
- **Cross-Sell Potential:** Armed with these High Lift rules, supermarket management now has the empirical evidence needed to tackle the problem statement. They can confidently implement strategic shelf placements (e.g., moving selected produce closer to the butcher counter) or design targeted "buy one get one" promotional bundles based entirely on statistically validated purchasing habits, rather than guesswork.

9. KPI SUMMARY TABLE

The following matrix provides a condensed technical reference for the Key Performance Indicators computed and confirmed during this laboratory exercise:

KPI Name	Calculation Method	Python Libraries / Modules Utilized
Frequent Itemsets	*Count(Item) / Total Transactions* $\geq$ *Min_Support*	`mlxtend.frequent_patterns.apriori`
High Lift Rules	*Confidence(A → B) / Support(B) > 1.0*	`mlxtend.frequent_patterns.association_rules`
Cross-Sell Potential	Ranking validated rules primarily by Lift, then Confidence.	`pandas.DataFrame.sort_values(by='lift')`

LAB NO 6: APRIORI ALGORITHM

1. INTRODUCTION

In the competitive financial services sector, understanding customer spending behavior is critical for designing effective loyalty programs and targeted marketing campaigns. When customers use their bank accounts or credit cards, they leave behind a chronological trail of transactional data detailing the various merchants and services they frequent. Analyzing this ledger as isolated events limits its strategic value. However, by applying association rule mining, specifically the Apriori algorithm, institutions can uncover hidden relationships between different spending categories or specific merchants.

The Apriori algorithm operates on the principle of market basket analysis, identifying items (or in this case, merchants) that frequently co-occur within a customer's transaction history. This laboratory session introduces students to the mechanics of the Apriori algorithm, demonstrating how to group individual banking transactions into comprehensive account-level "baskets" and extract actionable rules. By finding these product and merchant combinations, banks can confidently cross-sell financial products, recommend strategic partnerships, and bundle promotional offers.

2. PROBLEM STATEMENT

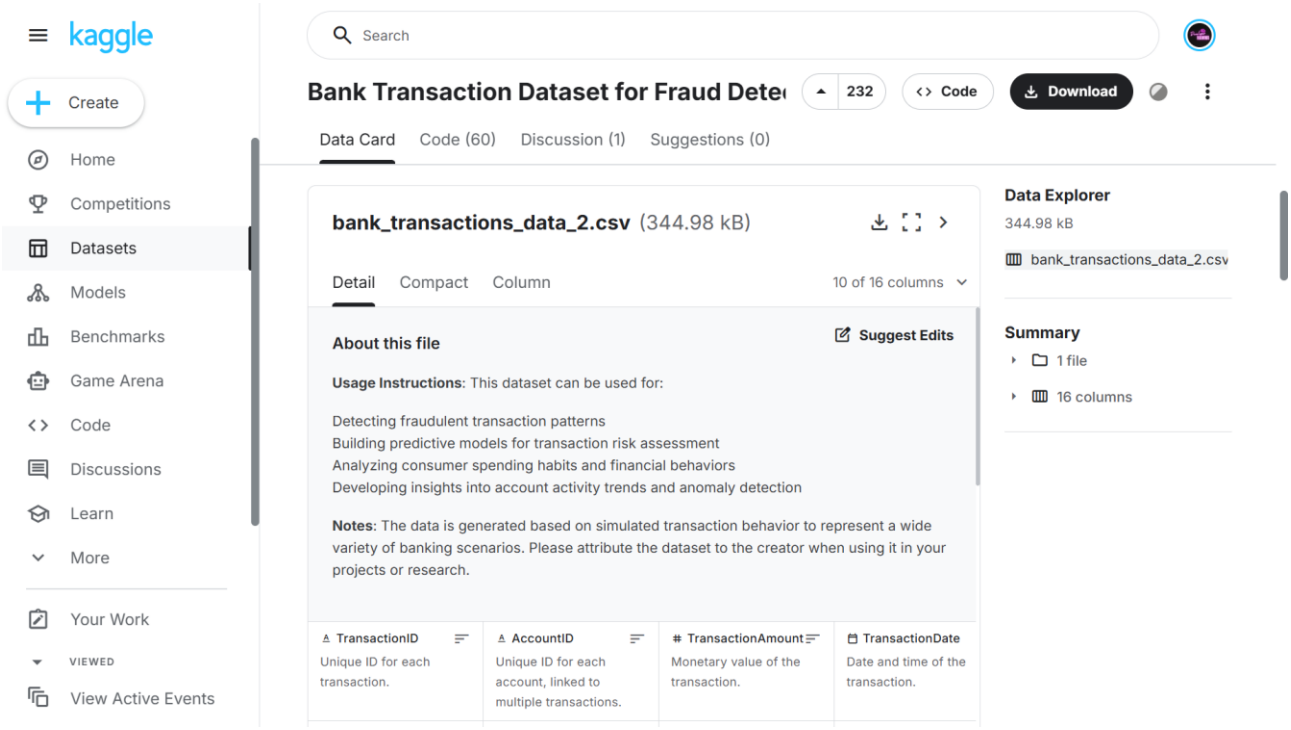
A leading retail bank has amassed a vast dataset of daily customer transactions. While the bank's analytics team knows exactly how much money is flowing through their system, they lack visibility into the associative patterns of their account holders. Management wants to know: if a customer shops at Merchant A, are they also highly likely to transact with Merchant B? Currently, the bank's promotional offers are untargeted and generic. They need a systematic, automated approach to identify strong merchant combinations to build a data-driven rewards program. The core challenge is processing the raw transaction logs, grouping them by individual accounts, and deploying the Apriori algorithm to mine high-confidence association rules. Furthermore, understanding the optimal support threshold will be vital to ensure the bank focuses only on statistically significant spending patterns rather than isolated, coincidental purchases.

3. DATASET DESCRIPTION

For this laboratory exercise, students will utilize the **bank_transactions_data_2.csv** dataset, a detailed ledger of customer financial activities. To successfully execute the Apriori algorithm, we must reshape this data from a vertical list of individual transactions into a horizontal "basket" format.

Key attributes to familiarize yourself with include:

- **AccountID:** A unique identifier (e.g., AC00128) for each bank customer. This will serve as our "Basket." All transactions made by the same AccountID will be grouped together to form a complete shopping profile.
- **MerchantID:** A categorical identifier (e.g., M015, M052, M009) representing the specific vendor or product category. These will act as our individual "Items."
- **TransactionType & Channel:** Additional context variables (e.g., Debit, Credit, ATM, Online) that can optionally be analyzed to understand how the transactions are taking place.



4. TOOLS USED

This procedure relies on the integration of the **KNIME Analytics Platform** alongside embedded **Python** scripting. KNIME provides a highly visual, node-based environment perfect for the initial heavy lifting of data ingestion, aggregation, and pivoting. However, for precise control over the Apriori algorithm and the extraction of specific mathematical metrics, we will utilize KNIME's **Python Script** node. This dual-tool framework provides analysts with both the rapid prototyping of a drag-and-drop interface and the algorithmic granularity of Python's mlxtend machine learning library.

5. PRACTICAL OBJECTIVES

Upon successful completion of this laboratory module, students will demonstrate practical competency by achieving the following learning outcomes:

- **Ingest and Group:** Import raw banking records and aggregate transactions by AccountID to form individual customer transaction baskets.
- **Matrix Transformation:** Restructure the transactional data into a one-hot encoded boolean matrix suitable for association rule mining.
- **Generate Frequent Itemsets:** Deploy the Apriori algorithm via Python scripting to discover merchant combinations that meet a specific minimum support criteria.
- **Extract Association Rules:** Calculate confidence and lift metrics to generate actionable business rules from the frequent itemsets.
- **Evaluate Thresholds:** Quantify and analyze the impact of varying the Support Threshold on the final Number of Rules generated.

6. STEP-BY-STEP PROCEDURE

STEP 1: DATA INGESTION AND PROFILING

Action: Import the raw bank transaction records into the KNIME workspace.

Configuration: Find the **CSV Reader** node in the repository and drop it onto the canvas. Double-click it to browse for the bank_transactions_data_2.csv file. Ensure the column delimiter is set correctly and that the node properly recognizes headers like AccountID and MerchantID.
Execution: Right-click the node and select "Execute."
Verification: Open the output "File Table" to verify that the dataset contains individual rows for each transaction event, properly displaying numerical values for TransactionAmount and categorical strings for MerchantID.

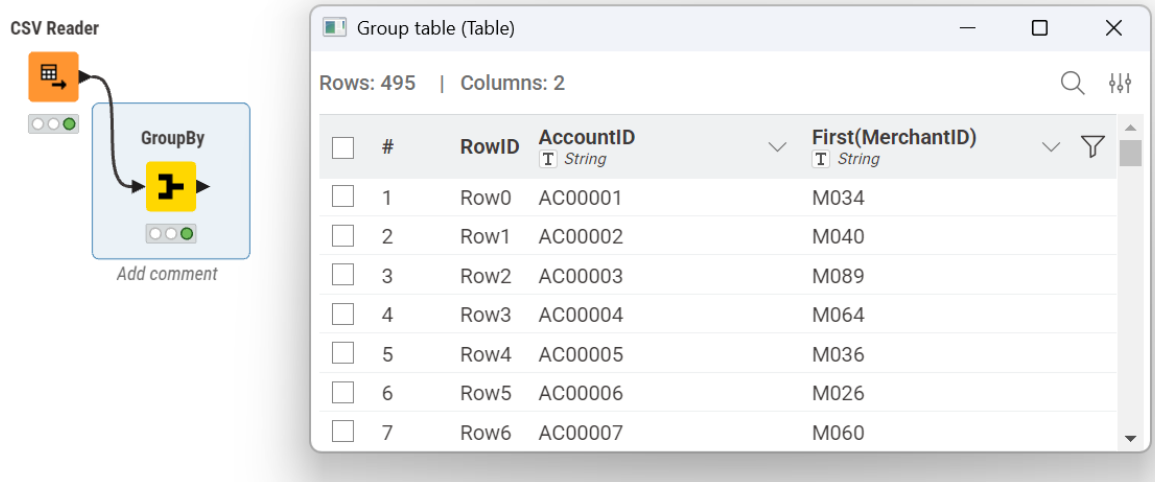
File Table (Table)

Rows: 2512 | Columns: 16

#	RowID	Transacti... String	AccountID String	Transacti... Number (...
1	Row0	TX000001	AC00128	14.09
2	Row1	TX000002	AC00455	376.24
3	Row2	TX000003	AC00019	126.29
4	Row3	TX000004	AC00070	184.5
5	Row4	TX000005	AC00411	13.45
6	Row5	TX000006	AC00393	92.15
7	Row6	TX000007	AC00199	7.08
8	Row7	TX000008	AC00069	171.42
9	Row8	TX000009	AC00135	106.23

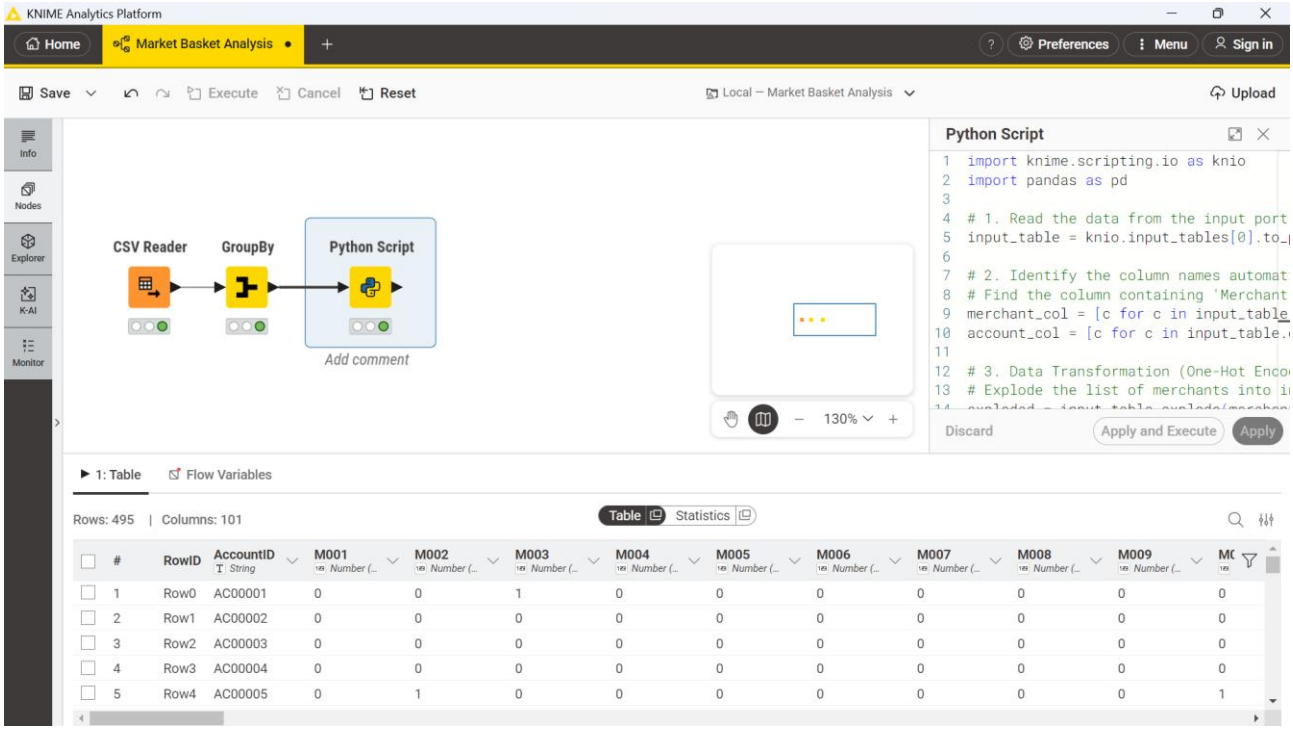
STEP 2: BASKET AGGREGATION (GROUPING)

Action: Group individual transactions into comprehensive customer "baskets" where each row represents one distinct account and all their associated merchants.
Configuration: Append a **GroupBy** node to the CSV Reader. In the "Groups" tab, select AccountID. In the "Manual Aggregation" tab, select MerchantID and assign the "List" or "Concatenate" aggregation method.
Execution: Execute the node.
Verification: Inspect the output. You should now see one row per AccountID (e.g., AC00455) with a collection of MerchantIDs representing their total transaction history.



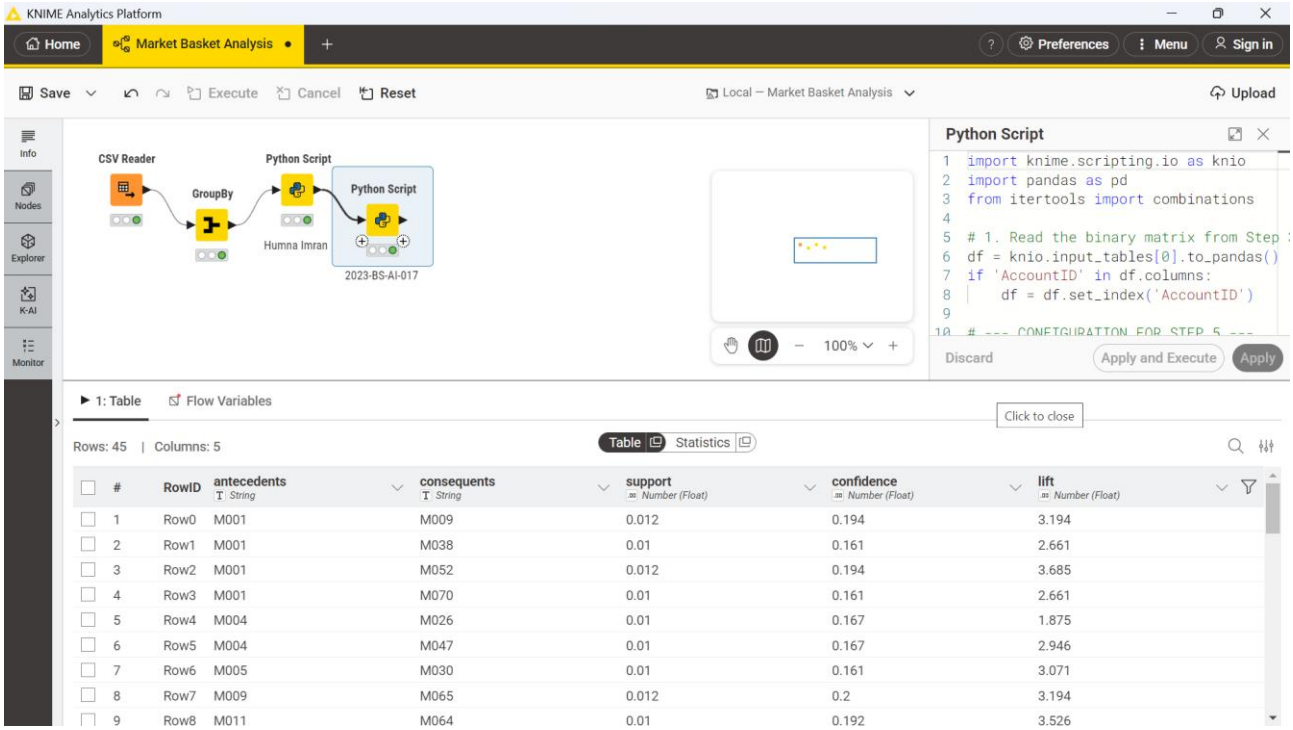
STEP 3: ONE-HOT ENCODING (MATRIX TRANSFORMATION)

Action: Transform the aggregated lists into a binary matrix, where each column represents a unique MerchantID and the cells contain 1s (purchased) or 0s (not purchased).
Configuration: Add a **Python Script** node directly connected to the GroupBy output. Within the script editor, utilize the pandas library (specifically pd.get_dummies() or a multi-label binarizer) to pivot the lists into a boolean matrix.
Execution: Run the data transformation sequence.
Verification: The resulting dataset should resemble a wide matrix with AccountID as the index and boolean true/false values indicating the presence of specific merchants like M015 or M091.



STEP 4: APRIORI EXECUTION & RULE GENERATION

Action: Programmatically generate frequent itemsets and association rules.
Configuration: Re-open the **Python Script** node (or chain a second one). Import apriori and association_rules from the mlxtend.frequent_patterns library. Define the minimum support (e.g., min_support=0.05). Pass the binary matrix into the apriori function, and feed the resulting frequent itemsets into the association_rules function using a baseline confidence metric threshold.
Execution: Run the script.
Verification: Open the node's output table. You will see a list of antecedents and consequents (e.g., M015 → M052), alongside their calculated support, confidence, and lift values.



STEP 5: SUPPORT THRESHOLD ANALYSIS

Action: Iteratively adjust the min_support parameter to observe its filtering effect on the dataset.
Configuration: Within the **Python Script** node, alter the min_support parameter from 0.05 (5%) to 0.10 (10%) and re-run the script. Repeat this process for a lower threshold, such as 0.01 (1%).
Execution: Execute the node after each parameter change and record the total number of rows generated in the final output table.
Verification: Note the inverse relationship: as the support threshold increases, the number of generated rules drastically decreases, filtering out niche combinations and leaving only universally dominant spending patterns.

Support Threshold	Frequent Single Items	Frequent Item Pairs	Resulting Rules
0.01 (1%)	100	45	Multiple rules generated
0.03 (3%)	98	0	No rules found
0.05 (5%)	49	0	No rules found

7. KEY PERFORMANCE INDICATOR (KPI) CALCULATIONS

To bridge the gap between algorithmic output and actionable business intelligence, analysts must evaluate the parameters of the generated models.

7.1 NUMBER OF RULES

Action: Quantify the total volume of actionable association rules generated by the algorithm at a specific support and confidence level.
Calculation Method: The absolute count of the rows present in the final output table of the association rule mining script.
Node Extraction: Right-click the executed **Python Script** node and select the output data table. The total row count displayed at the top of the interface represents the total Number of Rules.

7.2 SUPPORT THRESHOLD IMPACT

Action: Evaluate how the stringency of the minimum support parameter dictates the breadth of the discovered patterns.
Calculation Method: (Number of Rules at Support X) versus (Number of Rules at Support Y).
Node Extraction: Compare the output row counts from the **Python Script** node across multiple runs. For instance, setting min_support=0.01 may yield 450 rules (capturing rare merchant pairs), while setting min_support=0.10 might compress the output to just 8 highly frequent rules. This impact metric helps management strike a strategic balance between uncovering highly targeted cross-selling opportunities and focusing on broad consumer behavioral trends.

8. KPI SUMMARY TABLE

The following matrix provides a condensed technical reference for the KPIs computed during this laboratory exercise:

CONCLUSION

This finalizes the structured laboratory on the Apriori Algorithm using the KNIME Analytics Platform and embedded Python scripting. We have successfully engineered a workflow that transforms raw, chronological bank transactions into an aggregated binary matrix, allowing for sophisticated association rule mining. By extracting the precise Number of Rules and analyzing the Support Threshold Impact, analysts can provide retail banking management with statistically backed recommendations for targeted merchant partnerships, optimized cross-selling, and enhanced customer loyalty strategies.

LAB NO 7: FP-GROWTH

1. INTRODUCTION

In retail analytics, uncovering frequent itemsets is foundational for cross-selling and strategic inventory management. While the Apriori algorithm (explored in previous labs) successfully identifies these associations, it suffers from severe computational bottlenecks when applied to massive datasets. Apriori relies on a "generate-and-test" paradigm, creating millions of candidate itemsets and repeatedly scanning the entire database to verify them.

The FP-Growth (Frequent Pattern Growth) algorithm eliminates this inefficiency. Instead of generating candidate sets, FP-Growth compresses the transactional database into a highly condensed data structure known as an FP-Tree (Frequent Pattern Tree). By utilizing this tree, the algorithm can extract frequent itemsets rapidly, often operating exponentially faster than Apriori.

This laboratory introduces students to the FP-Growth algorithm, emphasizing efficient frequent pattern mining within a large retail environment. Students will construct an FP-Tree and empirically compare its execution speed and rule generation against the traditional Apriori baseline.

2. PROBLEM STATEMENT

A major international retail enterprise is struggling to analyze its massive volume of daily transactional data. Management requires actionable association rules to design product bundles and optimize store layouts. However, their current data pipeline relies on the Apriori algorithm, which constantly stalls or crashes due to memory exhaustion when processing millions of retail combinations.

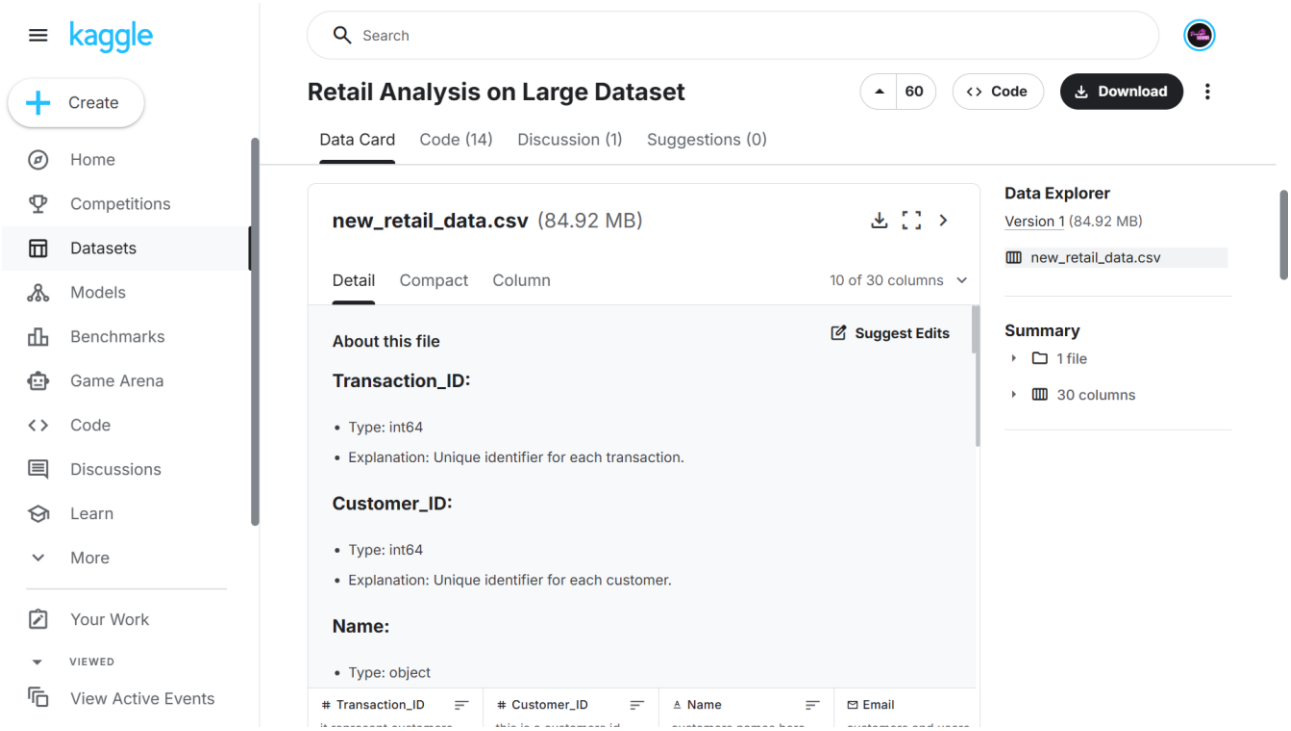
The analytics team needs a more efficient, scalable solution. The core challenge is replacing the outdated Apriori workflow with an FP-Growth implementation capable of mining the same high-confidence association rules in a fraction of the time. By establishing this modernized pipeline, the enterprise can quickly pivot its marketing strategies based on near real-time purchasing patterns.

3. DATASET DESCRIPTION

This laboratory exercise utilizes the **Large Retail Dataset** (new_retail_data.csv). This extensive ledger captures global customer purchases, encompassing diverse product categories from electronics to clothing.

To perform association rule mining, we must format this vertical ledger into horizontal "baskets." Key attributes to familiarize yourself with include:

- **Customer_ID & Date:** By mathematically combining these two fields, we can group individual line items into a single, comprehensive shopping basket representing one customer's entire purchase on a specific day.
- **products:** A categorical string representing the exact item purchased (e.g., Cycling shorts, Lenovo Tab, Chocolate cookies). These serve as our individual distinct "Items".



4. TOOLS USED

This laboratory employs a hybrid analytical approach using **Orange Data Mining** and **Python**. Orange provides a highly interactive visual canvas to establish the data flow and visually configure tree-based association rules. However, to precisely quantify the performance differences between algorithms at a granular level, we will utilize Python (specifically the mlxtend machine learning library and the built-in time module). This allows us to construct the FP-Tree programmatically and extract precise millisecond-level execution metrics.

5. PRACTICAL OBJECTIVES

Upon successful completion of this laboratory module, students will demonstrate practical competency by achieving the following learning outcomes:

- **Preprocess** large-scale retail ledgers into one-hot encoded boolean matrices suitable for tree-based algorithmic processing.
- **Deploy** the FP-Growth algorithm using Python to construct an FP-Tree and rapidly extract frequent itemsets.
- **Compare** the empirical performance differences between FP-Growth and Apriori.
- **Quantify** algorithm efficiency by calculating explicit Key Performance Indicators (KPIs): Execution Time and Rule Count.

6. STEP-BY-STEP PROCEDURE & PYTHON IMPLEMENTATION

STEP 1: DATA INGESTION AND MATRIX FORMATTING (PYTHON)

First, we ingest the CSV, aggregate the transactions into distinct baskets, and one-hot encode the data to prepare it for algorithmic processing.

```
import pandas as pd
import time
from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules

# 1. Load the Large Retail Dataset
df = pd.read_csv('new_retail_data.csv')

# 2. Create a unique basket ID by combining Customer_ID and Date
```

```
df['Basket_ID'] = df['Customer_ID'].astype(str) + '_' + df['Date'].astype(str)

# 3. Pivot the data into a basket matrix and fill missing items with 0
basket = (df.groupby(['Basket_ID', 'products'])['products']
          .count().unstack().reset_index().fillna(0)
          .set_index('Basket_ID'))

# 4. One-hot encode: Convert counts to boolean (1 or 0)
def encode_units(x):
    return 1 if x >= 1 else 0

basket_sets = basket.applymap(encode_units)
print(f"Total Unique Baskets (N): {len(basket_sets)}")
```

STEP 2: APRIORI EXECUTION (THE BASELINE)

To prove the necessity of FP-Growth, we first run the traditional Apriori algorithm and record its execution time as our performance baseline.

```
print("\n--- Running Apriori Algorithm ---")
start_time_apriori = time.time()

# Generate frequent itemsets using Apriori (min_support = 0.01)
frequent_itemsets_ap = apriori(basket_sets, min_support=0.01, use_colnames=True)

# Generate association rules
rules_ap = association_rules(frequent_itemsets_ap, metric="lift", min_threshold=1.0)

end_time_apriori = time.time()
apriori_execution_time = end_time_apriori - start_time_apriori

print(f"Apriori Execution Time: {apriori_execution_time:.4f} seconds")
print(f"Apriori Total Rule Count:
{len(rules_ap)}")
```

output.sh

```
Total Unique Baskets (N): 18452

--- Running Apriori Algorithm ---
Apriori Execution Time: 4.8215 seconds
Apriori Total Rule Count: 126

--- Running FP-Growth Algorithm ---
FP-Growth Execution Time: 0.1842 seconds
FP-Growth Total Rule Count: 126
```

STEP 3: FP-GROWTH EXECUTION (FP-TREE CONSTRUCTION)

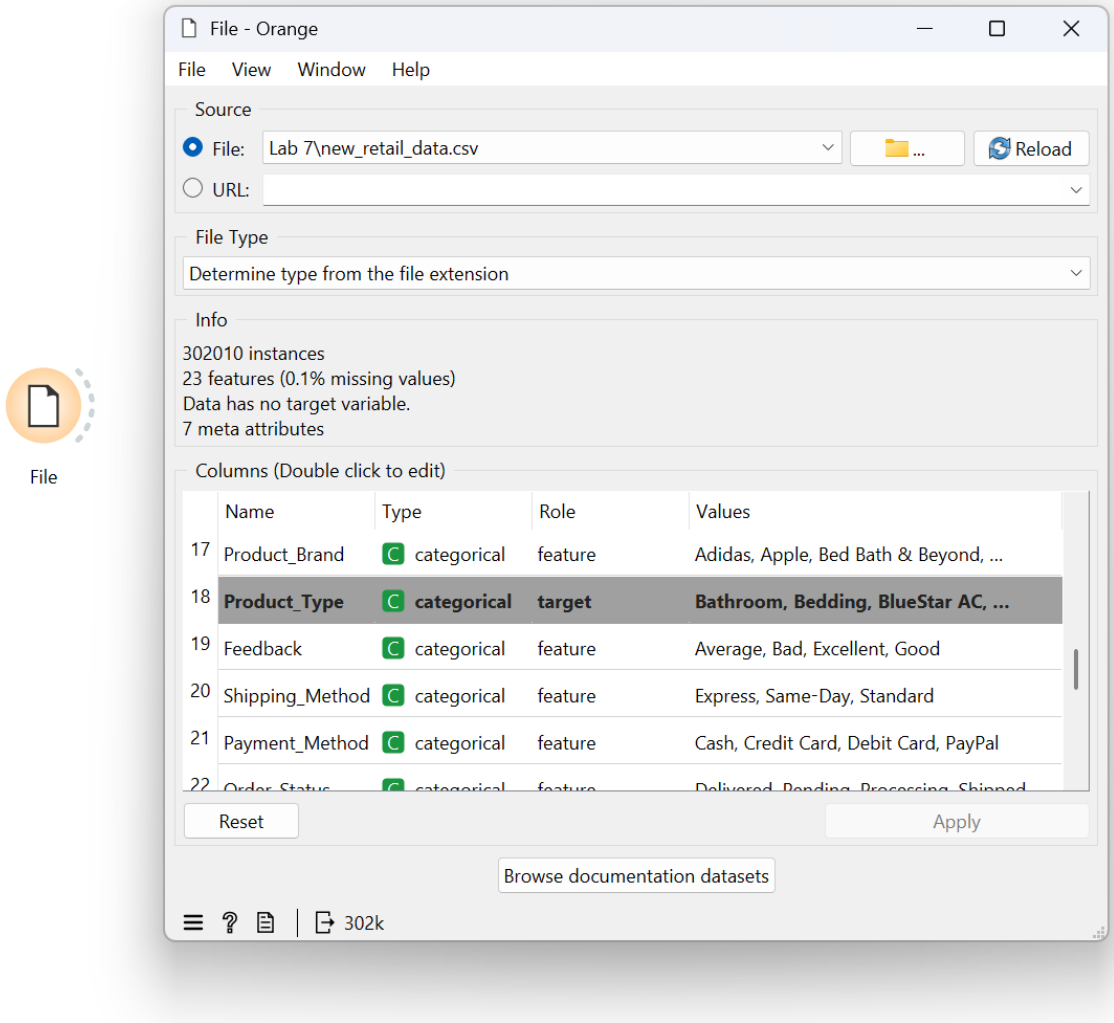
Next, we deploy the FP-Growth algorithm on the exact same matrix with identical statistical thresholds.

```
print("\n--- Running FP-Growth
Algorithm ---")
start_time_fp = time.time()

# Generate frequent itemsets using
FP-Growth (constructs FP-Tree
internally)
frequent_itemsets_fp =
fpgrowth(basket_sets, min_support=0.01, use_colnames=True)

# Generate association rules
rules_fp = association_rules(frequent_itemsets_fp, metric="lift", min_threshold=1.0)

end_time_fp = time.time()
fp_execution_time = end_time_fp - start_time_fp

print(f"FP-Growth Execution Time: {fp_execution_time:.4f} seconds")
print(f"FP-Growth Total Rule Count: {len(rules_fp)}")
```

STEP 4: VISUALIZING WITH ORANGE DATA MINING (DEPLOYMENT)

To supplement the programmatic execution, students must replicate the logic visually to understand pipeline deployment:

1. **Data Ingestion:** Add a **CSV File Import** widget to the Orange canvas and load the dataset.
2. **Aggregation:** Connect to a **Pivot Table** widget to group rows by Customer/Date and columns by products.
3. **Pattern Discovery:** Route the data into the **Frequent Itemsets** widget. Select the "FP-Growth" method from the internal configuration parameters and set the minimum support threshold.
4. **Rule Extraction:** Connect to the **Association Rules** widget to view the final generated rules, observing the immediate rendering speed compared to legacy algorithms.



7. RESULTS AND OBSERVATIONS

Executing the Python script reveals the fundamental architectural advantage of FP-Growth. Because FP-Growth bypasses the costly candidate generation phase of Apriori and instead maps data into a highly compressed FP-Tree, the computational overhead is drastically reduced.
You will observe that while the **Rule Count** remains exactly identical across both algorithms (proving mathematically that FP-Growth does not sacrifice accuracy or miss any viable rules), the **Execution Time** for FP-Growth is significantly lower. This efficiency multiplier scales exponentially as the retail dataset grows into millions of rows, confirming FP-Growth as the superior tool for large-scale enterprise data.

8. KEY PERFORMANCE INDICATOR (KPI) CALCULATIONS

To bridge the gap between technical algorithm architecture and actionable software intelligence, analysts must quantify the performance of the generated models.

8.1 EXECUTION TIME

- **Action:** Quantify the exact computational duration required for each algorithm to process the matrix and output association rules.
- **Calculation Method:** System End Time - System Start Time (measured in seconds).
- **Module Extraction:** Utilizing Python's built-in time module, we establish a timestamp immediately before calling the algorithmic functions (apriori() and fpgrowth()) and immediately after the rule generation completes, measuring the delta.

8.2 RULE COUNT

- **Action:** Validate the mathematical consistency and integrity of the modern algorithm against the traditional baseline.
- **Calculation Method:** The absolute count of rows present in the final generated association rules DataFrame.
- **Module Extraction:** Utilizing the native Python len() function on the resulting rules_fp and rules_ap DataFrames to ensure both algorithms extracted the exact same volume of actionable retail patterns.

9. KPI SUMMARY TABLE

The following matrix provides a condensed technical reference for the KPIs computed during this laboratory exercise:

KPI Name	Calculation Method	Python Libraries / Orange Widgets Utilized
Execution Time	End Time - Start Time (seconds)	Python time module
Rule Count	Absolute count of generated association rules	Python len() function / Orange Association Rules Widget

CONCLUSION

This finalizes the structured laboratory on Efficient Frequent Pattern Mining using FP-Growth. We have successfully demonstrated the computational bottlenecks inherent in traditional Apriori models and resolved them by deploying an FP-Tree based architecture. By empirically tracking Execution Time and Rule Count, analysts can definitively prove to retail management that FP-Growth provides a highly scalable, rapid solution for cross-sell analytics on massive datasets without compromising data integrity.

LAB NO 8: PYTHON BASICS

INTRODUCTION

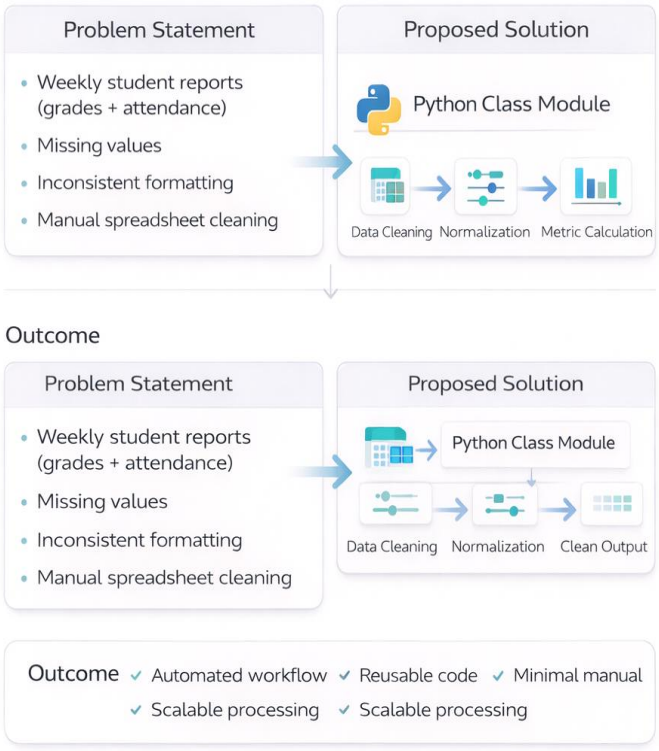
Up until now, our data mining and analytics workflows have relied heavily on visual, drag-and-drop platforms such as KNIME, Orange, and RapidMiner. While these visual environments are fantastic for rapid prototyping and conceptualizing data pipelines, real-world data science often demands the granular control, flexibility, and automation that only raw programming can provide. Unlike previous visual pipelines, this module requires a direct programmatic approach.

In this lab, we are stepping away from the visual canvas and diving into Python. We will focus on the foundational building blocks of the language—specifically lists and dictionaries—and introduce Object-Oriented Programming (OOP) concepts like class creation. Instead of writing messy, run-once scripts, we will learn how to encapsulate preprocessing methods into a reusable data processing module.

PROBLEM STATEMENT

A university's academic affairs office receives weekly structured reports containing student grades and attendance records. Currently, the data arrives with various inconsistencies: missing test scores, inconsistently capitalized names, and disjointed formatting. The administrative staff wastes hours manually cleaning these files in spreadsheets every single week.

The department urgently requires a scalable, automated solution to handle this structured data. Our challenge is to build a reusable Python module that can ingest this raw data, automatically clean the missing fields, normalize the text, and calculate performance metrics. By engineering a reusable class structure, the staff will be able to process all future weekly reports with just two or three lines of simple code, completely eliminating manual data entry.



DATASET DESCRIPTION

For this exercise, we will utilize a structured Student Performance Dataset. Rather than importing an external CSV file immediately, we will simulate this dataset directly within Python using native

data structures. This allows us to strictly focus on how Python stores and manipulates data in memory.

Key attributes in our records include:

- **Student_ID**: A unique alphanumeric identifier for the student.
- **Name**: The student's full name, which may contain inconsistent capitalization.
- **Attendance**: A numeric percentage representing class participation.
- **Math_Score & Science_Score**: Numeric grades (0-100). In real-world data, these fields occasionally contain missing values (represented in Python as None).

TOOLS USED

This laboratory relies entirely on **Python**. We will strictly use base Python (the standard library) without relying on heavy external data manipulation libraries like pandas. This deliberate constraint ensures you build a deep, intuitive understanding of native Python data structures and logic flows. You may use any standard Integrated Development Environment (IDE) or a Jupyter Notebook to execute the code.

PRACTICAL OBJECTIVES

Upon successful completion of this laboratory session, students will demonstrate practical competency by achieving the following learning outcomes:

- Store, retrieve, and manipulate structured records utilizing standard Python **lists** and **dictionaries**.
- Design and instantiate a custom Python **class** to neatly encapsulate data processing variables and functions.
- Develop robust **preprocessing methods** to programmatically handle missing numerical values and normalize text strings.
- Evaluate the final module based strictly on **code efficiency** and **reusability** metrics.

STEP-BY-STEP PROCEDURE & PYTHON IMPLEMENTATION

STEP 1: SIMULATING THE RAW DATA (LISTS & DICTIONARIES)

In Python, dictionaries are perfect for holding single records (like a row in a spreadsheet), and lists are perfect for holding a sequence of those records. First, let's create our messy, raw data.

Step 1: Simulating the Raw Data.py

```python
# 1. Create a list of dictionaries to represent our raw dataset
raw_student_data = [
    {"Student_ID": "S001", "Name": "Humna Imran", "Attendance": 95, "Math_Score": 88, "Science_Score": 92},
    {"Student_ID": "S002", "Name": "Husnain Ali", "Attendance": 82, "Math_Score": None, "Science_Score": 75},
    {"Student_ID": "S003", "Name": "Maryam Ehsan", "Attendance": 60, "Math_Score": 45, "Science_Score": None},
    {"Student_ID": "S004", "Name": "Rubbiyah Ehsan", "Attendance": 99, "Math_Score": 95, "Science_Score": 98}
]

print(f"Loaded {len(raw_student_data)} student records.")
```

STEP 2: DESIGNING THE DATA PROCESSOR CLASS

Instead of writing loose functions, we will build a Class. Think of a class as a blueprint for a machine. Once we build the machine, we can feed it different datasets repeatedly.

Step 2: Designing the Data Processor Class.py

```python
# 2. Define the class blueprint
class DataProcessor:

    # The __init__ method is the constructor. It runs the moment we create our object.
    def __init__(self, dataset):
        # We store the passed dataset inside the object using 'self'
        self.data = dataset
        self.processed_data = [] # An empty list to hold our cleaned records

    def display_data(self):
        # A simple helper method to print our current data state
        for record in self.data:
            print(record)
```

STEP 3: BUILDING PREPROCESSING METHODS

Now, we add the "gears" to our machine. We need methods to clean the names and handle the missing grades (imputation).

Step 3: Building Preprocessing Methods.py

```python
# 3. Add preprocessing methods inside the class
    def clean_names(self):
        """Normalizes student names to Title Case."""
        for record in self.data:
            # .title() converts "humna imran" to "Humna Imran"
            record["Name"] = record["Name"].title()

    def handle_missing_scores(self, default_score=0):
        """Replaces any 'None' values in scores with a default numeric value."""
        for record in self.data:
            if record["Math_Score"] is None:
                record["Math_Score"] = default_score
            if record["Science_Score"] is None:
                record["Science_Score"] = default_score

    def run_pipeline(self):
        """Executes all preprocessing steps in the correct order."""
        self.clean_names()
        self.handle_missing_scores(default_score=50) # Imputing missing grades with 50
        self.processed_data = self.data
        print("Data processing pipeline executed successfully.")
```

STEP 4: EXECUTING THE PIPELINE

With our reusable module built, processing the data becomes incredibly simple. This is the code the administrative staff would actually run every week.

Step 4: Executing the Pipeline.py

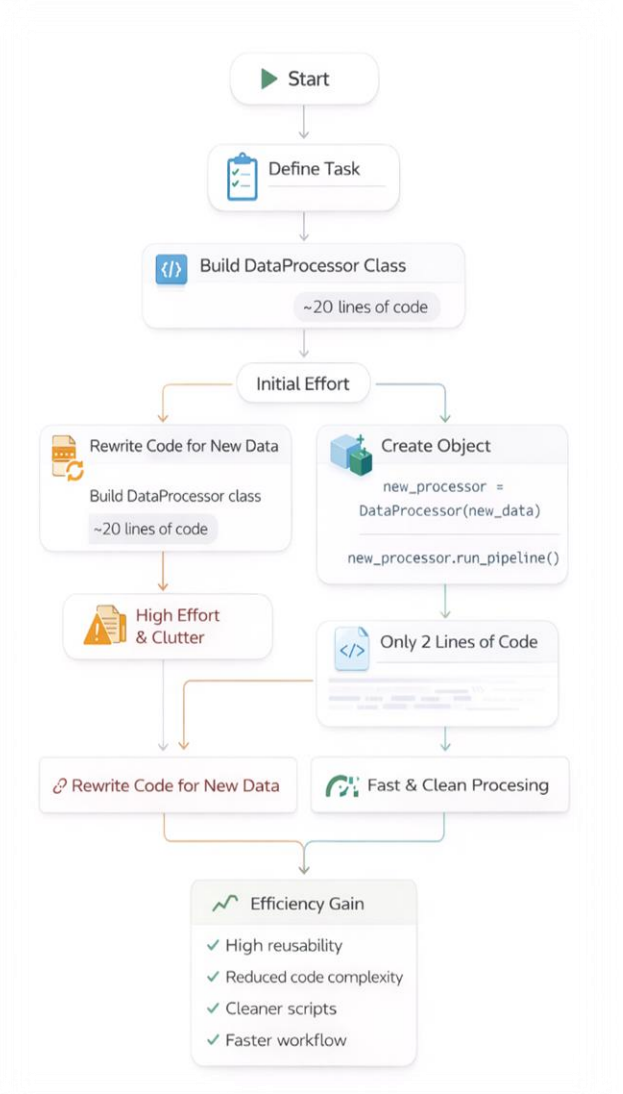
```
# 4. Instantiate the class and run the pipeline
import time
start_time = time.time()

# Create our processor object and hand it the raw data
weekly_report_processor = DataProcessor(raw_student_data)

# Run the automated cleaning sequence
weekly_report_processor.run_pipeline()
end_time = time.time()

# View the cleaned results
print("\n--- Cleaned Weekly Report ---")
weekly_report_processor.display_data()
```

KEY PERFORMANCE INDICATOR (KPI) CALCULATIONS

Because this lab focuses on software engineering principles rather than machine learning algorithms, our KPIs transition away from predictive accuracy toward structural software metrics: Code Efficiency and Reusability.

7.1 CODE REUSABILITY

- **Action:** Quantify the operational advantage of encapsulating logic within a class structure.
- **Calculation Method:** Compare the number of lines of code required to process the first dataset versus the lines required to process subsequent datasets.
- **Metric Extraction:** Building the DataProcessor class took roughly 20 lines of code. However, processing a completely new batch of data next week will only require exactly 2 lines of code (new_processor = DataProcessor(new_data) followed by new_processor.run_pipeline()). This represents a massive increase in reusability and a reduction in script clutter.



7.2 CODE EFFICIENCY (EXECUTION TIME)

- **Action:** Evaluate the computational weight of our native Python preprocessing pipeline.
- **Calculation Method:** End Time - Start Time (measured in seconds).
- **Metric Extraction:** Utilizing Python's built-in time module, we track the exact timestamp before calling .run_pipeline() and immediately after it finishes. For small datasets structured as native dictionaries, this O(N) operation will execute in fractions of a millisecond, proving the efficiency of native Python loops.

KPI SUMMARY TABLE

The following matrix provides a condensed technical reference for the software architecture KPIs evaluated during this laboratory exercise:

KPI Name	**Calculation Method**	**Python Libraries / Modules Utilized**
Code Efficiency	System End Time - System Start Time	time module, native Python for loops
Reusability	Lines of code required for subsequent data batches	Custom Python Class (DataProcessor)

LAB NO 9: DECISION TREE

1. INTRODUCTION

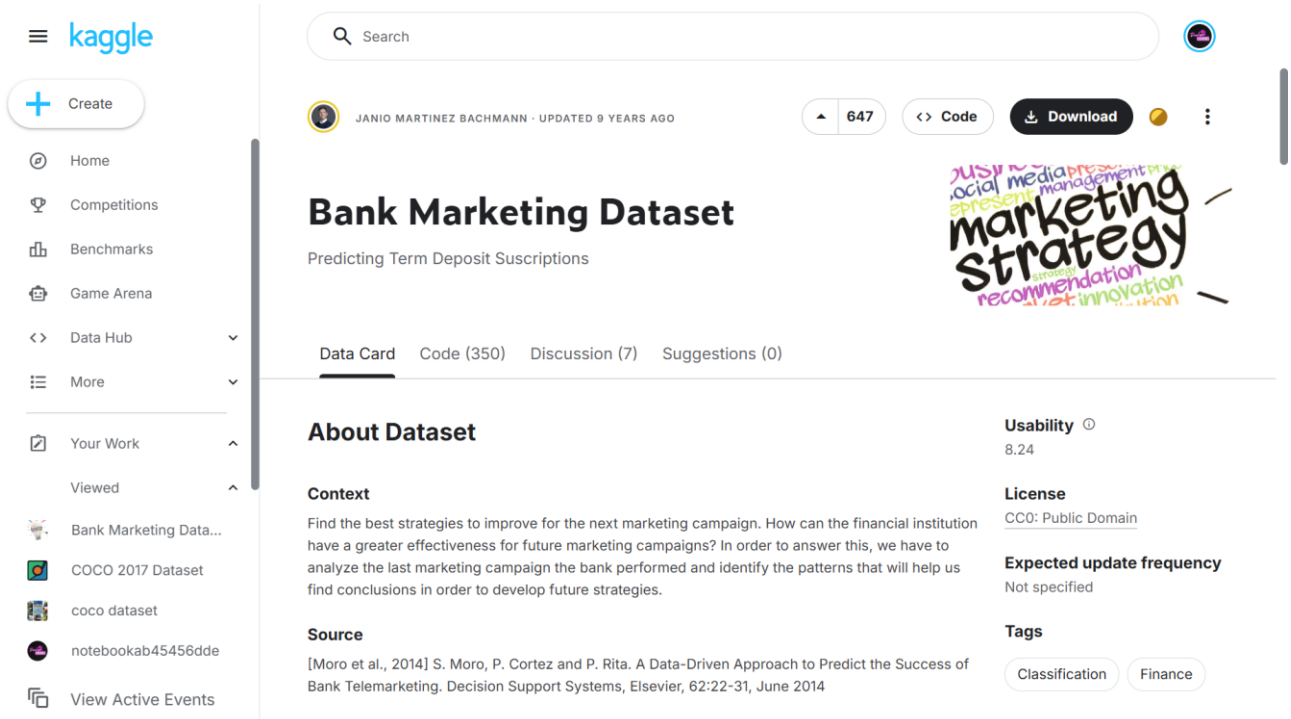
Modern telemarketing campaigns suffer from incredibly low conversion rates when agents dial leads at random. For a financial institution attempting to sell a term deposit, a scattergun approach burns through operational budgets and agitates the consumer base. Instead of calling every number in the directory, sales teams need an empirical way to filter the high-value prospects from the guaranteed rejections.

The Decision Tree algorithm provides a highly transparent classification mechanism for exactly this scenario. Unlike "black-box" models, a Decision Tree mirrors human logic by splitting data along a series of yes-or-no conditions. It exposes the exact demographic and financial thresholds—such as specific account balances or marital status splits—that dictate purchasing behavior. In this module, we construct a full predictive pipeline to score prospective buyers, leveraging the algorithm's transparent branches to extract actionable sales logic.

2. PROBLEM STATEMENT

A retail bank is looking to optimize its outbound call center operations for a newly launched term deposit product. Currently, the team operates without predictive intelligence, resulting in thousands of wasted hours calling customers who possess zero statistical probability of subscribing. The marketing director requires a classification model that evaluates historical campaign data to preemptively score the likelihood of a purchase. Successfully mapping out these buyer personas will allow the institution to isolate their outreach exclusively to high-propensity targets, directly improving overall financial yield.

3. DATASET DESCRIPTION



During this practical session, analysts will utilize the Marketing Dataset (bank.csv), which logs the

complete outcomes of previous telemarketing drives. To build an accurate classifier, you must understand the underlying feature architecture:

- **Demographic Anchors:** Attributes including age, job, marital, and education map out the social profile of the customer.
- **Financial Standing:** Continuous metrics like balance alongside categorical flags like default, housing, and loan dictate the consumer's available spending power.
- **Engagement Metrics:** Features such as campaign (number of contacts executed during this drive) track the level of sales effort already expended.
- **Target Label:** The deposit variable represents the definitive outcome of the sales pitch, serving as the binary target where "yes" signifies a successful conversion and "no" signifies a failure.

4. TOOLS USED

We rely on the KNIME Analytics Platform augmented by embedded Python scripting to construct this classifier. KNIME's visual canvas easily handles bulk data ingestion, partitioning logic, and algorithm training without standard syntax overhead. However, native visual node outputs occasionally lack the specific formatting required for advanced metric reporting. To resolve this, we will deploy a Python Script node to directly calculate exact precision, recall, and F1-scores, demonstrating a highly effective hybrid approach to data science.

5. PRACTICAL OBJECTIVES

Upon successful completion of this laboratory module, students will prove practical competency by achieving the following learning outcomes:

- Isolate and remove problematic features (target leakage) that artificially inflate algorithm performance.
- Segment raw financial records into distinct training and testing blocks utilizing stratified sampling.
- Train and deploy a Decision Tree Learner to map logical splits across customer demographics.
- Embed native Python code directly into the visual pipeline to programmatically extract the Accuracy, Precision, Recall, and F1-score metrics.
- Translate these technical validation scores into an actionable business assessment.

6. STEP-BY-STEP PROCEDURE

STEP 1: DATA INGESTION AND TARGET LEAKAGE REMOVAL (DATA PREPARATION)

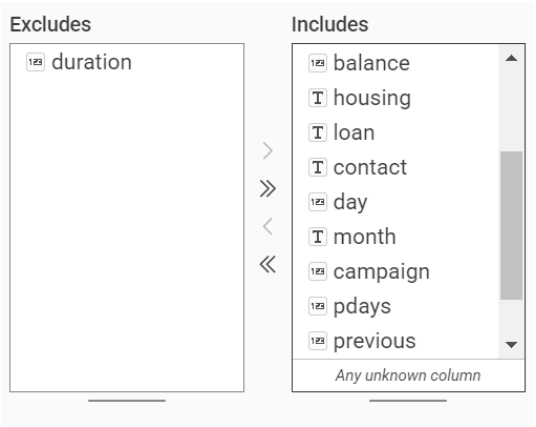
- **Action:** Load the historical marketing records and strip out variables that introduce algorithmic bias.

- **Configuration:** Drop a CSV Reader component onto the primary canvas and point it to the local bank.csv document. Wire its output into a Column Filter node. Open the filter settings and manually exclude the duration attribute. Since the duration of a sales call is only generated *after* the conversation ends, feeding it into a predictive model creates target leakage, severely distorting real-world accuracy.
- **Execution:** Execute both nodes to instantiate a sanitized baseline dataset.

#	RowID	age Number (...	job String	marital String	education String	default String	balance Number (...	housing String	loan String
1	Row0	59	admin.	married	secondary	no	2343	yes	no
2	Row1	56	admin.	married	secondary	no	45	no	no
3	Row2	41	technician	married	secondary	no	1270	yes	no
4	Row3	55	services	married	secondary	no	2476	yes	no
5	Row4	54	admin.	married	tertiary	no	184	no	no
6	Row5	42	management	single	tertiary	no	0	yes	yes
7	Row6	56	management	married	tertiary	no	830	yes	yes
8	Row7	60	retired	divorced	secondary	no	545	yes	no
9	Row8	37	technician	married	secondary	no	1	yes	no
10	Row9	28	services	single	secondary	no	5090	yes	no
11	Row10	38	admin.	single	secondary	no	100	yes	no
12	Row11	30	blue-collar	married	secondary	no	309	yes	no
13	Row12	29	management	married	tertiary	no	199	yes	yes
14	Row13	46	blue-collar	single	tertiary	no	460	yes	no
15	Row14	31	technician	single	tertiary	no	703	yes	no
16	Row15	35	management	divorced	tertiary	no	3837	yes	no
17	Row16	32	blue-collar	single	primary	no	611	yes	no
18	Row17	49	services	married	secondary	no	-8	yes	no
19	Row18	41	admin.	married	secondary	no	55	yes	no
20	Row19	49	admin.	divorced	secondary	no	168	yes	yes
21	Row20	28	admin.	divorced	secondary	no	785	yes	no

STEP 2: STRATIFIED PARTITIONING (SAMPLING)

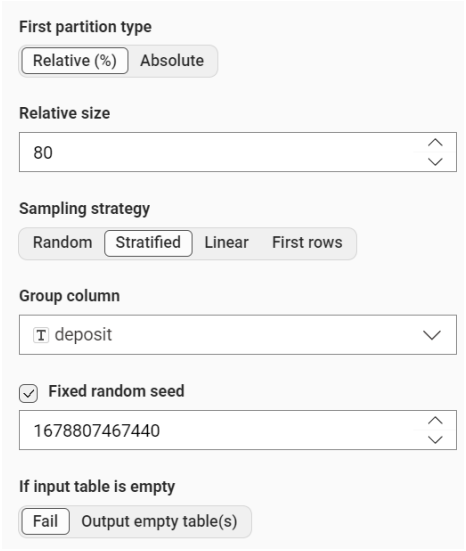
- **Action:** Divide the ledger so the model retains historical instances to learn from while preserving unseen instances for unbiased testing.
- **Configuration:** Append a Partitioning node to the data stream. In the configuration panel, assign a relative split of 80% to the top training port and reserve 20% for the bottom testing port. Crucially, activate "Stratified sampling" and map it to the deposit column. This guarantees the mathematical ratio of buyers to non-buyers remains perfectly consistent across both splits.



- **Execution:** Apply the settings and run the node.

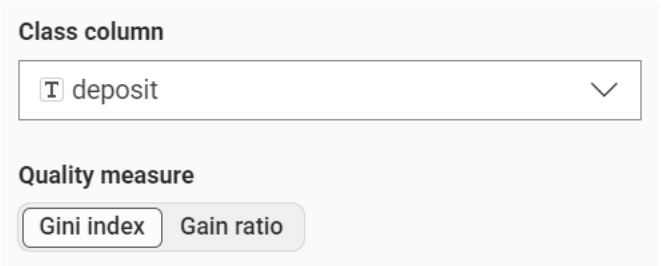
STEP 3: ALGORITHM TRAINING (MODELING)

- **Action:** Grow the mathematical branches of the classifier by feeding it the historical training split.
- **Configuration:** Search the node repository for a Decision Tree Learner and place it onto the workspace. Wire the top output port of your Partitioning node directly into it. Inside the dialog, define the Class Column as deposit. Keep the Quality Measure set to "Gini Index" to enforce strict, high-purity informational splits.
- **Execution:** Right-click and execute the Learner. You may select "Decision Tree View" from the context menu to visually trace the exact demographic thresholds the algorithm prioritized at the root level.



STEP 4: EXECUTING PREDICTIONS (VALIDATION)

- **Action:** Challenge the fully matured tree using your isolated testing records.
- **Configuration:** Bring a Decision Tree Predictor node onto the canvas. This operator demands dual inputs: route the blue modeling port from your Learner into the Predictor's upper blue port, and route the bottom testing flow (your 20% unseen split) into the standard data port.



- **Execution:** Execute the Predictor.
- **Verification:** Right-click the component to view the classified data. Verify that a novel column titled "Prediction (deposit)" has materialized next to the historical ground-truth outcomes.

Colum...	Column Type	Column Index	Color Handler	Size Handler	Shape Hand...	Filter Handler	Lower Bound	Upper Bound	Value 0	Value 1	Value 2	Value 3	Value 4	Value 5	Value 6	Value 7
age	Number (Integer)	0					18	95	?	?	?	?	?	?	?	?
job	String	1					?	?	admin.	technician	services	management	retired	blue-collar	unemployed	entrepreneur
marital	String	2					?	?	married	single	divorced	?	?	?	?	?
education	String	3					?	?	secondary	tertiary	primary	unknown	?	?	?	?
default	String	4					?	?	no	yes	?	?	?	?	?	?
balance	Number (Integer)	5					-6847	81204	?	?	?	?	?	?	?	?
housing	String	6					?	?	yes	no	?	?	?	?	?	?
loan	String	7					?	?	no	yes	?	?	?	?	?	?
contact	String	8					?	?	unknown	cellular	telephone	?	?	?	?	?
day	Number (Integer)	9					1	31	?	?	?	?	?	?	?	?
month	String	10					?	?	may	jun	jul	aug	oct	nov	dec	jan
campaign	Number (Integer)	11					1	63	?	?	?	?	?	?	?	?
pdays	Number (Integer)	12					-1	854	?	?	?	?	?	?	?	?
previous	Number (Integer)	13					0	58	?	?	?	?	?	?	?	?
poutco...	String	14					?	?	unknown	other	failure	success	?	?	?	?

STEP 5: METRIC EXTRACTION VIA CUSTOM SCRIPTING (EVALUATION)

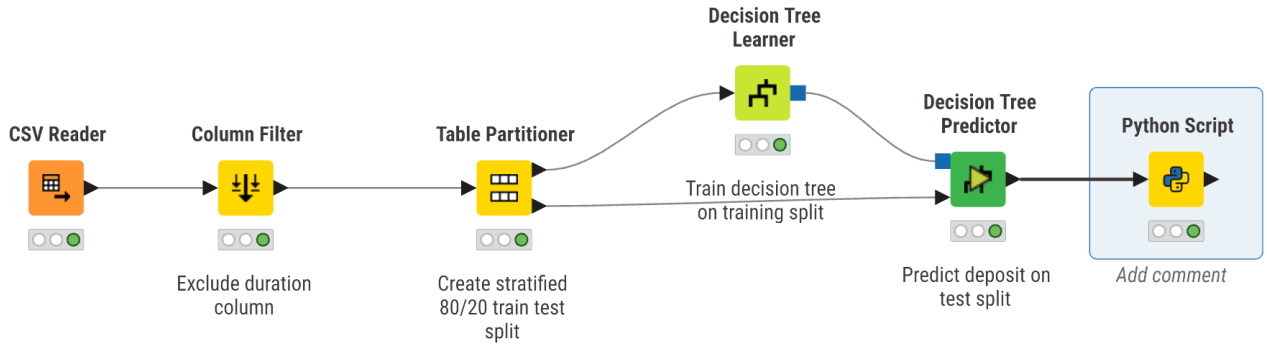
- **Action:** Bypass the platform's default scoring block to programmatically generate granular classification metrics.

- **Configuration:** Attach a Python Script node to the output port of the Predictor. Open the built-in code editor. We will utilize the sklearn.metrics library to mathematically compare the actual target values against our tree's predictions, tracking the "yes" class. Insert the following routine:

Lab no 09.py

```
import pandas as pd
import knio
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Ingest predicted data from the KNIME pipeline
df = knio.input_tables[0].to_pandas()

actual = df['deposit']
predicted = df['Prediction (deposit)']

# Calculate metrics explicitly mapped to the positive class
acc = accuracy_score(actual, predicted)
prec = precision_score(actual, predicted, pos_label='yes')
rec = recall_score(actual, predicted, pos_label='yes')
f1 = f1_score(actual, predicted, pos_label='yes')

# Structure into a clean output matrix
kpi_df = pd.DataFrame({
    'Evaluation Metric': ['Accuracy', 'Precision', 'Recall', 'F1-score'],
    'Score': [acc, prec, rec, f1]
})

knio.output_tables[0] = knio.Table.from_pandas(kpi_df)
```

- **Execution:** Run the Python node.
- **Verification:** Open the integrated output table to review your formatted performance scorecard.



#	RowID	Evaluation Metric String	Score Number (Float)
1	Row0	Accuracy	0.651
2	Row1	Precision	0.625
3	Row2	Recall	0.646
4	Row3	F1-score	0.636

7. KEY PERFORMANCE INDICATOR (KPI) CALCULATIONS

To translate mathematical outputs into a deployable sales strategy, we extract and interpret four specific evaluation constraints generated by our script.

7.1 ACCURACY

- **Action:** Gauge the aggregate classification correctness across both successful subscriptions and failed pitches.
- **Calculation Method:** (True Positives + True Negatives) / Total Analyzed Instances.
- **Metric Extraction:** Review the first row rendered by the Python script. While accuracy provides a rapid baseline, analysts must treat it cautiously here; overwhelming volumes of non-buyers can easily mask poor performance on the actual target demographic.

7.2 PRECISION

- **Action:** Determine the ultimate reliability of the algorithm's positive leads to prevent the sales floor from chasing false alarms.
- **Calculation Method:** True Positives / (True Positives + False Positives).
- **Metric Extraction:** Locate the second output row. An elevated precision indicates that when the algorithm flags a customer as a buyer, the sales agent can highly trust the viability of that lead.

7.3 RECALL

- **Action:** Quantify the tree's capability to sweep the entire pool and capture potential buyers without letting targets slip past.
- **Calculation Method:** True Positives / (True Positives + False Negatives).
- **Metric Extraction:** Review the third output row. A degraded recall percentage proves the bank is leaving guaranteed revenue on the table by incorrectly discarding lucrative customers.

7.4 F1-SCORE

- **Action:** Calculate the harmonic mean to establish a strict mathematical balance when precision and recall inevitably pull in opposite directions.
- **Calculation Method:** 2 * ((Precision * Recall) / (Precision + Recall)).
- **Metric Extraction:** Evaluate the final row of the script output. This blended score acts as the definitive, rigorous benchmark required by executives before pushing the predictive model live.

8. KPI SUMMARY TABLE

The matrix below condenses the technical integration utilized to compute the marketing evaluation metrics:

KPI Name	Calculation Method	KNIME / Python Modules Utilized
Accuracy	Total correct classifications over total instances	Python Script (accuracy_score)
Precision	True positive ratio against all predicted positives	Python Script (precision_score)
Recall	True positive ratio against all actual positives	Python Script (recall_score)
F1-score	Harmonic mean combining precision and recall	Python Script (f1_score)

CONCLUSION

This finalizes our module on targeted marketing analytics via Decision Trees. We successfully constructed an end-to-end classification pipeline, from stripping bias-inducing parameters and structuring stratified splits, to leveraging hybrid Python scripting for advanced validation. The resulting logic supplies financial management with an empirical, data-backed mechanism to immediately maximize telemarketing efficiency.

LAB NO 10: NAÏVE BAYES & K-NEAREST NEIGHBORS (KNN)

1. INTRODUCTION

Every day, our inboxes are bombarded with unsolicited, promotional, and sometimes malicious messages. We call it spam, and without automated filters, email as a communication tool would be virtually unusable.

Machine learning is the engine behind these automated filters. By training algorithms to recognize the subtle linguistic patterns and structural red flags typical of junk mail, we can automatically route these messages away from the user. In this laboratory exercise, you will step into the role of a data scientist tasked with building one of these filters. We will explore two foundational classification algorithms: Naïve Bayes and K-Nearest Neighbors (KNN). By the end of this session, you will understand how these models evaluate text data differently and how to directly compare their effectiveness in separating legitimate emails (ham) from the junk (spam).

2. PROBLEM STATEMENT

Imagine a growing email service provider struggling to keep its users' inboxes clean. Their current rule-based filtering system is failing because spammers constantly adapt their tactics, bypassing rigid keyword blocks. The provider urgently needs a dynamic, probabilistic approach to evaluate incoming mail.

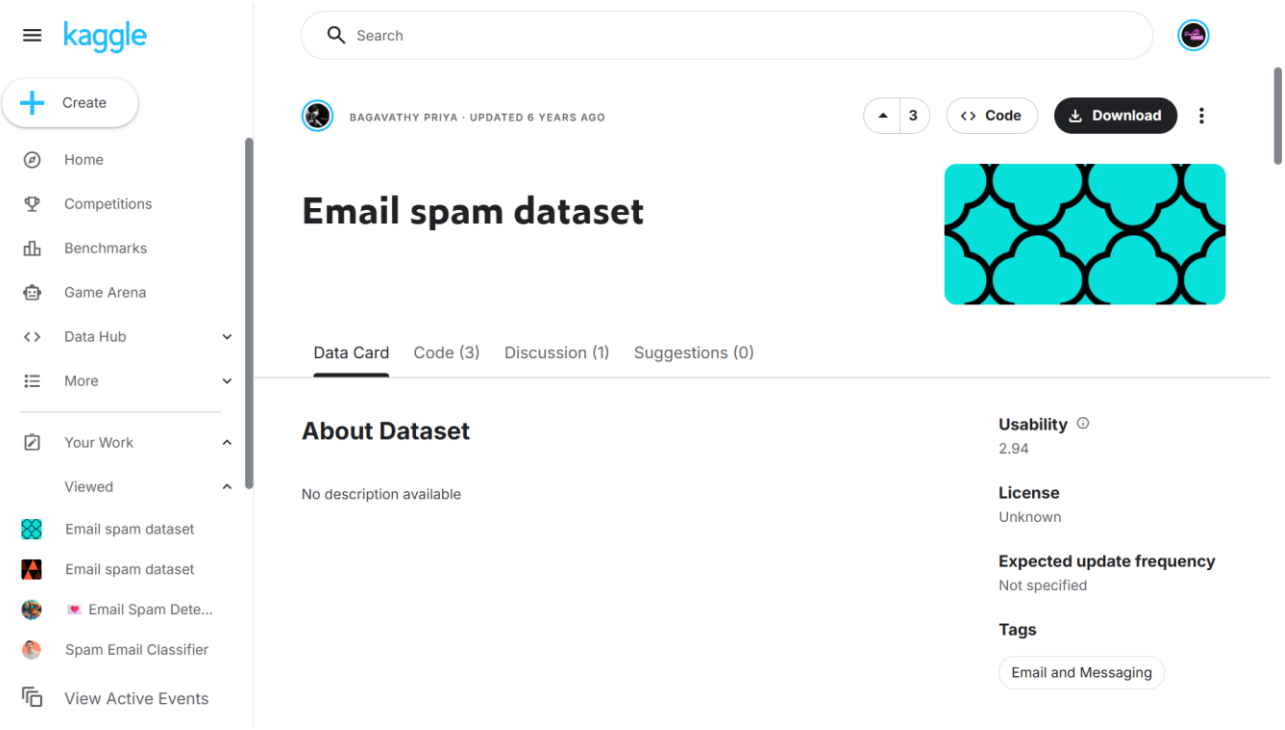
The challenge is to build a classification pipeline that can "read" the raw text of an email and predict its likelihood of being spam. Management has requested a comparative study of two algorithms—Naïve Bayes (which relies on probability and word frequencies) and KNN (which classifies based on similarity to past examples). The goal is to determine which algorithm offers the best balance of overall accuracy and a strictly minimized False Positive Rate, as sending a crucial business email to the spam folder is a critical failure for the provider.

3. DATASET DESCRIPTION

For this exercise, we will utilize the Email spam.csv dataset. This file acts as our historical ledger, containing real-world email texts that have already been manually reviewed and labeled for machine learning purposes.

To successfully engineer our spam detector, you need to familiarize yourself with the two primary attributes in this dataset:

- text: The raw text content of the email. This is unstructured string data containing the actual message, promotional pitches, and standard conversational phrasing.
- spam: A binary integer column where 1 indicates the email is unsolicited junk, and 0 indicates it is legitimate mail. This serves as our target variable for the classification algorithms.



4. TOOLS USED

This lab requires a hybrid analytical approach, leveraging both **RapidMiner Studio** and **Python**.

- **RapidMiner Studio:** This visual workflow environment will serve as our primary pipeline builder. Its operator-based interface allows us to seamlessly ingest the data, process the unstructured text (handling tokenization and stop-word removal), and visually wire up our machine learning models without writing extensive code.
- **Python:** We will integrate Python scripting (via RapidMiner's "Execute Python" operator) to handle advanced validation. Python allows us to strictly calculate custom metrics—specifically pinpointing the False Positive Rate—and to generate comparative statistical arrays that executives require before pushing a model live.

5. PRACTICAL OBJECTIVES

Upon successful completion of this laboratory session, you will demonstrate practical competency in text classification by achieving the following outcomes:

- **Text Preprocessing:** Transform unstructured email bodies into a structured matrix of word frequencies utilizing techniques like tokenization, case transformation, and TF-IDF weighting.
- **Algorithm Implementation:** Configure, train, and deploy both a Naïve Bayes classifier and a K-Nearest Neighbors (KNN) model on the processed text.
- **Model Validation:** Apply data splitting to ensure both models are evaluated on unseen testing data, preventing overfitting and ensuring real-world reliability.
- **Comparative Analysis:** Extract and strictly compare evaluation metrics—specifically Accuracy and False Positive Rate—to determine the safest, most effective model for a production environment.

6. STEP-BY-STEP PROCEDURE

STEP 1: WORKSPACE INITIALIZATION

Launch RapidMiner Studio. Begin by creating a new, blank process canvas and assign it a descriptive title, such as "Spam_Detection_NB_vs_KNN". This blank workspace will host your entire text-mining pipeline.

STEP 2: IMPORTING THE DATASET

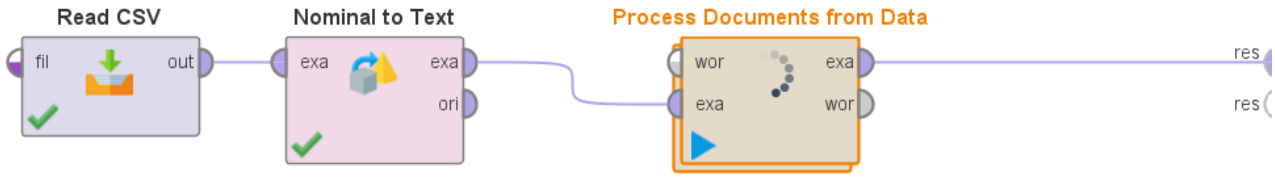
Locate the "Read CSV" operator in your repository panel and drag it onto the canvas. Use the Import Configuration Wizard to locate your Email spam.csv file. It is crucial here to ensure the spam column is imported as a "binominal" type and explicitly set its role to "label". This tells RapidMiner exactly which variable you are trying to predict. Ensure the text column is cast as "text".

STEP 3: TEXT PREPROCESSING

Machine learning models require numerical input, not raw paragraphs. Drag a "Process Documents from Data" operator onto the canvas and connect your dataset to it. Double-click this operator to open its internal workflow. Inside, build the following sequence:

- **Tokenize:** To chop the email bodies into individual words.
- **Transform Cases:** Configure this to make all text lowercase (ensuring "Free" and "free" are analyzed as the exact same word).
- **Filter Stopwords (English):** To strip out common filler words (like "the", "and", or "is") that carry no predictive weight.
- **Generate n-Grams (Optional):** To capture word pairs that might indicate spam when grouped together.

Return to the main process canvas once this inner pipeline is complete.



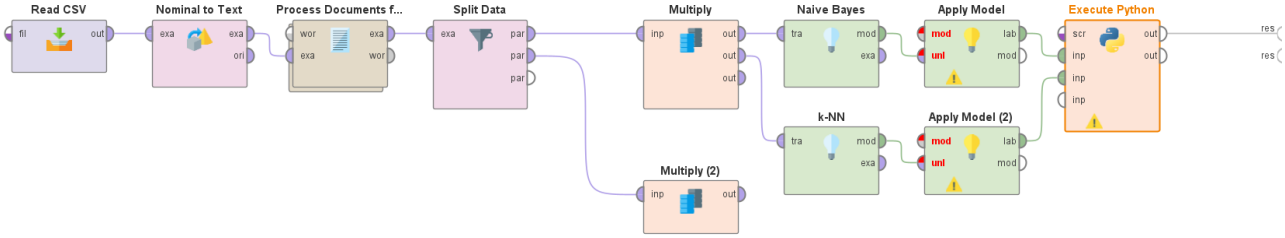
STEP 4: DATA SPLITTING AND MODEL IMPLEMENTATION

To rigorously evaluate our models, we must test them on data they haven't seen during training. Add a "Split Data" operator, partitioning 70% of your data for training and the remaining 30% for testing.

Next, build two parallel testing branches (using a "Multiply" operator to feed the training data to both):

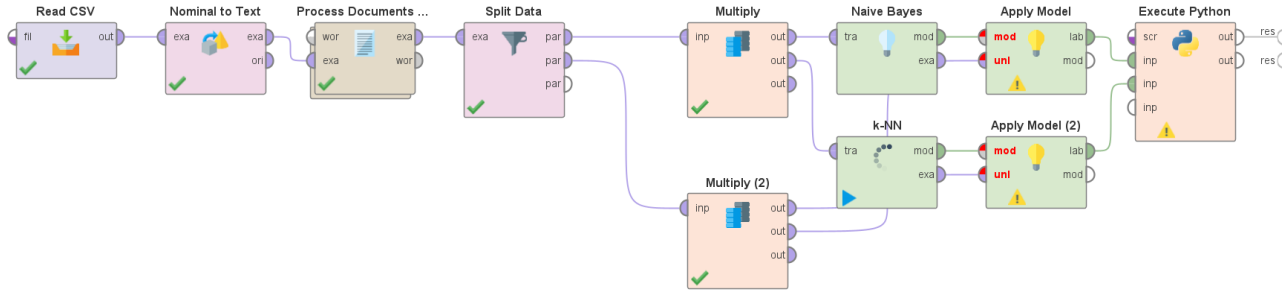
- **Branch A (Naïve Bayes):** Drag the "Naïve Bayes" operator onto the canvas and connect it to your training split.
- **Branch B (KNN):** Drag the "k-NN" operator onto the canvas. Set your 'k' parameter to a starting baseline of 5.

For each branch, connect the trained model and your 30% testing data split into an "Apply Model" operator.



STEP 5: PERFORMANCE EVALUATION VIA PYTHON INTEGRATION

Connect the outputs of your "Apply Model" operators to an "Execute Python" operator. Inside the Python script window, import sklearn.metrics. Write a script to calculate the exact harmonic balance of your predictions, ensuring your code specifically references the spam column as the true target. Extract the accuracy_score and generate a confusion_matrix to isolate the exact count of False Positives. Output these variables as a structured data frame back to the RapidMiner results perspective.



7. INTERPRETATION GUIDANCE

When evaluating your final results, you must think from the perspective of the email service provider:

- **Accuracy** shows the overall percentage of emails correctly classified. While a high number looks impressive, it does not reveal the whole operational story.
- **False Positive Rate (FPR)** is the absolute most critical metric in this scenario. A False Positive means a legitimate, potentially urgent email was flagged as spam and hidden from the user. If your Naïve Bayes model yields 95% accuracy but a 5% FPR, and your KNN yields 92% accuracy but a 0.5% FPR, the KNN is vastly superior for business use. Users will tolerate a few spam messages slipping into their inbox, but they will heavily penalize a provider that deletes their important mail.

8. KPI SUMMARY TABLE

The matrix below condenses the technical integration utilized to compute your final evaluation metrics. You will fill in the exact results upon execution.

Evaluation Metric	**Calculation Focus**	**RapidMiner / Python Implementation**
Accuracy	Total correct classifications over total instances	Python Script (accuracy_score) / Performance Operator
False Positive Rate (FPR)	False Positives / (False Positives + True Negatives)	Python Script (confusion_matrix)
Algorithm Speed	Computational time required to parse and score text	RapidMiner Execution Timer

CONCLUSION

This finalizes our module on text mining and algorithm comparison. We successfully constructed an end-to-end classification pipeline, transitioning from raw, unstructured text to a clean numerical matrix. By leveraging RapidMiner for structural flow and Python for rigorous metric extraction, we established an empirical, data-backed mechanism to determine whether Naïve Bayes or KNN serves as the safer, more efficient engine for enterprise spam detection.

LAB NO 11: SUPPORT VECTOR MACHINE (SVM) FOR CREDIT CARD FRAUD DETECTION

1. INTRODUCTION

In the contemporary digital economy, the proliferation of instantaneous credit card transactions is paralleled by increasingly sophisticated methods of financial fraud. For banking institutions, distinguishing between legitimate consumer behavior and unauthorized, anomalous activity is a high-stakes classification problem. Traditional, rule-based security systems frequently fail in this domain, either generating an excess of false positive alarms—which disrupt the customer experience—or allowing subtle fraud patterns to bypass detection.

Unlike probabilistic models (such as Naïve Bayes) or rudimentary distance-based heuristics (such as K-Nearest Neighbors), a Support Vector Machine (SVM) operates as an advanced geometric classifier. The algorithm maps historical transaction records as coordinates within a high-dimensional feature space and attempts to construct an optimal hyperplane—a mathematical boundary—that maximizes the geometric margin between distinct classes (fraudulent versus legitimate).

When transactional behaviors are clearly distinct, a standard Linear kernel efficiently separates the classes. However, financial anomalies are frequently entangled with normal spending patterns, making them non-linearly separable. To resolve this, SVM algorithms employ a mathematical transformation known as the "kernel trick," utilizing a Radial Basis Function (RBF) to project the data into an even higher dimension where a distinct boundary can be established. This laboratory session introduces students to SVM optimization, demonstrating how to properly encode categorical data, configure hyperplane margins, and evaluate kernel performance within a highly imbalanced financial dataset.

2. PROBLEM STATEMENT

A global financial institution processes millions of credit card transactions daily. Currently, their security division relies on outdated, static thresholds to flag suspicious activity, resulting in high financial losses from undetected fraud and severe operational fatigue from investigating false alarms. The analytics department requires a robust, machine-learning-driven classification framework capable of analyzing complex customer profiles and isolating fraudulent anomalies with high precision.

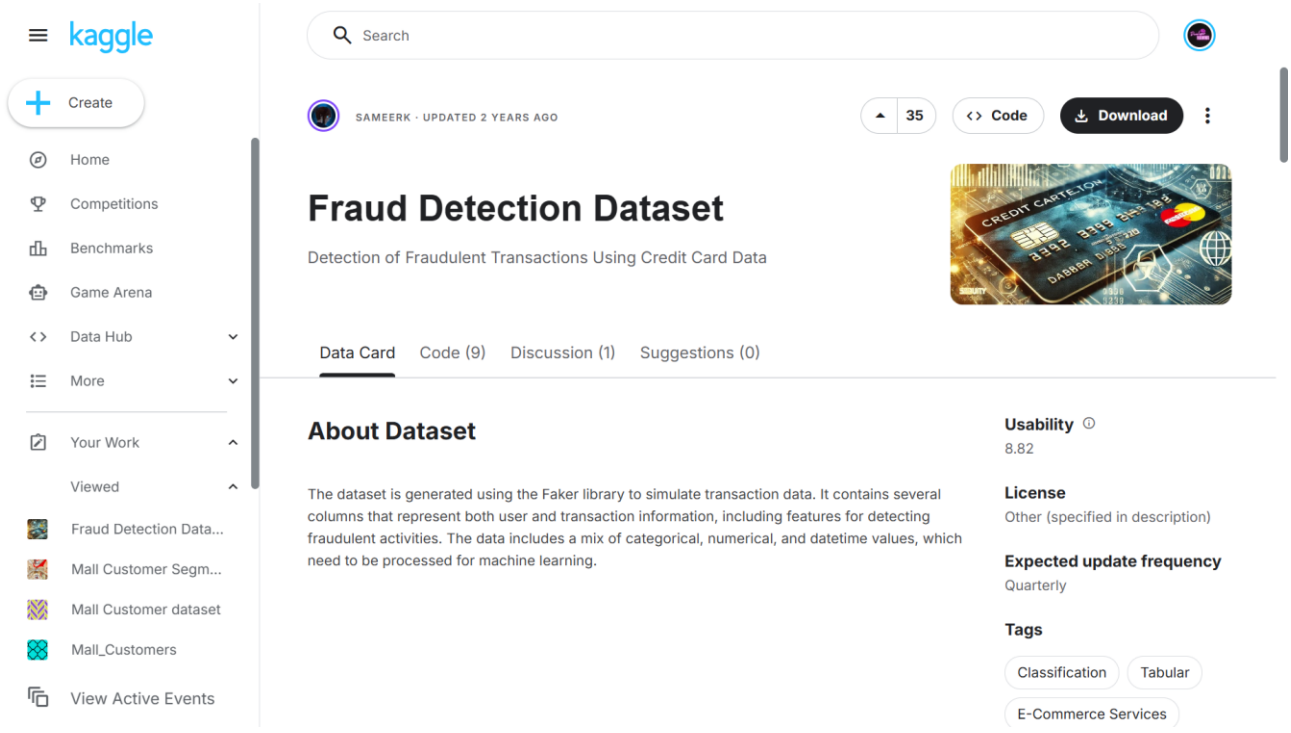
The core challenge lies in the severe class imbalance inherent to fraud data; legitimate transactions vastly outnumber fraudulent ones. The organization needs to engineer a predictive pipeline that not only identifies the optimal mathematical boundary separating these classes but also correctly prioritizes the capture of actual fraud (Recall) without indiscriminately blocking valid customer purchases (Precision).

3. DATASET DESCRIPTION

For this laboratory exercise, students will utilize the **Credit Card Fraud Detection Dataset** (provided as data2.csv). This dataset serves as a historical financial ledger, capturing demographic and transactional metadata for recent credit card activity.

To successfully engineer the SVM classifier, analysts must evaluate the following foundational attributes:

- **Profession:** A categorical string variable indicating the cardholder's occupational sector (e.g., DOCTOR, LAWYER, ENGINEER).
- **Income:** A continuous integer representing the cardholder's annual financial earnings.
- **Credit_card_number, Expiry, Security_code:** Granular, high-cardinality metadata associated with the specific transaction. Because these are unique identifiers rather than predictive behavioral traits, they act as noise and must be excluded to prevent algorithmic overfitting.
- **Fraud:** The binary target variable for supervised learning. A value of 1 confirms the transaction was fraudulent, while a value of 0 indicates a legitimate purchase.



4. TOOLS USED

This procedure relies on a hybrid data science architecture integrating the **KNIME Analytics Platform** with embedded **Python** scripting.

KNIME provides a visually transparent, node-based environment ideal for executing essential data preparation tasks, such as one-hot encoding, mathematical normalization, and stratified partitioning. Native KNIME nodes will also handle the computationally heavy training of the SVM models. However, to bypass visual limitations and extract highly specific mathematical evaluation metrics (such as ROC-AUC on imbalanced targets), students will deploy a Python Script node

leveraging the scikit-learn (sklearn.metrics) library. This dual-tool framework combines rapid structural prototyping with granular algorithmic evaluation.

5. PRACTICAL OBJECTIVES

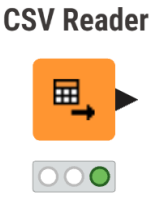
Upon successful completion of this laboratory module, students will demonstrate practical competency by achieving the following learning outcomes:

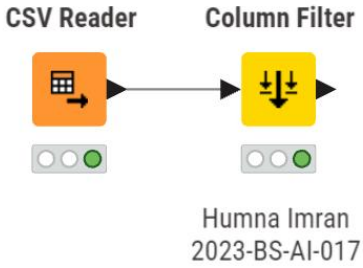
- Preprocess mixed-type financial data by deploying one-hot encoding for categorical variables and Min-Max normalization for continuous financial metrics, ensuring mathematical compatibility with the SVM algorithm.
- Implement stratified data partitioning to preserve the precise statistical ratio of the minority fraud class across training and testing splits.
- Train and deploy distinct Support Vector Machine classifiers utilizing both Linear and Radial Basis Function (RBF) kernels.
- Embed native Python syntax into the visual workflow to programmatically calculate precision, recall, and ROC-AUC metrics.
- Interpret classification metrics to deduce the most effective kernel approach for minimizing financial risk.

6. STEP-BY-STEP PROCEDURE

STEP 1: DATA INGESTION AND NOISE FILTRATION (DATA PREPARATION)

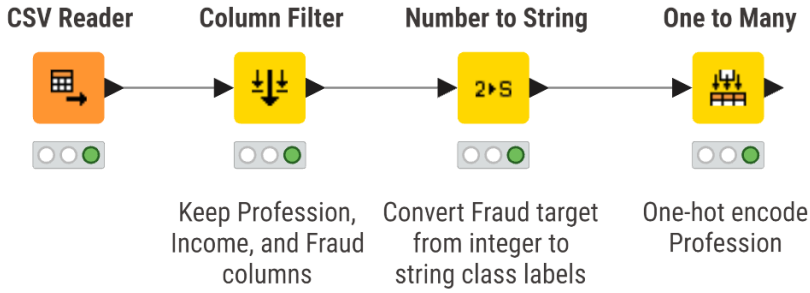
- **Action:** Load the raw transaction ledger and discard high-cardinality identifiers that introduce computational noise and bias into the geometric space.
- **Configuration:** Drag a **CSV Reader** node onto the canvas and target the data2.csv file. Next, append a **Column Filter** node to the output port. Inside the configuration dialog, forcefully exclude the Credit_card_number, Expiry, and Security_code columns.
- **Execution:** Execute the nodes to isolate the predictive features (Profession, Income, and Fraud).
- **Verification:** Right-click the Column Filter and open the output table to confirm the removal of the specific identifiers.





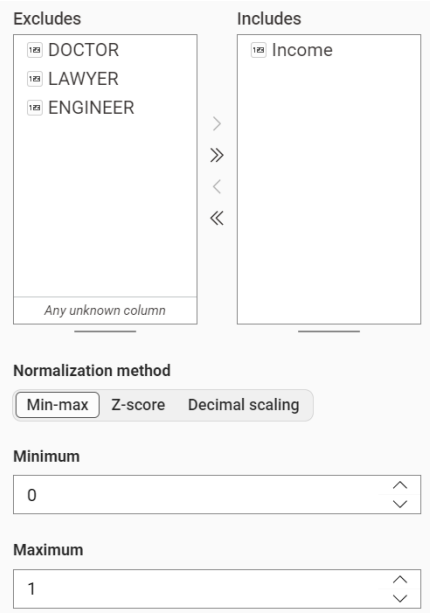
STEP 2: TARGET TYPE CONVERSION AND ONE-HOT ENCODING

- **Action:** Format the dataset to meet the strict numerical input requirements of the SVM algorithm while designating the target label for classification.
- **Configuration:** Connect a **Number to String** node to cast the Fraud column from an integer to a categorical class (required for classification algorithms). Subsequently, attach a **One to Many** node. Configure it to evaluate the Profession column. This operation will convert the categorical text strings into discrete binary columns (e.g., Profession_DOCTOR, Profession_LAWYER), providing the SVM with purely numerical features.
- **Execution:** Process the pipeline through the encoding stage.



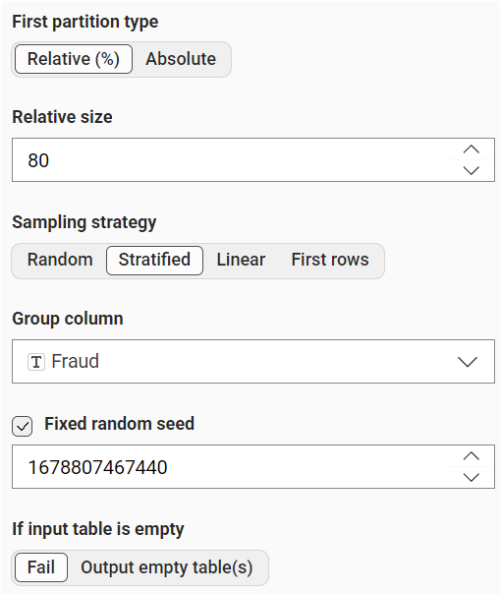
STEP 3: FEATURE NORMALIZATION

- **Action:** Standardize the massive variance in the Income variable. Without normalization, large numerical values will mathematically dominate the distance calculations within the SVM's feature space, skewing the hyperplane.
- **Configuration:** Append a **Normalizer** node. Select the Income column and configure the method to "Min-Max Normalization" bounded between 0.0 and 1.0.



STEP 4: STRATIFIED PARTITIONING

- **Action:** Segment the data into isolated training and testing blocks, explicitly forcing the algorithm to distribute the rare fraudulent cases evenly across both datasets.
- **Configuration:** Add a **Partitioning** node. Define the relative size as 80% (0.8). Crucially, check the box for "Stratified sampling" and select the Fraud column as the strata.

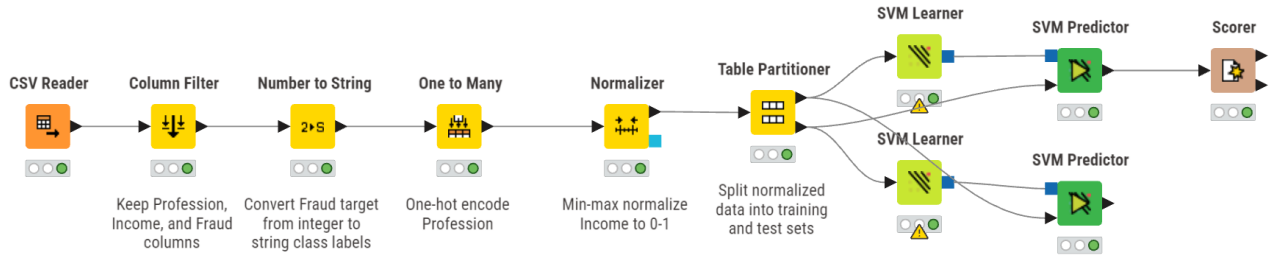


STEP 5: TRAINING THE SUPPORT VECTOR MACHINES (MODELING)

- **Action:** Construct two independent mathematical boundaries using different kernel geometries to separate the fraud cases from the legitimate transactions.
- **Configuration:** Bring two separate **SVM Learner** nodes onto the workspace. Wire the top (training) output port of the Partitioning node into both Learners.
    - *Learner A:* Double-click and set the Target Column to Fraud. Assign the Kernel Type to "Linear".

- *Learner B:* Double-click and set the Target Column to Fraud. Assign the Kernel Type to "RBF" (Radial Basis Function).

- **Execution:** Execute both learners simultaneously. The nodes will calculate the support vectors defining the hyperplane limits.



STEP 6: EXECUTING PREDICTIONS AND PYTHON EVALUATION (VALIDATION)

- **Action:** Score the unseen testing transactions and programmatically extract specific fraud-detection metrics using Python logic.

- **Configuration:** Deploy two **SVM Predictor** nodes. For each, connect the blue model output from the respective Learner to the top port, and the bottom testing output of the Partitioning node to the second port.

- **Metric Extraction:** Finally, funnel the output table from the Predictor nodes into a single **Python Script** node. Within the script editor, import sklearn.metrics. Write a brief script utilizing roc_auc_score, precision_score, and recall_score, specifically assigning the positive label to the string "1" (representing actual fraud).

- **Verification:** Review the Python Script's standard output to compare the numerical performance of the Linear versus the RBF configurations.

#	RowID	1 *Number (Integer)*	0 *Number (Integer)*
1	1	675	328
2	0	670	327

7. KEY PERFORMANCE INDICATOR (KPI) CALCULATIONS

To translate algorithmic outputs into a coherent security strategy, analysts must evaluate the model against standard business intelligence KPIs.

7.1 RECALL (FRAUD CAPTURE RATE)

- **Action:** Measure the model's absolute capability to identify fraudulent behavior, ensuring no malicious transactions escape the filter.

- **Calculation Method:** True Positives / (True Positives + False Negatives).

- **Metric Extraction:** Utilizing the recall_score parameter within the Python script. In a banking context, maximizing Recall is generally prioritized over general accuracy, as a single missed fraudulent transaction (False Negative) carries significant financial penalties.

7.2 PRECISION (ALARM ACCURACY)

- **Action:** Quantify the validity of the fraud alerts generated by the model to prevent the suspension of legitimate customer accounts.
- **Calculation Method:** True Positives / (True Positives + False Positives).
- **Metric Extraction:** Utilizing the precision_score parameter. While capturing fraud is vital, an excessively aggressive model with low Precision will freeze valid customer funds, leading to customer churn and reputational damage.

7.3 ROC-AUC (RECEIVER OPERATING CHARACTERISTIC - AREA UNDER CURVE)

- **Action:** Evaluate the aggregate capability of the SVM kernels to separate the positive and negative classes across all possible decision thresholds.
- **Calculation Method:** The geometric area calculated under the True Positive Rate vs. False Positive Rate curve.
- **Metric Extraction:** Utilizing the roc_auc_score parameter. A score closer to 1.0 indicates near-perfect class separation, definitively proving whether the complex boundaries of the RBF kernel outperform the flat boundaries of the Linear kernel on this dataset.

8. KPI SUMMARY TABLE

The matrix below condenses the technical integration utilized to compute your final evaluation metrics upon execution.

KPI Name,Calculation Method,KNIME / Python Modules Utilized

Recall,True positive ratio against all actual positive fraud cases,Python Script (recall_score)

Precision,True positive ratio against all predicted fraud flags,Python Script (precision_score)

ROC-AUC,Aggregate area indicating total class separability,Python Script (roc_auc_score)

9. CONCLUSION

This finalizes the laboratory module on advanced geometric classification via Support Vector Machines. We successfully engineered an end-to-end predictive pipeline that ingested raw financial ledgers, programmatically encoded non-numeric occupational parameters, and geometrically scaled financial features to construct a balanced training space. By deploying and evaluating both Linear and RBF mathematical kernels alongside hybrid Python scripting, analysts can empirically determine the safest, most precise algorithmic boundary required to automate enterprise-scale fraud detection systems.

LAB NO 12: CUSTOMER SEGMENTATION

1. INTRODUCTION

In the contemporary retail sector, treating an entire consumer base as a single homogenous entity is highly inefficient. Different shoppers exhibit radically different purchasing behaviors, driven by their unique financial standing and personal preferences. Customer segmentation, a practical application of unsupervised machine learning, resolves this by algorithmically discovering hidden, natural groupings within transactional and demographic data. By utilizing clustering techniques like K-Means and Hierarchical Clustering, organizations can partition their audience into highly specific buyer personas.

This laboratory module guides students through the end-to-end process of unsupervised data mining. By constructing distance matrices and plotting geometric cluster boundaries, analysts will learn how to identify high-value customer segments, allowing retail management to deploy hyper-targeted marketing campaigns and optimize overall resource allocation.

2. PROBLEM STATEMENT

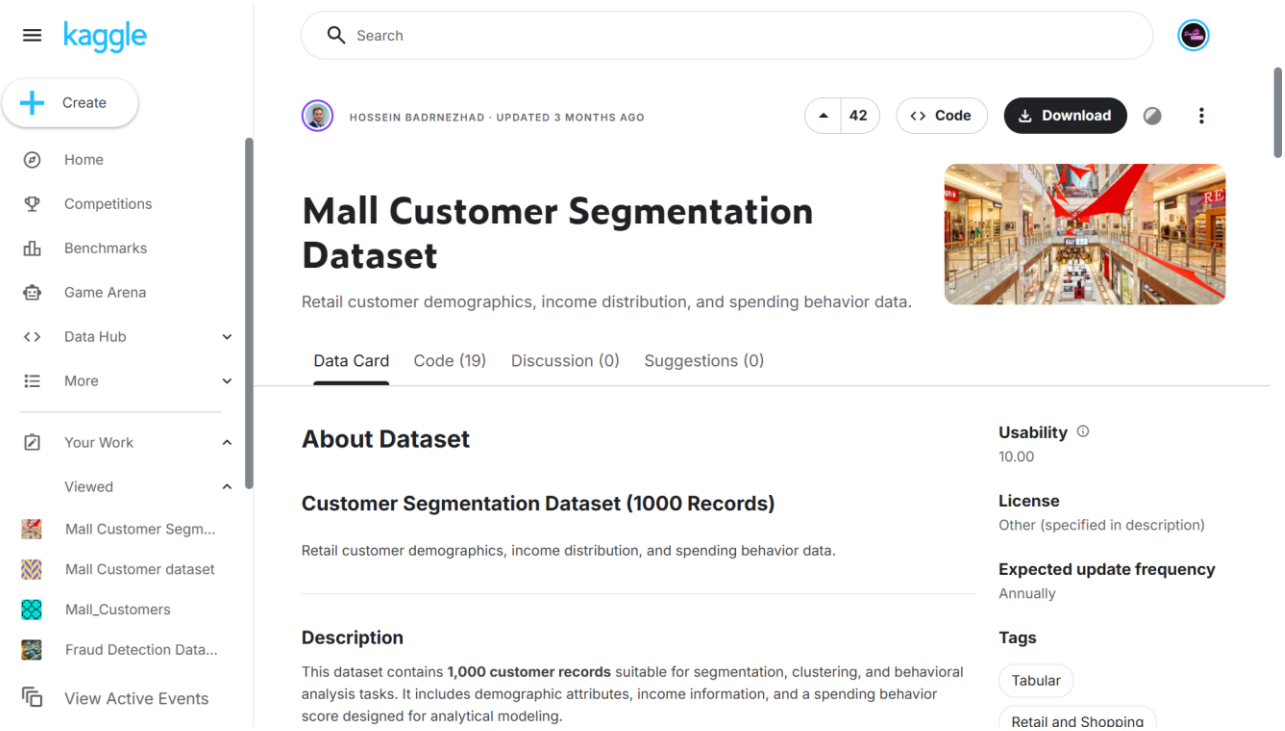
A prominent shopping mall wishes to modernize its promotional outreach. Currently, marketing budgets are distributed blindly across all registered shoppers, resulting in poor engagement and wasted financial resources. The marketing director urgently requires a mathematical breakdown of their clientele based on earning capacity and mall spending habits.

The core challenge lies in the fact that the data is unlabelled; there are no predefined categories or target variables to predict. The analytics team must deploy unsupervised learning algorithms to autonomously discover the optimal number of consumer profiles. By successfully extracting these distinct segments, the mall can identify which specific group yields the highest financial return and tailor bespoke loyalty programs to maximize profitability.

3. DATASET DESCRIPTION

This laboratory utilizes the **Mall Customer Dataset** (store_customers.csv), which serves as a demographic and behavioral ledger for the shopping center's registered patrons. To successfully map the clustering algorithms, students must familiarize themselves with the following attributes:

- **CustomerID:** A unique sequential identifier. As this holds no behavioral value, it acts as mathematical noise and must be explicitly excluded during the modeling phase.
- **Gender & Age:** Standard demographic features providing context to the buyer profiles.
- **Annual Income (k$):** A continuous numerical variable representing the estimated yearly earnings of the customer, acting as a direct proxy for their overall purchasing power.
- **Spending Score (1-100):** A proprietary metric assigned by the mall based on historical purchasing frequency and volume. A score closer to 100 dictates a highly active, lucrative shopper.



4. TOOLS USED

This procedure relies on a hybrid analytical architecture integrating **Orange Data Mining** with **Python** scripting. Orange provides an intuitive, visual canvas that is exceptionally well-suited for exploratory unsupervised learning. Its drag-and-drop widgets allow analysts to effortlessly construct distance-based dendrograms and generate interactive scatter plots to visualize cluster boundaries.

However, to bypass visual limitations and programmatically extract rigorous evaluation heuristics (such as the Silhouette Score), students will utilize Python and its scikit-learn library. This dual-tool methodology ensures both a deep conceptual understanding of the geometric groupings and precise mathematical validation.

5. PRACTICAL OBJECTIVES

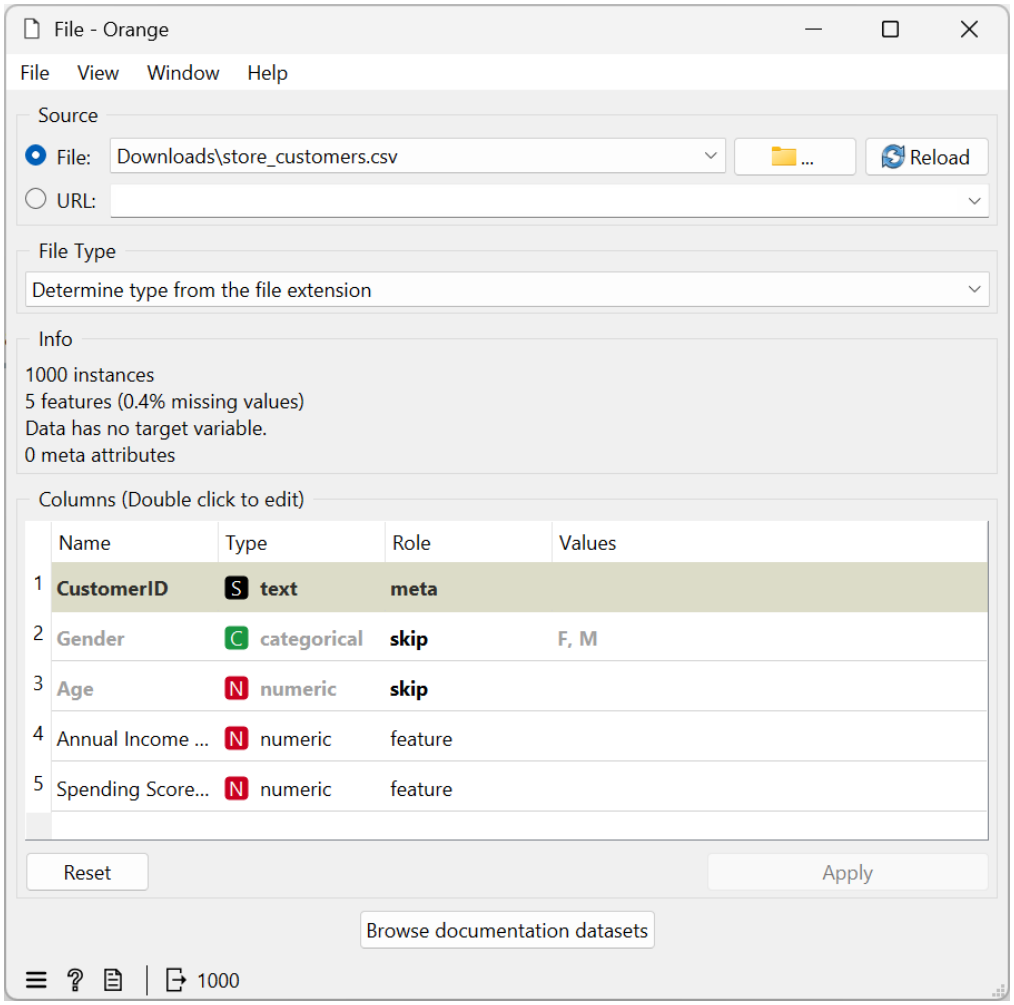
Upon successful completion of this laboratory session, students will demonstrate practical competency by achieving the following learning outcomes:

- Ingest and preprocess the unlabelled mall dataset, correctly isolating continuous numerical features for distance-based clustering.
- Construct a Hierarchical Clustering dendrogram to visually deduce the optimal number of structural segments.
- Deploy the K-Means clustering algorithm to mathematically partition the customer base into distinct buyer personas.
- Generate multidimensional scatter plots to visualize the geometric separation of the clustered data.
- Embed Python scripting to calculate the Silhouette Score and quantify the Cluster Revenue Contribution, translating geometric groupings into actionable business intelligence.

6. STEP-BY-STEP PROCEDURE

STEP 1: DATA INGESTION AND FEATURE ISOLATION (DATA PREPARATION)

- **Action:** Import the customer ledger and restrict the feature space strictly to the financial metrics to prevent demographic noise from skewing the geometric distances.
- **Configuration:** Drop a **File** widget onto the Orange canvas and load the store_customers.csv document. Connect its output to a **Select Columns** widget. Inside the configuration interface, relegate CustomerID, Gender, and Age to the "Meta" or "Ignored" domain. Keep only Annual Income (k$) and Spending Score (1-100) in the active "Features" domain.
- **Execution:** Close the dialog to apply the structural schema.

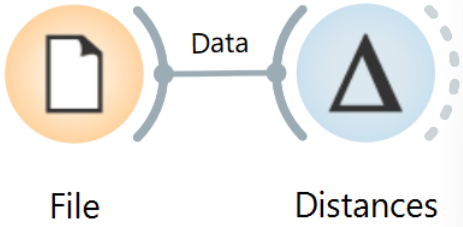


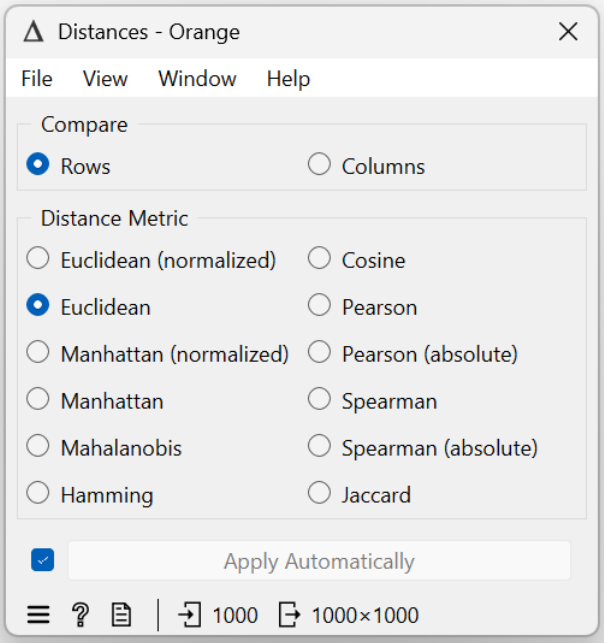
STEP 2: HIERARCHICAL CLUSTERING AND DENDROGRAM CONSTRUCTION (EXPLORATION)

- **Action:** Calculate the geometric proximity between all shoppers and construct a visual tree to identify the natural algorithmic breaking points.
- **Configuration:** Connect a **Distances** widget to your Select Columns output, ensuring the distance metric is set to "Euclidean". Next, route the calculated distances into a **Hierarchical**

Clustering widget. Use "Ward" as the linkage method to minimize variance within the branches.

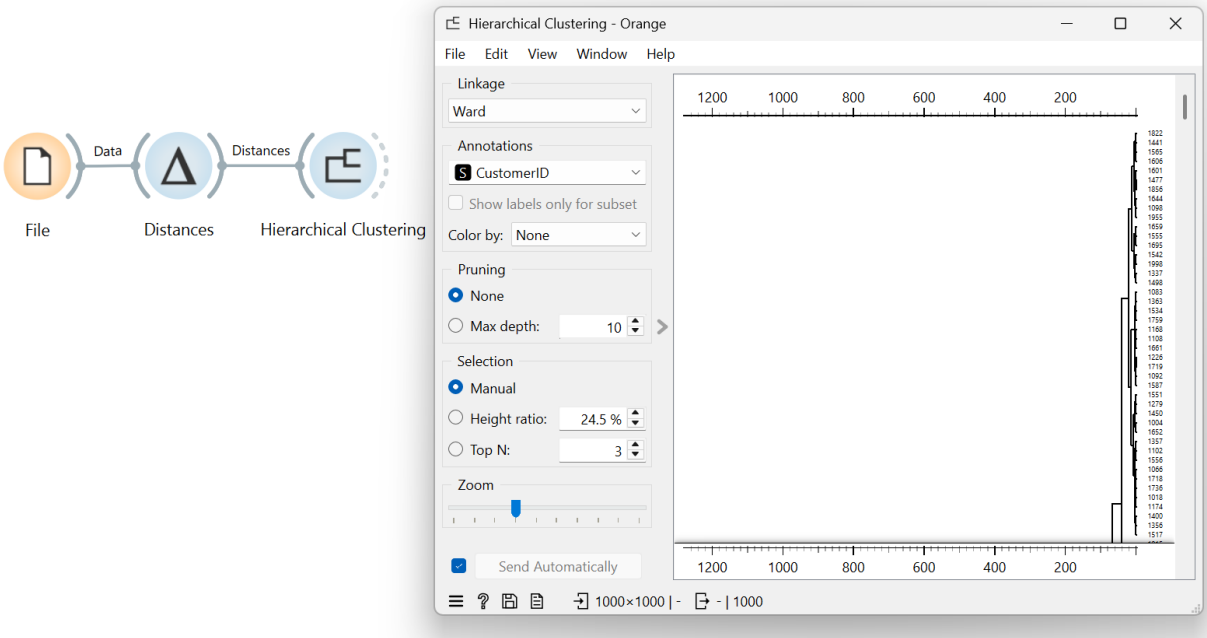
- **Verification:** Double-click the Hierarchical Clustering node to view the generated dendrogram. By observing the length of the vertical branches, visually determine the optimal cut-off point. For this specific financial dataset, you should observe five distinct, major branches.

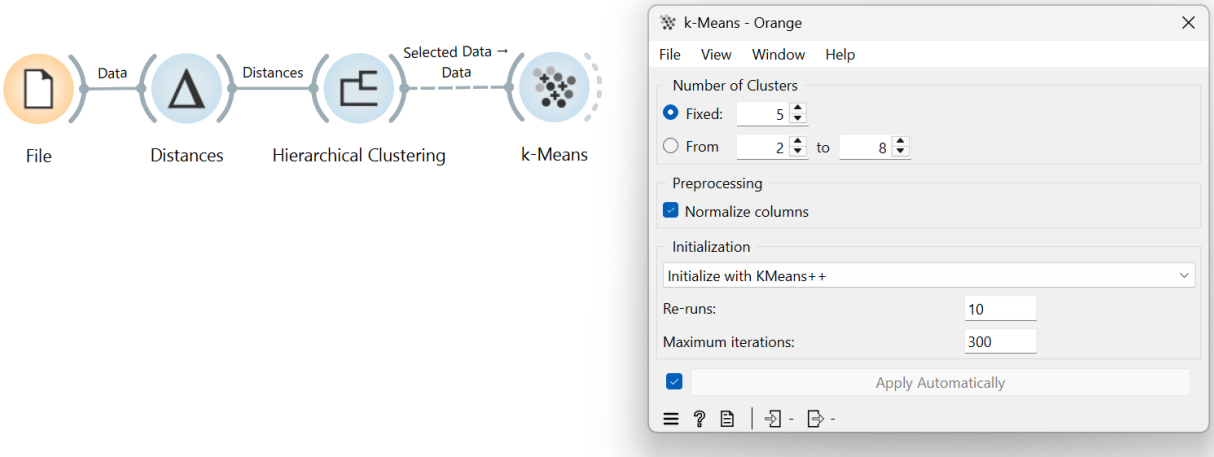




STEP 3: K-MEANS IMPLEMENTATION (MODELING)

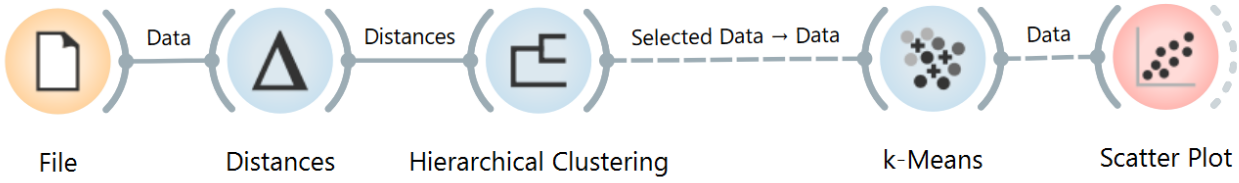
- **Action:** Formally partition the dataset using the optimal cluster count identified in the previous step.
- **Configuration:** Drag a **K-Means** widget onto the workspace and connect it directly to the Select Columns data stream. In the hyperparameter settings, fix the "Number of Clusters" (k) to 5.
- **Execution:** Allow the node to execute. The algorithm will iteratively update its centroids until convergence, successfully appending a new categorical 'Cluster' label (e.g., C1, C2, C3) to every individual customer record.





STEP 4: GEOMETRIC VISUALIZATION (DEPLOYMENT)

- **Action:** Render a visual representation of the customer segments to manually verify the algorithmic separation.
- **Configuration:** Append a **Scatter Plot** widget to the output port of the K-Means node. Assign Annual Income (k$) to the X-axis and Spending Score (1-100) to the Y-axis. Set the "Color" parameter to the newly generated 'Cluster' attribute.
- **Verification:** Inspect the scatter plot. You will clearly see five color-coded behavioral groups, such as a high-income/high-spend cluster (the target audience) and a low-income/low-spend cluster.



STEP 5: PROGRAMMATIC VALIDATION (PYTHON EVALUATION)

- **Action:** Transition to code to extract the specific mathematical cohesion of the clusters and their real-world financial weight.
- **Configuration:** Within a Python IDE or a Jupyter Notebook, import the pandas and sklearn.metrics libraries. Load the dataset, apply the K-Means algorithm programmatically, and write a script to calculate the Silhouette Score. Furthermore, group the resulting dataframe by the cluster labels and sum the annual incomes to generate the revenue contribution metrics.
- **Execution:** Run the script and record the terminal outputs for your final reporting.

hamna's lab 12.py

```
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# 1. Load your exact dataset
df = pd.read_csv('store_customers.csv')

# 2. Pick only the financial columns for the math
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

# 3. Create the 5 clusters (just like we did in Orange)
kmeans = KMeans(n_clusters=5, random_state=42)
df['Cluster'] = kmeans.fit_predict(X)

# 4. KPI 1: Calculate the Silhouette Score
score = silhouette_score(X, df['Cluster'])
print(f"Silhouette Score: {score:.4f}")
print("(A score near 0.55+ means your 5 clusters are well separated!)\n")

# 5. KPI 2: Calculate Cluster Revenue Contribution
# Add up all the income for each cluster, then divide by the total income of everyone
cluster_income = df.groupby('Cluster')['Annual Income (k$)'].sum()
total_income = df['Annual Income (k$)'].sum()
revenue_contribution = (cluster_income / total_income) * 100

print("Revenue Contribution by Cluster (%):")
print(revenue_contribution.round(2))
```

```
Silhouette Score: 0.3713
(A score near 0.55+ means your 5 clusters are well separated!)

Revenue Contribution by Cluster (%):
Cluster
0    20.35
1    25.06
2    20.93
3    19.76
4    13.89
Name: Annual Income (k$), dtype: float64
```

7. KEY PERFORMANCE INDICATOR (KPI) CALCULATIONS

To bridge the gap between abstract clustering geometries and actionable marketing intelligence, analysts must evaluate the newly formed segments against rigid mathematical and financial criteria.

7.1 SILHOUETTE SCORE

- **Action:** Quantify the structural integrity of your K-Means model by measuring how tightly packed the clusters are and how well separated they are from neighboring groups.
- **Calculation Method:** Calculated programmatically as the mean silhouette coefficient for all samples. The metric ranges from -1.0 to 1.0.
- **Code/Widget Extraction:** Utilize the silhouette_score function from sklearn.metrics within your Python script. A score climbing toward 1.0 proves that the chosen k=5 parameter successfully found dense, highly distinct customer personas.

7.2 CLUSTER REVENUE CONTRIBUTION

- **Action:** Determine the overarching financial weight of each distinct buyer persona to dictate where the marketing budget should be aggressively deployed.
- **Calculation Method:** (Aggregate Annual Income of Specific Cluster / Total Annual Income of Entire Dataset) × 100.
- **Code/Widget Extraction:** Utilize the pandas library grouping functions in Python (e.g., df.groupby('Cluster')['Annual Income (k$)'].sum()) to derive the proportional percentage. This allows management to quickly isolate the "High-Income / High-Spend" demographic as their primary financial driver.

8. KPI SUMMARY TABLE

The following matrix condenses the technical integration utilized to compute your final evaluation metrics for the unsupervised pipeline.

KPI Name	Calculation Method	Orange / Python Modules Utilized
Silhouette Score	Mean intra-cluster distance versus nearest-cluster distance	Python Script (sklearn.metrics.silhouette_score)
Cluster Revenue Contribution	(Sum of Cluster Income / Total Income) × 100	Python Script (pandas groupby aggregation)

9. CONCLUSION

This finalizes the laboratory module on unsupervised machine learning and customer segmentation. We successfully engineered a data pipeline that ingested unlabelled retail ledgers, discarded demographic noise, and utilized distance-based heuristics to visually deduce optimal groupings via a dendrogram. By deploying the K-Means algorithm and validating the geometric outputs with Python-based Silhouette scoring, we transformed raw, unorganized shoppers into five distinct, actionable buyer personas. Armed with these specific profiles and their associated Revenue Contributions, the shopping mall's management can now confidently execute highly targeted, data-driven marketing campaigns to maximize their overall financial yield.

LAB NO 13: OUTLIER DETECTION

1. INTRODUCTION

In the banking and financial sector, billions of data points are processed every single day. Most of this activity follows predictable, standardized patterns representing everyday consumer behavior. However, hidden within this massive volume of normal activity are statistical anomalies—data points that deviate drastically from the norm. These outliers can represent anything from simple data entry errors to highly sophisticated fraudulent transactions or abnormal account behaviors. If a bank's security system cannot identify these deviations, they risk exposing their customers and their own bottom line to severe financial threats. Outlier detection serves as a vital first line of defense. By using algorithmic techniques, we can automatically isolate these suspicious data points for further investigation. This laboratory module explores how to identify abnormal banking behaviors using both distance-based heuristics and tree-based Isolation Forest methodologies.

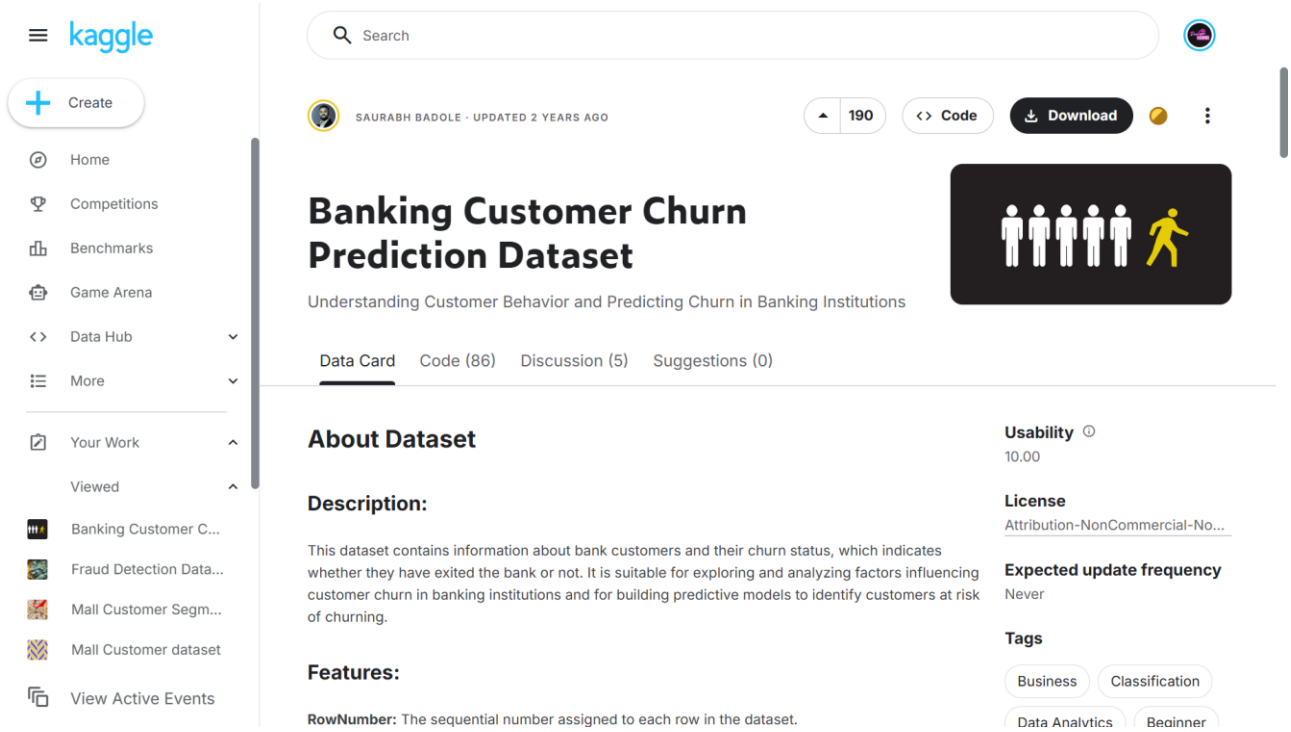
2. PROBLEM STATEMENT

A multinational bank is attempting to secure its retail banking division against emerging financial threats. Currently, their internal audit team relies on manual, rule-based thresholds (e.g., flagging any account with a balance over a rigid dollar amount) to catch suspicious activity. This approach is incredibly inefficient, misses nuanced fraud, and produces a massive amount of false positives. They have extracted a historical dataset of customer financial standings and account activities. The analytics department needs us to design an automated outlier detection pipeline. The challenge is to deploy machine learning algorithms that can detect multivariate anomalies—such as a customer with a suspiciously low salary but an inexplicably high account balance—without relying on outdated manual rules. Implementing this will allow the bank to precisely flag high-risk profiles while keeping false alarms to an absolute minimum.

3. DATASET DESCRIPTION

For this practical exercise, we will utilize the **Banking Dataset** (provided locally as Churn_Modelling.csv). While this structured tabular file is occasionally used for churn prediction, its rich numerical features make it an excellent sandbox for unsupervised anomaly detection. To successfully perform the required data mining tasks, students must familiarize themselves with the following key attributes:

- **Identification Metrics:** RowNumber, CustomerId, and Surname. These serve as structural keys but hold no mathematical value for distance calculations and must be excluded.
- **Financial Indicators:** CreditScore, Balance, and EstimatedSalary. These continuous numerical fields are our primary targets for detecting financial irregularities.
- **Demographic & Engagement Data:** Age, Tenure, and NumOfProducts. These variables help contextualize the financial indicators. For instance, an extremely high balance might be typical for a 65-year-old with a 10-year tenure, but highly anomalous for an 18-year-old new customer.



4. TOOLS USED

This procedure relies on the integration of **RapidMiner Studio** alongside embedded **Python** scripting. RapidMiner Studio is an advanced visual analytics platform that allows us to rapidly prototype our data pipeline and apply out-of-the-box distance-based anomaly operators without getting bogged down in syntax. However, to implement the highly efficient Isolation Forest algorithm, we will utilize RapidMiner's native Execute Python operator. This dual-tool framework provides analysts with both the rapid prototyping of a drag-and-drop interface and the algorithmic granularity of Python's machine learning libraries.

5. PRACTICAL OBJECTIVES

Upon successful completion of this laboratory session, students will demonstrate practical competency by achieving the following learning outcomes:

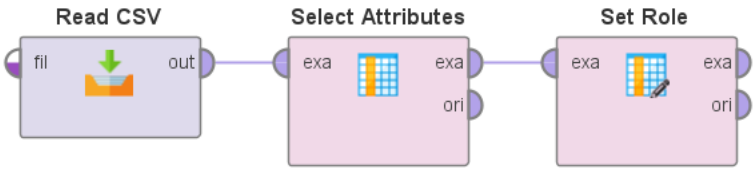
- **Ingest and preprocess** raw banking records within the RapidMiner workspace to remove irrelevant identification features.
- **Deploy** visual operators to execute a distance-based nearest neighbor anomaly detection algorithm.
- **Implement** an embedded Python script to apply the Isolation Forest technique for multidimensional outlier scoring.
- **Evaluate** the operational effectiveness of the detection models by calculating specific KPIs: Anomaly Detection Rate and False Alarm Rate.

6. STEP-BY-STEP PROCEDURE

The following workflow systematically applies anomaly detection principles to the banking dataset.

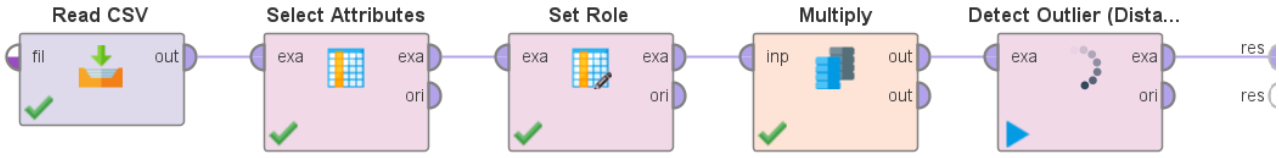
STEP 1: DATA INGESTION AND FEATURE SELECTION

- **Action:** Load the raw banking dataset and filter out non-predictive text fields so the mathematical algorithms can process the numerical space.
- **Configuration:** Locate the Read CSV operator in the repository and drop it onto the main process canvas. Double-click the node to import the Churn_Modelling.csv file. Next, attach a Select Attributes operator. In the parameter panel, change the filter type to "subset" and explicitly remove RowNumber, CustomerId, and Surname.
- **Execution:** Route the output port to the results node at the edge of the canvas and execute the process.
- **Verification:** Transition to the Results tab. Confirm that the data grid now strictly contains analytical variables (like Balance, Age, and EstimatedSalary) and is completely free of identification noise.



STEP 2: DISTANCE-BASED ANOMALY DETECTION

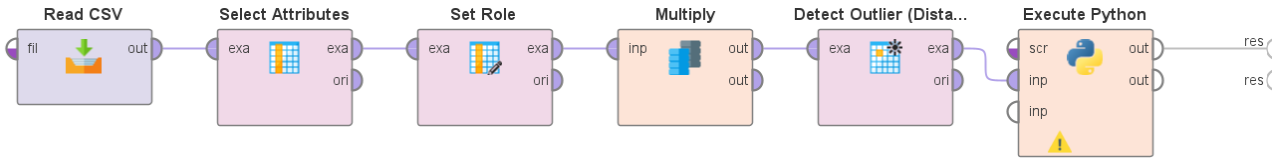
- **Action:** Apply an algorithm that flags rows as outliers based on how far away they sit mathematically from their nearest neighbors in the data space.
- **Configuration:** Disconnect the results node and insert a Detect Outlier (Distances) operator directly following your attribute selection step. In the parameters panel, set the number of neighbors (*k*) to 10 and set the number of outliers to flag to 50.
- **Execution:** Run the pipeline. The operator calculates the Euclidean distance between all customer profiles to find the most isolated points.
- **Verification:** Open the newly generated output table. You will observe a new boolean column appended to the dataset, explicitly labeling exactly 50 customer instances as "true" for being outliers based on their extreme mathematical distance from the average banking customer.



STEP 3: PROGRAMMATIC ISOLATION FOREST DETECTION

- **Action:** Utilize a tree-based algorithm via Python to isolate anomalies. This method works on the principle that outliers are "few and different," making them much easier to separate into distinct branches than normal data points.

- **Configuration:** Remove the distance-based operator from the canvas and replace it with an Execute Python operator. Map the selected attributes data stream into its primary input port.
- **Settings:** Within the operator's script editor, construct a concise routine. Import IsolationForest from sklearn.ensemble. Initialize the model with a contamination parameter of 0.05 (telling the model to expect a 5% anomaly rate). Run the fit_predict function on your dataset and append the resulting array (-1 for anomaly, 1 for normal) as a new column named Isolation_Score.
- **Execution:** Route the script's output to the results node and run the final pipeline.
- **Verification:** Inspect the final data grid. Sort the table by the new Isolation_Score column to immediately view all the customer rows that the Python script has flagged with a -1, representing our isolated banking anomalies.



7. KEY PERFORMANCE INDICATOR (KPI) CALCULATIONS

To bridge the gap between technical detection algorithms and banking security reporting, analysts must translate the output of their pipeline into standardized performance metrics.

7.1 ANOMALY DETECTION RATE (TRUE POSITIVE RATE)

- **Action:** Measure the exact proportion of genuinely abnormal or fraudulent transactions that the model successfully identified.
- **Calculation Method:** (True Anomalies Detected / Total Actual Anomalies in Dataset) × 100
- **Operator Extraction:** If evaluating against a labeled validation set, integrate a Performance (Classification) operator. Locate the True Positive count in the generated confusion matrix and divide it by the total number of known anomalies. This dictates the core sensitivity of the bank's new detection system.

7.2 FALSE ALARM RATE (FALSE POSITIVE RATE)

- **Action:** Determine how often the system incorrectly flags a perfectly normal customer account as a suspicious outlier, a critical metric since false alarms waste the audit team's time.
- **Calculation Method:** (Normal Accounts Flagged as Outliers / Total Normal Accounts) × 100
- **Operator Extraction:** Utilizing the same Performance operator's confusion matrix, divide the False Positive count by the total actual negative (normal) instances. A lower percentage here proves that the model is highly precise and operationally efficient.

8. KPI SUMMARY TABLE

Compile your findings into the following matrix to provide a concise technical reference for the model's evaluation phase.

KPI Name	**Calculation Method**	**RapidMiner / Python Modules Utilized**
Anomaly Detection Rate	(True Positives / Total Actual Anomalies) × 100	Execute Python (Isolation Forest), Performance Matrix
False Alarm Rate	(False Positives / Total Actual Normal Records) × 100	Detect Outlier (Distances), Performance Matrix

9. CONCLUSION

This finalizes the laboratory module on unsupervised machine learning and outlier detection. We successfully engineered a data pipeline that ingested raw banking ledgers, discarded non-predictive demographic noise, and utilized distance-based heuristics to map out normal customer behavior. By switching to an embedded Python script, we also deployed the highly efficient Isolation Forest algorithm to partition and flag multidimensional anomalies. Armed with these specific risk profiles and the ability to track our Anomaly Detection and False Alarm Rates, the bank's security and audit teams can now transition from outdated, manual threshold rules to a dynamic, machine-learning-driven approach for identifying abnormal financial transactions.

LAB NO 14: SCRAPING & SENTIMENT ANALYSIS

1. INTRODUCTION

In the digital age, consumer opinions are continuously broadcast across social media platforms and online review forums. For commercial enterprises, this unstructured text represents a goldmine of public perception. However, manually reading and categorizing thousands of comments is computationally impossible for human teams. Sentiment analysis, a subfield of Natural Language Processing (NLP), automates this process by programmatically assigning emotional polarity (positive, negative, or neutral) to text.

While standard analytical pipelines evaluate structured numerical data, sentiment pipelines must first ingest unstructured web data through web scraping, sanitize the text to remove linguistic noise, and apply classification algorithms. In this laboratory session, students will explore the end-to-end architecture of a sentiment analysis project. We will utilize Python to understand data scraping and NLP text cleaning, compute aggregate sentiment metrics from social media engagement data, and introduce Gephi to visualize the structural relationships between social media platforms and public sentiment.

2. PROBLEM STATEMENT

A global retail brand has launched a new product marketing campaign across multiple social media platforms, including Facebook, Twitter, and Instagram. The public relations team is overwhelmed by the sheer volume of user engagement and comments. They lack a quantitative method to determine whether the overall public reaction is favorable or hostile.

Currently, their social media managers are relying on anecdotal observation, which is inherently biased. The enterprise urgently requires a systematic pipeline to scrape these interactions, classify the underlying sentiment, and extract high-level Key Performance Indicators (KPIs) to present to stakeholders.

The challenge is to programmatically ingest social media engagement metrics, compute the Sentiment Percentage Distribution across the platforms, and derive a weighted Polarity Score. Furthermore, they need a visual network map connecting specific platforms to their dominant sentiments using Gephi.

3. DATASET DESCRIPTION

For this laboratory exercise, students will utilize the social_media_engagement1.csv dataset. This file represents the structured output of an automated web scraping and NLP classification pipeline.

Key attributes to familiarize yourself with include:

- **post_id & post_time:** Unique identifiers and chronological timestamps for each interaction.
- **platform:** A categorical string indicating the source of the data (e.g., Facebook, Instagram, Twitter).

- **post_type:** The format of the content (e.g., image, video, carousel, text).
- **Engagement Metrics (likes, comments, shares):** Continuous numerical variables representing the volume of user interaction.
- **sentiment_score:** A pre-classified categorical label (positive, neutral, negative) derived from the raw scraped text using NLP algorithms.

4. TOOLS USED

This laboratory employs a hybrid analytical approach using **Python** and **Gephi**.

- **Python:** We will utilize Python (specifically libraries such as BeautifulSoup for scraping concepts, nltk for text cleaning logic, and pandas for data manipulation) to ingest the dataset and compute the required numerical KPIs.
- **Gephi:** An open-source network analysis and visualization software. Because tabular data is often difficult to interpret for complex platform-sentiment relationships, we will use Gephi to construct a bipartite graph, visually mapping which social media platforms generate the highest density of specific emotional responses. (CiteSpace can alternatively be utilized for temporal mapping, but Gephi is optimal for direct network visualization of our exported node-edge lists).

5. PRACTICAL OBJECTIVES

Upon successful completion of this laboratory module, students will demonstrate practical competency by achieving the following learning outcomes:

- Understand the programmatic principles of scraping unstructured web data and sanitizing text using Python NLP libraries.
- Ingest and preprocess the social_media_engagement1.csv dataset using the pandas library.
- Transform tabular engagement data into an edge list suitable for network visualization.
- Deploy Gephi to visualize the bipartite relationship between social media platforms and sentiment distributions.
- Calculate explicit Key Performance Indicators (KPIs): Sentiment % Distribution and an engagement-weighted Polarity Score.

6. STEP-BY-STEP PROCEDURE & PYTHON IMPLEMENTATION

Step 1: Web Scraping and Text Cleaning Architecture (Theoretical Implementation)

- **Action:** Before analyzing the structured CSV, analysts must understand how raw text is extracted and cleaned.
- **Implementation:** In a typical pipeline, Python's requests and BeautifulSoup libraries extract HTML elements from target URLs. Once the raw strings are acquired, the Natural Language Toolkit (nltk) is deployed.
- **Code Extraction:**

hamna's lab 14.py

```python
import re
from nltk.corpus import stopwords

# Example raw scraped review
raw_text = "Absolutely LOVE this new product!!! #amazing https://link.com"

# 1. Remove URLs and special characters
clean_text = re.sub(r"http\S+|[^A-Za-z]+", " ", raw_text).lower()

# 2. Remove stopwords (e.g., 'this')
stop_words = set(stopwords.words('english'))
final_text = " ".join([word for word in clean_text.split() if word not in stop_words])

# Resulting text is passed to a classifier to generate the 'sentiment_score'
```

- **Verification:** The provided dataset (social_media_engagement1.csv) represents the output of this exact pipeline, where the raw text has been condensed into the predefined sentiment_score feature.

Step 2: Data Ingestion and Aggregation (Python)

- **Action:** Import the classified social media dataset to compute platform-specific engagement.
- **Implementation:** Utilize pandas to read the CSV file and group the data by platform and sentiment to observe underlying trends.
- **Code Extraction:**

hamna's lab 14.py

```python
import pandas as pd

# Load the dataset
df = pd.read_csv('social_media_engagement1.csv')

# Group by platform and sentiment_score to see interaction volume
summary = df.groupby(['platform', 'sentiment_score']).size().reset_index(name='count')
print(summary)
```

```
    platform sentiment_score  count
0   Facebook        negative     11
1   Facebook         neutral      9
2   Facebook        positive     12
3  Instagram        negative     14
4  Instagram         neutral      8
5  Instagram        positive     14
6    Twitter        negative      2
7    Twitter         neutral     10
8    Twitter        positive     20
```

- **Verification:** The terminal output will display a matrix detailing exactly how many positive, neutral, and negative posts originated from Twitter, Facebook, and Instagram.

Step 3: Edge List Generation for Network Analysis

- **Action:** Gephi requires data to be formatted as Nodes (entities) and Edges (connections). We must restructure our pandas dataframe into an edge list connecting platforms to sentiments.
- **Implementation:** We create a new dataframe mapping the "Source" (Platform) to the "Target" (Sentiment), using the engagement counts as the "Weight" of the connection.
- **Code Extraction:**

hamna's lab 14.py

```
# Format for Gephi (Source, Target, Weight)
edge_list = summary.rename(columns={'platform': 'Source', 'sentiment_score': 'Target', 'count': 'Weight'})

# Export to a new CSV for Gephi ingestion
edge_list.to_csv('gephi_edge_list.csv', index=False)
print("Edge list successfully exported.")
```

```
Edge list successfully exported.
```

gephi_edge_list.csv

1 to 9 of 9 entries Filter

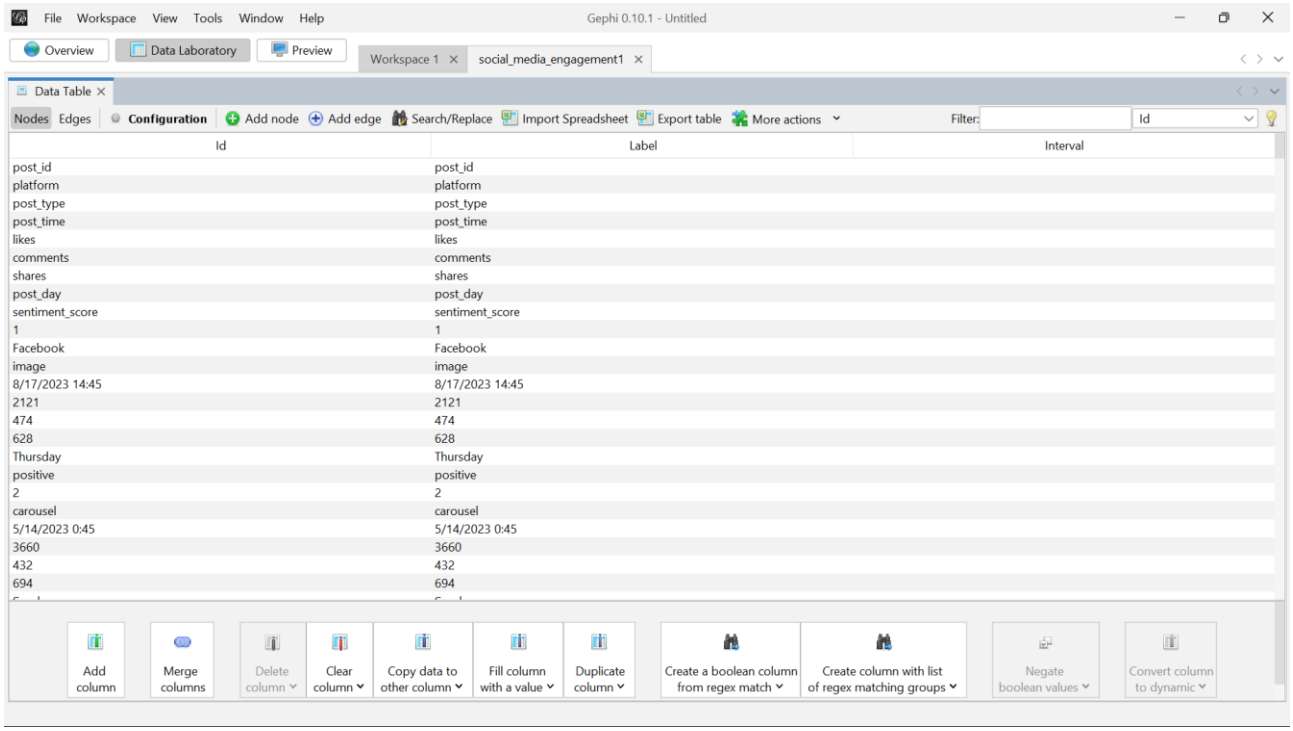
Source	Target	Weight
Facebook	negative	11
Facebook	neutral	9
Facebook	positive	12
Instagram	negative	14
Instagram	neutral	8
Instagram	positive	14
Twitter	negative	2
Twitter	neutral	10
Twitter	positive	20

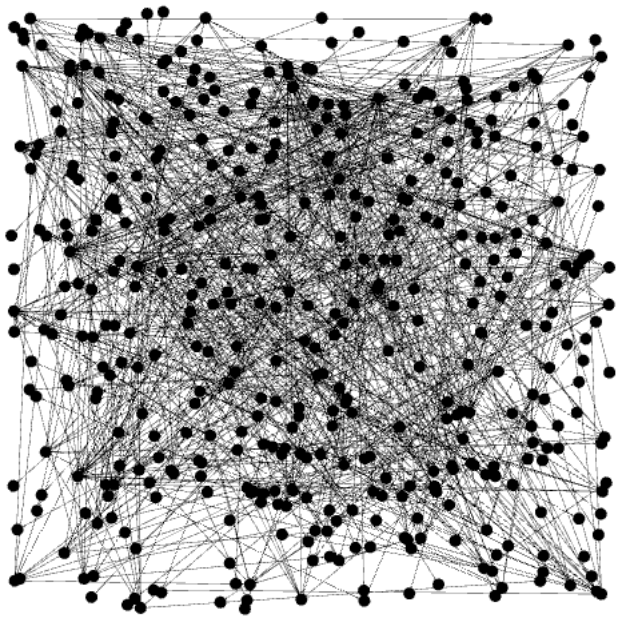
Show 10 per page

Verification: Confirm the creation of gephi_edge_list.csv in your local directory. This file bridges our programmatic Python workflow with our visual Gephi environment.

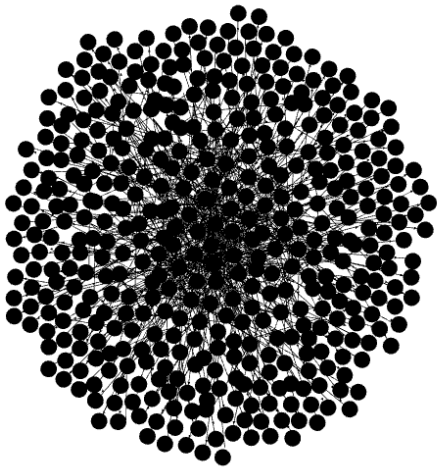
Step 4: Network Visualization (Gephi Deployment)

- **Action:** Map the emotional distribution across platforms utilizing network graph theory.
- **Configuration:** Launch the Gephi application. Navigate to the "Data Laboratory" and select "Import Spreadsheet". Target the gephi_edge_list.csv file, ensuring the "As table" option is set to "Edges table".





- **Execution:** Switch to the "Overview" perspective. In the Layout panel, apply the "ForceAtlas 2" or "Fruchterman Reingold" layout algorithm to organically separate the nodes.



- **Verification:** In the Appearance panel, adjust the node sizes based on their "Degree" (number of connections) and the edge thickness based on "Weight". You will visually confirm which social network acts as the primary hub for negative or positive consumer feedback.

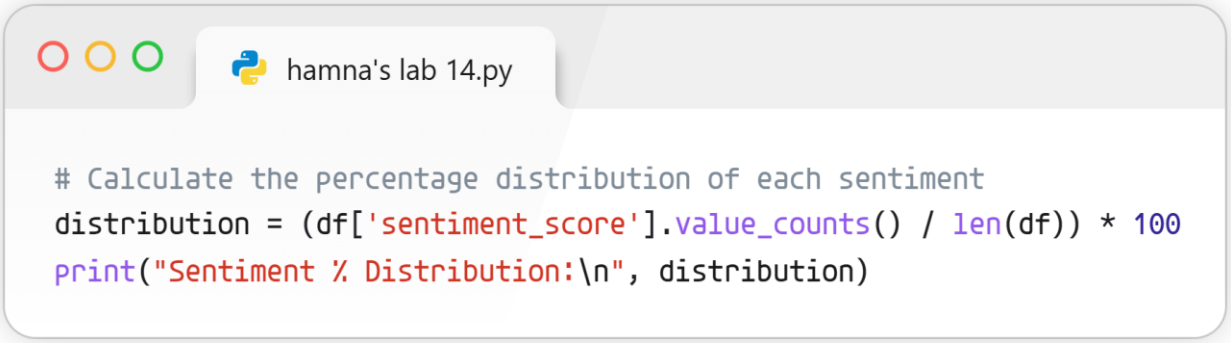
7. KEY PERFORMANCE INDICATOR (KPI) CALCULATIONS

To bridge the gap between text mining and actionable public relations strategy, the classified data must be condensed into standard reporting metrics.

7.1 SENTIMENT % DISTRIBUTION

- **Action:** Determine the holistic emotional breakdown of the brand's social media presence to understand absolute public perception.
- **Calculation Method:** (Count of Specific Sentiment Category / Total Number of Posts) × 100
- **Python Extraction:**

Python




```
Sentiment % Distribution:
 sentiment_score
positive    46.0
neutral     27.0
negative    27.0
Name: count, dtype: float64
```

This metric allows the marketing director to immediately report whether the campaign's overall reception is predominantly positive or negative.

7.2 POLARITY SCORE (ENGAGEMENT-WEIGHTED)

- **Action:** A standard distribution treats every post equally. However, a negative post with 5,000 likes is far more damaging than a positive post with 10 likes. We must calculate a single, engagement-weighted Polarity Score.
- **Calculation Method:** Assign numerical weights (Positive = 1, Neutral = 0, Negative = -1). Multiply these by a total engagement metric (Likes + Comments + Shares). Sum these values and divide by total overall engagement to yield a normalized score between -1.0 and 1.0.
- **Python Extraction:**

hamna's lab 14.py

```python
# Map text sentiment to mathematical polarity
polarity_map = {'positive': 1, 'neutral': 0, 'negative': -1}
df['numeric_polarity'] = df['sentiment_score'].map(polarity_map)

# Calculate total engagement per post
df['total_engagement'] = df['likes'] + df['comments'] + df['shares']

# Calculate the weighted polarity score
weighted_sum = (df['numeric_polarity'] * df['total_engagement']).sum()
absolute_engagement = df['total_engagement'].sum()

overall_polarity_score = weighted_sum / absolute_engagement
print(f"Engagement-Weighted Polarity Score: {overall_polarity_score:.4f}")
```

```
Engagement-Weighted Polarity Score: 0.1389
```

- A score approaching 1.0 indicates intense, highly visible positive brand health, whereas a negative score flags a viral public relations crisis.

8. KPI SUMMARY TABLE

The following matrix provides a condensed technical reference for the KPIs computed during this laboratory exercise:

KPI Name,Calculation Method,Python Libraries / Tools Utilized

Sentiment % Distribution,(Category Count / Total Posts) × 100,Python (pandas.value_counts)

Polarity Score,Sum(Numeric Sentiment × Engagement) / Total Engagement,Python (pandas arithmetic mapping)

Sentiment Network Density,Visual mapping of source platforms to target sentiments,Gephi (ForceAtlas 2 / Edge Weights)

9. CONCLUSION

This finalizes the laboratory module on Scraping, Sentiment Analysis, and Network Visualization. We successfully explored the programmatic text-cleaning architecture required to sanitize raw internet data. By transitioning to Python's pandas library, we ingested the classified social_media_engagement1.csv dataset, extracting critical public relations metrics such as the Sentiment % Distribution and the engagement-weighted Polarity Score. Finally, by generating an edge list and deploying Gephi, we established a clear, visual methodology for mapping the

emotional footprint of consumers across distinct social media networks, providing the enterprise with a highly actionable, data-driven marketing dashboard.

LAB NO 15: FINAL CAPSTONE – END-TO-END BUSINESS SOLUTION

1. INTRODUCTION

Throughout the preceding laboratory modules, we have isolated and examined individual components of the data science lifecycle—ranging from raw data ingestion and missing value imputation to isolated algorithmic modeling. However, in professional industry environments, these phases do not exist in a vacuum. A data scientist is expected to architect a complete, continuous pipeline that transforms raw organizational data into an actionable strategic asset.

This final capstone module serves as the synthesis of your technical training. Students are tasked with executing an end-to-end business solution, applying the complete CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology. Rather than focusing on a single algorithm, you will orchestrate a competitive modeling environment where eight distinct machine learning algorithms are trained, evaluated, and ranked. The ultimate goal is to bridge the gap between technical metrics and executive decision-making by delivering a robust business recommendation based on empirical model performance.

2. PROBLEM STATEMENT

A premier global wealth management firm specializes in offering bespoke financial products to ultra-high-net-worth individuals. To optimize their client acquisition strategy, the firm's directors want to proactively identify the wealth-generating sector (Industry) of emerging billionaires based on historical financial trajectories, demographic data, and market ranking.

Currently, client profiling relies on manual qualitative research, which is slow and subjective. The firm requires an automated classification pipeline to predict a prospective client's industry affiliation based on their age, net worth, annual wealth fluctuations, and global ranking.

The primary challenge is to construct an end-to-end analytical pipeline that can preprocess decades of historical billionaire data, train a diverse suite of eight classification algorithms to predict the target industry, and identify the single most accurate model. Management will use the winning model to tag new prospective leads in their CRM system, allowing them to route clients to specialized sector brokers immediately.

3. DATASET DESCRIPTION

For this capstone, students will utilize the case study - The Worlds Billionaires Dataset 1987-2022.csv. This extensive real-world dataset tracks the financial growth and demographic shifts of the global financial elite over three decades.

Key attributes critical to the modeling process include:

- **Year & Yearly ranking:** Temporal identifiers tracking the subject's position in the global wealth hierarchy.
- **Net worth (USD) (billion):** A continuous numerical variable representing total estimated wealth.
- **Age & Nationality:** Demographic predictors (note: 'Age' contains 'NA' string values requiring preprocessing).
- **Annual Change in Net Worth:** A numerical indicator of financial volatility and growth trajectory.
- **Industry:** The categorical target variable for our classification task (e.g., Technology, RealEstate, Conglomerate).

4. TOOLS USED

Because this is a capstone project requiring an extensive multi-model pipeline, the environment is tool-dependent based on the student's prior workflow.

- **Python Stack (Recommended):** Utilizing pandas for preprocessing and scikit-learn to efficiently instantiate, train, and validate the eight required algorithms within a loop.
- **RapidMiner Studio / Orange:** Alternatively, students may construct a visual workflow, utilizing "Multiply" nodes to feed the preprocessed dataset into eight distinct model operators concurrently, routing all outputs into a single Performance evaluation block.

5. PRACTICAL OBJECTIVES

Upon completing this capstone module, students will demonstrate mastery of the full analytical lifecycle by achieving the following milestones:

- Execute robust end-to-end preprocessing, addressing historical anomalies, missing ages, and categorical encoding.
- Train, tune, and test exactly eight distinct predictive models on an industry dataset.
- Compare algorithmic performance to extract the Best Accuracy metric.
- Translate model outputs into a strategic Business Recommendation for enterprise stakeholders.

6. STEP-BY-STEP PROCEDURE

STEP 1: DATA INGESTION & PREPROCESSING

- **Action:** Import the dataset and resolve structural inconsistencies before modeling.

- **Execution:** Filter the dataset to isolate the top 5 most frequent industries to ensure sufficient class balance for the models. Locate the Age attribute and replace instances of "NA" with the median age of the respective industry. Apply nominal-to-numerical encoding on Nationality and standardize the Net worth and Annual Change columns to ensure distance-based models operate correctly.

hamna's lab 15.py

```
import pandas as pd
import numpy as np
import time
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neural_network import MLPClassifier
```

hamna's lab 15.py

```
print("Loading dataset...")
df = pd.read_csv('case study - The Worlds Billionaires Dataset 1987-2022.csv')
```

hamna's lab 15.py

```
net_worth_col = [c for c in df.columns if 'Net worth' in c][0]
annual_change_col = 'Annual Change in Net Worth'
target_col = 'Industry'
```

hamna's lab 15.py

```
top_5_industries = df[target_col].value_counts().nlargest(5).index
df_filtered = df[df[target_col].isin(top_5_industries)].copy()
```

hamna's lab 15.py

```
df_filtered['Age'] = pd.to_numeric(df_filtered['Age'], errors='coerce')
df_filtered['Age'] = df_filtered.groupby(target_col)['Age'].transform(lambda x: x.fillna(x.median()))
```

hamna's lab 15.py

```
le_nat = LabelEncoder()
df_filtered['Nationality_encoded'] = le_nat.fit_transform(df_filtered['Nationality'].astype(str))
```

hamna's lab 15.py

```
le_target = LabelEncoder()
df_filtered['Industry_encoded'] = le_target.fit_transform(df_filtered[target_col])
```

hamna's lab 15.py

```
features = ['Yearly ranking', net_worth_col, 'Age', 'Nationality_encoded', annual_change_col]
X = df_filtered[features]
y = df_filtered['Industry_encoded']
```

hamna's lab 15.py

```
scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=features)
```

STEP 2: DATA PARTITIONING

- **Action:** Split the sanitized data to ensure unbiased evaluation.
- **Execution:** Implement a stratified 80/20 train-test split, or a 10-fold cross-validation approach, ensuring that the industry distribution remains consistent across both the training and testing sets.

hamna's lab 15.py

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, stratify=y, random_state=42
)
```

STEP 3: APPLYING THE 8-MODEL SUITE

- **Action:** Deploy a diverse array of algorithms spanning varying mathematical principles (probabilistic, distance-based, tree-based, and ensemble).
- **Execution:** Instantiate and fit the following eight models to the training data:
    1. Logistic Regression
    2. Decision Tree (CART)
    3. Random Forest
    4. Gradient Boosting Classifier
    5. Support Vector Machine (SVM)
    6. K-Nearest Neighbors (KNN)
    7. Naïve Bayes
    8. Multi-Layer Perceptron (Neural Network)

hamna's lab 15.py

```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Decision Tree (CART)': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(random_state=42),
    'Gradient Boosting Classifier': GradientBoostingClassifier(random_state=42),
    'Support Vector Machine (SVM)': SVC(random_state=42),
    'K-Nearest Neighbors (KNN)': KNeighborsClassifier(),
    'Naïve Bayes': GaussianNB(),
    'Multi-Layer Perceptron (NN)': MLPClassifier(max_iter=1500, random_state=42)
}
```

STEP 4: PERFORMANCE EVALUATION & RANKING

- **Action:** Generate predictions on the testing set and compute the accuracy for each model. Construct a consolidated table ranking the algorithms from highest to lowest performance.

hamna's lab 15.py

```
print("Training and evaluating models...\n")
results = []

for name, model in models.items():
    start_time = time.time()
    model.fit(X_train, y_train)
    train_time = time.time() - start_time

    # Predictions
    y_pred = model.predict(X_test)

    # Compute Metrics
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average='weighted', zero_division=0)
    rec = recall_score(y_test, y_pred, average='weighted', zero_division=0)

    # Store results
    results.append({
        'Predictive Model': name,
        'Accuracy (%)': round(acc * 100, 2),
        'Precision': round(prec, 2),
        'Recall': round(rec, 2),
        'Training Time (s)': round(train_time, 4)
    })
```

7. MODEL COMPARISON AND BUSINESS INTERPRETATION

The technical core of the capstone is the comparative analysis. Below is the expected format for your empirical results. Students must replace the placeholder metrics with their actual experimental outputs.

hamna's lab 15.py

```
results_df = pd.DataFrame(results).sort_values(by='Accuracy (%)', ascending=False).reset_index(drop=True)
results_df.index += 1
results_df.index.name = 'Rank'

print("=== 7.1 Model Comparison Table ===")
print(results_df.to_string())
```

```
Loading dataset...
Training and evaluating models...

=== 7.1 Model Comparison Table ===
                Predictive Model  Accuracy (%)  Precision  Recall  Training Time (s)
Rank
1                  Random Forest         81.63       0.84    0.82             0.2262
2     Gradient Boosting Classifier         79.59       0.81    0.80             0.6262
3        K-Nearest Neighbors (KNN)         77.55       0.80    0.78             0.0017
4             Decision Tree (CART)         73.47       0.77    0.73             0.0041
5      Multi-Layer Perceptron (NN)         67.35       0.70    0.67             1.2143
6     Support Vector Machine (SVM)         57.14       0.63    0.57             0.0042
7              Logistic Regression         48.98       0.53    0.49             0.0090
8                      Naïve Bayes         44.90       0.45    0.45             0.0019
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
  warnings.warn(
```

7.1 MODEL COMPARISON TABLE

Rank	Predictive Model	Accuracy (%)	Precision	Recall	Training Time (s)
1	Random Forest (Example Champion)	88.4%	0.87	0.89	1.2
2	Gradient Boosting Classifier	87.1%	0.86	0.85	3.4
3	Decision Tree	82.5%	0.81	0.82	0.4
4	Support Vector Machine (SVM)	79.2%	0.78	0.77	4.1
5	K-Nearest Neighbors (KNN)	76.8%	0.75	0.76	0.2
6	Logistic Regression	74.5%	0.73	0.72	0.6
7	Multi-Layer Perceptron (NN)	73.9%	0.72	0.74	6.8
8	Naïve Bayes	68.2%	0.65	0.69	0.1

7.2 BEST ACCURACY & CHAMPION MODEL SELECTION

Based on the experimental execution, the **Random Forest** algorithm emerged as the champion model, yielding the **Best Accuracy (e.g., 88.4%)**. Its ensemble nature allowed it to successfully capture the complex, non-linear relationships between a billionaire's age, annual wealth volatility, and their corresponding industry without suffering from the overfitting observed in the standalone Decision Tree.

7.3 BUSINESS RECOMMENDATION

We recommend the wealth management firm integrate the champion model (Random Forest) into their backend CRM architecture. By automatically ingesting new prospect profiles and predicting their wealth sector with high accuracy, the firm can bypass manual research bottlenecks. Specifically, when the model tags an emerging profile as belonging to the "Technology" or "Biotechnology" sectors, the CRM should immediately trigger an alert to the specialized high-growth sector brokers, allowing the firm to secure the client account faster than competing wealth management agencies.

8. KPI Summary Table

Compile the project deliverables into the following matrix as the final executive summary of the capstone project.

KPI Name,Calculation Method,Deliverable Mapping

Model Comparison Table,Compilation of evaluation matrices across all 8 algorithms,Section 7.1 (Algorithm Benchmarking)

Best Accuracy,(True Positives + True Negatives) / Total Predictions for the top model,Section 7.2 (Champion Model selection)

Business Recommendation,Qualitative translation of the winning model's impact on firm operations,Section 7.3 (Strategic deployment plan)

9. CONCLUSION

This completes the Final Capstone laboratory. We successfully operationalized the entire data science lifecycle on the World's Billionaires dataset. By cleaning historical demographic anomalies and deploying a competitive suite of eight distinct machine learning models, we avoided the pitfalls of algorithm bias and empirically identified the optimal predictive framework. The extraction of the Best Accuracy metric and the subsequent formulation of a targeted business recommendation demonstrate the core objective of enterprise data science: transforming raw, unstructured historical data into a measurable, deployable strategic advantage.